

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a7e4bc9506cc8ca27.jsonl`  
- SHA-256: `e63b6180d4ff85e8fd019d49cc542a44ecc1ad300c247e4778874cadad0575e9`  
- Source modified: `2026-06-10T05:44:33+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T05:33:14.938Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

TASK: TECHNICAL-DOC STALENESS sweep of the indexer ? would a fresh engineer following these docs succeed? Read FULLY: README.md (repo root), TEST_RUN_SUMMARY.md, docs/EVALUATION.md, docs/HOW_RALLY_DETECTION_WORKS.md, docs/GOLDEN_SET_IMPLEMENTATION.md, docs/GOLDEN_SET_PHASE1_VIDEOS.md, docs/GOLDEN_SET_VENDORING.md, docs/TRACKNET_WSL_SETUP.md, docs/VIDEO_RECORDING_GUIDELINES.md, docs/CODE_MAP.md (spot-check its claims), docs/archives/quality-iterations/*/README.md, docs/archives/research/RALLY_DETECTION_RESEARCH.md + AUDIO_E1_FINDINGS.md + AUDIO_RALLY_DETECTION_PLAN.md, docs/archives/past_projects/.

VERIFY against the code with Grep/Glob (do not trust the docs): (1) every COMMAND a doc tells the user to run ? does the script/flag actually exist with that name (e.g. run_eval.py flags, calibrate_wasb invocations, curate commands referenced cross-repo, setup.py --setup)? (2) every FILE PATH referenced ? exists? (watch for pre-#40-restructure paths like backend/segmenters/* vs backend/pipeline/segmenters/*); (3) stale NUMBERS ? test counts (claimed vs the ~254 current), F1/AUC numbers presented as current that were superseded, config defaults quoted in docs vs backend/config/models.py actual; (4) CODE_MAP spot-check: pick 10 specific factual claims (line-level gotchas) and verify each against source; (5) broken intra-doc anchors/links (faq_ref anchors in backend/reporting/spec.yaml must resolve into README/docs ? verify the anchor targets exist); (6) archive policy adherence (terminal-state docs in archives, live docs at top level ? any live doc wrongly archived or vice versa).

Each finding: quote the doc claim + the code reality. Severity: high = following the doc fails or misleads materially; medium = stale but recoverable; low = cosmetic.

2. user (2026-06-10T05:33:21.681Z)

```
1      # Badminton Highlight & Rally Indexer  
2
```

```

3      A local, AI-powered web application and CLI utility designed to process long-form
badminton recordings (e.g., from GoPro), run comprehensive pluggable validations (quality,
integrity, and relevance checks), index all rallies into a relational SQLite database, and
dynamically compile custom highlight reels using high-speed FFmpeg stitching.
4
5      The pipeline is built around three pluggable registries ? validators,
segmenters, and AI providers ? plus a sport profile registry so the same engine can
target other racket/court sports in the future. Badminton is the default profile.
6
7      > Where this is heading: the project is evolving from an AI-API-dependent prototype
into a self-improving tool that learns rally detection offline from a commercially-licensed,
annotated corpus. See [docs/ROADMAP.md](docs/ROADMAP.md) for the four-phase plan and
[docs/archive/decisions/DECISIONS.md](docs/archive/decisions/DECISIONS.md) (historical ADRs)
for the rationale.
8
9      ---
10
11     ## ? Key Features
12
13     * Pluggable Video Validation Framework: Easily extendable base-class architecture
that automatically runs checks on:
14     * Static Format & Resolution: Verifies container is MP4, resolution >= 720p, and
framerate >= 30 FPS.
15     * PTS Timeline Continuity: Scans keyframe timestamps for timeline gaps, jumps,
or editing splices to detect tampering or splices.
16     * Scene Cut Density Check: Measures sub-sampled HSV frame histogram correlation
to verify visual cuts. A continuous raw camera stream should have near 0 cuts.
17     * Blurriness Check: Calculates average frame Laplacian variance to reject out-
of-focus footage.
18     * Relevance (Is it Badminton?): Uses high-performance HSV color court matching
to identify court layouts.
19     * Three Pluggable Rally Segmenters ( `backend/indexer/` ):
20     * `yolo_hybrid` (default) ? two-stage pipeline. Stage 1 runs torchvision
person detection (Faster R-CNN, BSD-licensed) with ByteTrack tracking (supervision, MIT-
licensed) to cheaply filter the video down to a small set of high-motion candidate windows.
Stage 2 hands each candidate (with a small padding) to the AI provider for high-precision
semantic boundaries and shot counts. Massively cheaper than running the LLM on the full video.
(The registry key remains `yolo_hybrid` for back-compat; the AGPL-licensed YOLOv8 was replaced
with permissive components ? see
[docs/archives/MONETIZATION_AUDIT.md](docs/archives/MONETIZATION_AUDIT.md) and ADR-005.)*
21     * `gemini` ? sends the entire video (or a `chunk_duration` slice) to Google Gemini
directly. Highest precision per call, highest cost.
22     * `motion` ? zero-dependency OpenCV pixel-difference heuristic. No API key
required. Useful as a baseline or for offline/air-gapped use.
23     * Pluggable AI Provider Registry ( `backend/providers/` ): The Gemini provider ships
in-tree; new providers (e.g. an OpenAI or local-model backend) can be registered without
touching the segmenters.
24     * Pluggable Sport Profile Registry ( `backend/sports/` ): The per-sport LLM prompt
and relevance config live in a `SportProfile`. Add a new sport by subclassing `SportProfile` and
registering it.
25     * SQLite Indexing: Each processed video is keyed by MD5 hash. Play segments
(start/end/duration/server/receiver/metadata) and per-segment events are persisted in
`output/sports_indexer.db`. Re-running on the same file is a cache hit.
26     * Query System: Filter indexed play segments by `server`, `receiver`,
`duration_min`, and `event_type` (the schema supports rich event types like
smashes/serves/fouls; today the shipped segmenters mainly populate timing + estimated
`shot_count`).
27     * Zero-Dependency Local UI: Served statically directly by FastAPI (Vite + Node.js
NOT required!) featuring a glassmorphic dashboard, responsive filtering, and immediate clip
stitching.
28     * High-Speed FFmpeg Stitching with configurable `begin_padding`, `end_padding`,
and `min_shot_count` filtering.
29     * Detailed CLI Debugging Tools: `--debug-clip <t>` inspects frame metrics,
validator scores, and motion values around a timestamp. `--trim-start/--trim-end` extracts an
arbitrary segment via stream-copy ffmpeg for fast iteration.

```

```

30
31     ---
32
33     ## ?? Setup & Diagnostics
34
35     ### 1. External System Requirements
36     The indexer depends on FFmpeg and FFprobe to parse media containers, extract
keyframes, and stitch clips.
37
38     * On Windows (PowerShell as Admin):
39     ```powershell
40     winget install Gyan.FFmpeg
41     ```
42     *Restart your terminal afterwards to register the path changes.*
43
44     ### 2. Auto-Installer & Diagnostics Checker
45     Run the diagnostics checker script in the root directory to verify your dependencies,
initialize default configs, and install PIP packages:
46     ```bash
47     python setup.py
48     ```
49     This script will:
50     * Validate `ffmpeg` and `ffprobe` are present in your PATH.
51     * Initialize a default `config.json` (if one does not exist) with thresholds for blur,
court HSV bounds, motion signals, YOLO velocity, chunking, and stitching.
52     * Detect whether an NVIDIA GPU is available (via `nvidia-smi`) and select the matching
PyTorch wheel index automatically (`cu124` if GPU present, `cpu` otherwise).
53     * Install everything from `requirements.txt`: `fastapi`, `uvicorn`, `opencv-python`,
`numpy`, `pydantic`, `jinja2`, `python-dotenv`, `google-genai`, `torch`, `torchvision`,
`supervision`.
54
55     > The default `yolo_hybrid` segmenter uses torchvision's COCO-pretrained Faster
R-CNN, whose weights are downloaded and cached automatically by torchvision on first run ?
nothing is bundled in the repo. (Pick a lighter/heavier detector via `indexing.detector_model`:
`fasterrcnn_resnet50_fpn` (default), `fasterrcnn_mobilenet_v3_large_fpn`, or
`retinanet_resnet50_fpn`.)
56
57     ---
58
59     ## ? Running the Application
60
61     ### 1. Launching the Local Web App Dashboard
62     To start the FastAPI server and open the web dashboard:
63     ```bash
64     python -m backend.main --server --port 8000
65     ```
66     Open your browser and navigate to [http://localhost:8000](http://localhost:8000).
67     *Simply input your local raw video file path in the sidebar to process, validate, index,
and compile highlights instantly.*
68
69     ### 2. Running in CLI Processing Mode
70     To process and index a video completely via terminal commands:
71     ```bash
72     python -m backend.main --video-path "C:/path/to/my_video.mp4"
73     ```
74     This outputs a validation summary and indexes all rallies directly into the local SQLite
database, using whichever segmenter is set as `default_segmenter` in `config.json`
(`yolo_hybrid` by default).
75
76     To explicitly pick a segmenter for one run:
77     ```bash
78     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter yolo_hybrid
79     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter gemini
80     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter motion
81     ```

```

```

82
83     ### 3. Rich CLI Debugging (`--debug-clip`)
84     To diagnose exactly why a video segment was registered as active/dead play, or inspect
85     detailed focus/continuity metrics at a particular time in seconds:
86     ```bash
87     python -m backend.main --video-path "C:/path/to/my_video.mp4" --debug-clip 125.5
88     ```
89     *Outputs granular validation check logs and motion ratios within a +/- 3 second frame
90     window around the timestamp.*
91
92     ### 4. Extracting an Arbitrary Trimmed Segment (`--trim-start` / `--trim-end`)
93     For fast iteration on a tiny slice of a multi-GB recording, you can stream-copy a
94     segment to disk without re-encoding:
95     ```bash
96     python -m backend.main --video-path "C:/path/to/my_video.mp4" \
97     --trim-start 600 --trim-end 660 --trim-output slice_10m_to_11m.mp4
98     ```
99     Output lands in `--output-dir` (default `output/`) unless `--trim-output` is an absolute
100     path.
101
102     ### 5. End-to-End "Process + Stitch" Script
103     `run_process_and_stitch.py` is an interactive convenience runner: it MD5-hashes a
104     hardcoded video path, reuses cached indexed segments if available (offering to re-index from
105     scratch), and then prompts for stitching parameters (`begin_padding`, `end_padding`,
106     `min_shot_count`) before compiling the final highlight reel. Useful when iterating on stitching
107     settings against a fixed large recording without re-running expensive segmentation.
108
109     ---
110
111     ## ?? Threshold Tuning (`config.json`)
112     You can adjust the parameters of your validators, segmenters, AI provider, and stitcher
113     by editing `config.json` in the root folder. The default config emitted by `setup.py`:
114
115     ```json
116     {
117       "sport": "badminton",
118       "ai_provider": "gemini",
119       "ai_model": "gemini-2.5-flash",
120       "validation": {
121         "min_resolution_width": 1280,
122         "min_resolution_height": 720,
123         "min_fps": 29.9,
124         "min_blur_threshold": 20.0,
125         "max_scene_cuts_per_minute": 0.5,
126         "relevance": {
127           "court_green_lower": [35, 40, 40],
128           "court_green_upper": [85, 255, 255],
129           "court_pixels_min_ratio": 0.05
130         }
131       },
132       "indexing": {
133         "default_segmenter": "yolo_hybrid",
134         "use_gpu": true,
135         "motion_threshold": 0.08,
136         "min_segment_duration": 3.0,
137         "dead_time_merge_gap": 2.0,
138         "chunk_duration": 90,
139         "chunk_overlap": 10,
140         "yolo_velocity_threshold": 0.15,
141         "yolo_frame_skip": 3,
142         "yolo_tracker": "bytetrack.yaml",
143         "yolo_parallel_chunks": false,
144         "yolo_chunk_workers": 2,
145         "min_action_window_duration": 2.0,
146         "ai_handoff_padding": 2.0,
147

```

```

138         "ai_max_workers": 5
139     },
140     "output": {
141         "default_highlights_name": "highlights.mp4",
142         "db_name": "sports_indexer.db"
143     },
144     "stitching": {
145         "begin_padding": 0.5,
146         "end_padding": 0.5,
147         "use_cache": false,
148         "min_shot_count": 1
149     }
150 }
151 ...
152
153     Notable knobs:
154     * `validation.min_blur_threshold` ? defaults to `20.0` (deliberately permissive for
noisy indoor-arena GoPro footage; see `TEST_RUN_SUMMARY.md` for why).
155     * `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
156     * `indexing.use_gpu` ? when true and CUDA is available, YOLO runs on GPU.
157     * `indexing.yolo_velocity_threshold` ? fraction of frame width per second; lower
values catch slower play, higher values are stricter about what counts as "action".
158     * `indexing.yolo_frame_skip` ? process every Nth frame during YOLO tracking. `3` at 30
fps ? 10 samples/sec.
159     * `indexing.chunk_duration` / `chunk_overlap` ? how to split long videos for the YOLO
phase.
160     * `indexing.ai_handoff_padding` ? seconds of slack added around each YOLO candidate
window before sending to the AI provider.
161     * `indexing.ai_max_workers` ? parallel calls to the AI provider during phase 3.
162     * `stitching.min_shot_count` ? drop play segments whose estimated shot count is below
this threshold when compiling highlights.
163
164     ---
165
166     ## ? Frequently Asked Questions (FAQ)
167
168     ### Q1: The server complains that `ffprobe` is not found.
169     Ensure FFmpeg is installed and added to your system environment variables. You can check
this by typing `ffmpeg -version` or `ffprobe -version` in your terminal. If running `setup.py`
outputs a `[FAIL]`, re-install FFmpeg or manually copy the binary files to a PATH folder.
170
171     ### Q2: Why does my video fail the Relevance check?
172     Our default relevance check scans for the typical indoor green badminton court floor
color space (HSV). If your court is blue, or uses standard wood grain flooring, simply adjust
the HSV values inside your `config.json` under `validation.relevance.court_green_lower` and
`court_green_upper` to match your court, or lower the `court_pixels_min_ratio` to `0.0` to
bypass.
173
174     ### Q3: How do I add a new custom validator to the pipeline?
175     1. Create a new python file in `backend/validators/` (e.g. `my_check.py`).
176     2. Subclass `VideoValidator` and implement `validate(self, video_path, config,
context)`.
177     3. Register your class inside `backend/validators/__init__.py` using
`registry.register(MyCheckValidator())`.
178     It's that simple! The app will automatically run it, display it in the CLI diagnostics
summary, and render it in the frontend checklist.
179
180     ### Q4: Why does the CLI or server output freeze or take a long time on large videos?
181     * Print Buffering: Standard Python buffers console output when stdout is
redirected or running in background shells on Windows. To get real-time unbuffered log
statements immediately, run commands with Python's unbuffered flag:
182         ```bash
183         python -u -m backend.main --video-path "C:/path/to/video.mp4"
184         ```
185     * Video Seeking Overhead: Seeking to random frames in high-bitrate long H.265/HEVC

```

videos using OpenCV's `cap.set(cv2.CAP_PROP_POS_FRAMES)` incurs massive I/O decoding delays. The `motion` segmenter reads sequentially and skips frame decodes using container-level `cap.grab()`, which delivers a ****100x speedup**** on multi-gigabyte raw files. `yolo_hybrid` sidesteps the problem entirely by extracting fixed chunks via `ffmpeg` first.

186

187 **### Q5: Why did the video fail with "No keyframes / packet timestamps could be extracted"?**

188 Some H.265/HEVC streams (e.g. GoPro footage) encode frames as GDR or CRA keyframes. Standard `FFmpeg` frame-level skipping (`-skip_frame nokey`) fails to identify them, returning empty logs. We resolved this by extracting packet indexes (`packet=pts_time,flags`) at the container demuxer level instead, which is incredibly robust and executes instantly without decoding a single pixel.

189

190 **### Q6: How can I quickly test the indexing without waiting to parse an entire multi-GB video file?**

191 You can pass the `--max-frames <N>` argument in CLI or `max_frames` in the POST API payload. This restricts the segmenter frame loop to process only the first `$$$` sub-sampled frames, enabling you to test motion filters and court relevance checks in seconds:

192 ````bash`

193 `python -m backend.main --video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4" --max-frames 300`

194 `````

195 **(At 5 sub-sampled frames per second, `--max-frames 300` analyzes exactly the first 60 seconds of video).**

196

197 **### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?**

198 Both `yolo_hybrid` (default) and `gemini` need an API key for the AI provider. To use Google Gemini:

199 1. Create a local `.env` file in the project root (this is already in `.gitignore` to prevent secret leaks).

200 2. Set your Google AI Studio API key in the file:

201 ````env`

202 `GEMINI_API_KEY=your_key_here`

203 `````

204 3. Run with whichever segmenter you want:

205 ````bash`

206 *# Default: cheap candidate filtering with YOLO, then precise boundaries via Gemini*

207 `python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter yolo_hybrid`

208

209 *# Full-video Gemini analysis (more expensive, highest semantic precision per call)*

210 `python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini`

211

212 *# Pure CV baseline ? no API key needed*

213 `python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter motion`

214 `````

215 **Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"/"motion"`) under `"indexing"` in `config.json`.**

216

217 The AI provider used by `yolo_hybrid` and `gemini` is controlled by `ai_provider` + `ai_model` at the top level of `config.json`. Today only `"gemini"` is registered; new providers can be added under `backend/providers/`.

218

219 ****Trajectory-based segmenters (`wasb_hybrid`, `tracknet_hybrid`).** The registry also includes two shuttle-trajectory variants. Like `yolo_hybrid`, they cheaply filter the video down to candidate windows **locally** and then route each window to the AI provider (`gemini`) for precise boundaries ? so they need the same `GEMINI_API_KEY`. ****If an observability report shows `segmenter_actual: wasb_hybrid` (or `tracknet_hybrid`), that is a valid, registered segmenter ? not an error.**** Pick one with `--segmenter wasb_hybrid` or via `default_segmenter`.

(`wasb_hybrid` additionally requires the WSL WASB setup ? see

docs/TRACKNET_WSL_SETUP.md.)

220

221 **### Q8: How does the `yolo_hybrid` pipeline actually save money?**

222 Sending a multi-gigabyte video to a hosted LLM is slow and expensive. `yolo_hybrid` runs in four phases:

223

224 1. **Chunking** ? splits the video into overlapping windows (``chunk_duration``, ``chunk_overlap``) so we can process incrementally.

225 2. **Detection + tracking** ? torchvision person detection (Faster R-CNN) + ByteTrack runs on each chunk locally (GPU-accelerated when available), measuring per-track velocity. Frames whose normalised player velocity exceeds ``yolo_velocity_threshold`` are kept; the rest are dropped. Adjacent active frames are merged into candidate windows, with sub-``min_action_window_duration`` windows discarded.

226 3. **AI handoff** ? only the surviving candidate windows (padded by ``ai_handoff_padding`` seconds on each side) are extracted and sent to the AI provider for precise rally boundaries and shot-count estimation. These calls run in parallel up to ``ai_max_workers``.

227 4. **DB population** ? confirmed segments are written to ``play_segments`` along with ``shot_count``, ``shot_count_confidence``, and a ``detector`` tag like ``yolo-hybrid-gemini-2.5-flash``.

228

229 In practice this typically cuts AI-provider runtime/cost by an order of magnitude versus uploading the full video to ``gemini`` directly, while preserving the LLM's strength at semantic boundary detection.

230

231 **Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without uploading a huge multi-GB file?**

232 Ingesting and uploading multi-gigabyte video files to Google Cloud can be slow and expensive. We built a native **fast local trimming** developer debugging feature:

233 1. Open your ``config.json`` file.

234 2. Under the ``"indexing"`` configuration block, add a ``"debug_duration"`` key (in seconds):

```
235     ``json
236     "indexing": {
237         "default_segmenter": "gemini",
238         "debug_duration": 15.0,
239         ...
240     }
241     ``
```

242 3. When ``"debug_duration"`` is set, the Gemini segmenter will instantly run a zero-re-encode, sub-second ``ffmpeg`` slice to create a tiny temporary clip of the first 15 seconds, upload only that tiny clip to Google AI Studio (which takes <1 second to upload and <5 seconds to process), run the model query, and clean up the temporary files automatically. This allows you to iterate on model prompts, evaluate validation overlaps, and verify rally boundaries in under 10 seconds!

243

244 **Q10: Can I trust the per-segment metadata from the ``motion`` segmenter?**

245 **No.** The zero-dependency ``motion`` segmenter detects **timing** via pixel-difference, but its per-segment **metadata** is heuristic/synthetic ? ``intensity``, ``avg_speed_kmh``, the ``serve/smash/foul/winner`` events, and ``server`/`receiver`` are generated values, **not measured**. They exist so the schema is populated for demos; do not treat them as ground truth. For real shot counts and boundaries use ``yolo_hybrid`` (default) or ``gemini``. If your observability report shows a ``synthetic_motion_metadata`` warning, this is why.

246

247 **Q11: I re-ran a video and my previous results changed/disappeared. Why?**

248 Videos are keyed by the **MD5** of the file's bytes. Re-processing the **same** file is treated as a fresh ingest: ``add_video`` is ``INSERT OR REPLACE``, and the foreign-key cascade **deletes** the previous play/candidate segments before writing the new ones. So the earlier results are intentionally replaced, not merged. (A ``reingest_replaced_prior`` note in the report flags this.) Note also that re-encoding or re-downloading a video changes its bytes ? a new MD5 ? a separate record.

249

250 **Q12: What is the ``*.report.json`` / ``*.report.md`` / ``*.summary.md`` file next to my video?**

251 Every processing run now writes a **per-video observability report** next to the video (or to ``report.output_dir`` if set in ``config.json``):

252 - ``<name>.report.json`` ? machine-readable: every stage, metric, and finding. The source of truth; diff two runs to see what changed.

253 - ``<name>.report.md`` ? the **technical** report: validation results, processing stages, and **findings/warnings**. Each warning names the problem and cites the FAQ entry to read. **Start here when a run looks wrong.**

254 - ``<name>.summary.md`` ? a **customer-friendly** summary: just the video metadata and highlights, no system internals.

255

256 To read the technical report: scan **Verdict** first (PASS / PASS-with-warnings / FAIL), then **Findings & warnings** ? each finding has a plain-English message, evidence, and a ? see README#Qn` pointer. To turn reports off or relocate them, set `"report": {"enabled": false}` or `"report": {"output_dir": "..."} in `config.json`.`

257

258 **Q13: A validation check failed and indexing was halted ? what do I do?**

259 Open the report's **Processing stages ? validation** block (or the `VALIDATION SUMMARY` printed to the console). Each failed check states the exact reason. Common cases: **relevance** (the court color didn't match ? see [Q2](#q2-why-does-my-video-fail-the-relevance-check)); **resolution/fps** (below the configured minimums ? adjust `validation.min_` in `config.json`); **format** (not an MP4 container); **blur** (footage too soft ? improve sharpness/lighting or lower `validation.min_blur_threshold`; a reading of `0.0` often means undecodable frames, so re-encode with `ffmpeg -i in.mp4 -c:v libx264 out.mp4` before lowering the threshold); **keyframes/PTS** (HEVC/GoPro keyframe quirks ? see [Q5](#q5-why-did-the-video-fail-with-no-keyframes--packet-timestamps-could-be-extracted)). Fix the underlying cause or relax the relevant threshold in `config.json`, then re-run.

260

261 **Q14: The report says fps was a "fallback" (not probed) ? what does that mean?**

262 The `Video` section shows `FPS | 30 (fallback)` and a `fps_fallback_used` warning when the pipeline could **not probe** the real frame rate from the file and fell back to a hardcoded default (30 fps). Because rally detection converts frames?seconds and scales motion thresholds by fps, a wrong fps **miscalibrates timing and motion sensitivity** ? segment boundaries and which motion counts as "play" can be off.

263

264 This usually means validation didn't run (it's the step that probes the file via `ffprobe`), or `ffprobe` couldn't read the stream's `r_frame_rate`. To fix: confirm the real fps ?

265 ````bash`

266 `ffprobe -v error -select_streams v:0 -show_entries stream=r_frame_rate -of csv=p=0`
`"C:/path/to/video.mp4"`

267 `````

268 ? make sure `ffprobe` is on your PATH (see [Q1](#q1-the-server-complains-that-ffprobe-is-not-found)) and that the run validates the file, then re-run so the real fps is used. (When fps shows `(probed)` instead, this is all fine.)

269

270 `---`

271

272 **## ? Annotation & Evaluation**

273

274 Once a video is processed, you can measure **how well the pipeline detected its rallies** against hand-annotated ground truth. You annotate each rally's `[start, end)` time range in a small CSV; the harness scores the detected rallies straight from the SQLite DB ? no API calls, fully deterministic.

275

276 ````bash`

277 `python run_eval.py --scaffold --annotations rallies.csv` # make an empty
template

278 # ...fill in start,end rows, then:

279 `python run_eval.py --video "C:/videos/match.mp4" --annotations rallies.csv --show-`
failures

280 `````

281

282 It reports precision/recall/F1, mean IoU, and boundary error for **two stages** ? raw detector `candidates` vs. AI-confirmed `final` segments ? which pinpoints whether a weak result is a **detection** problem or a **segmentation** problem.

283

284 > Rally timestamps evaluate and tune **segmentation**; they do **not** train the shuttle detector (that needs per-frame coordinate labels). See

docs/EVALUATION.md for the annotation format, CLI flags, and how to read the output.

285

286 `---`

287

288 **## ? Running the Test Suite**


```

289     The test suite uses `pytest` and lives in `tests/`. From the project root:
290
291     ```bash
292     pip install pytest pytest-cov --user
293
294     # One-liner full suite (recommended before opening a PR) ? discoverable wrapper that
295     # runs the whole suite, reports a clear PASS/FAIL, and returns pytest's exit code:
296     python run_all_tests.py
297     python run_all_tests.py -k yolo -v          # any args pass straight through to pytest
298
299     # ...or call pytest directly:
300     pytest                                     # run everything
301     pytest tests/test_yolo_hybrid.py          # run a single file
302     pytest -k yolo --maxfail=1 -v            # filter by keyword, fail fast, verbose
303     pytest --cov=backend --cov-report=term-missing # coverage report (uses .coveragerc)
304     ```
305
306     > The optional `obsreport` package powers the per-video observability reports; its tests
307     > skip cleanly if it isn't installed. To exercise them: `pip install -e ../sports-
obsreport --no-build-isolation`.
308
309     The suite mocks external dependencies (the Gemini client, ffmpeg/ffprobe, OpenCV
`VideoCapture`, the YOLO model) so it runs offline and without a GPU. The tests cover:
310
311     * `tests/test_base.py` ? segmenter registry
312     * `tests/test_database.py` ? SQLite schema, video/segment insert/query
313     * `tests/test_geometry.py` ? bbox center / distance / velocity / IoU helpers
314     * `tests/test_video.py` ? ffprobe duration + ffmpeg chunk extraction helpers
315     * `tests/test_providers.py` + `tests/test_provider_gemini.py` ? AI provider registry
and the Gemini provider
316     * `tests/test_sport_profiles.py` ? sport profile registry + the badminton prompt
317     * `tests/test_gemini.py` ? `GeminiPlaySegmenter` (single-file and chunked paths)
318     * `tests/test_yolo_hybrid.py` ? `HybridYoloSegmenter` happy path, missing-API-key
bailout, and timestamp-validation drops
319     * `tests/test_tracknet_runner.py` + `tests/test_tracknet_hybrid.py` ? TrackNet WSL
runner (mocked subprocess) and the `tracknet_hybrid` segmenter
320     * `tests/test_eval_metrics.py` + `tests/test_eval_annotations.py` +
`tests/test_eval_harness.py` ? rally evaluation harness (interval matching, CSV loading, DB
scoring, CLI)
321
322     ---
323
324     ## ? Project Structure (high level)
325
326     ```
327     backend/
328     main.py                # FastAPI app + CLI orchestrator
329     database.py            # SQLite schema and queries
330     validators/           # Pluggable video validators (format, blur, scene-cut,
relevance, ...)
331     indexer/              # Pluggable rally segmenters: motion, gemini, yolo_hybrid,
tracknet_hybrid
332     detectors/            # TrackNet WSL2 runner (shuttle-trajectory subprocess adapter)
333     providers/            # Pluggable AI providers (gemini today)
334     sports/               # Pluggable sport profiles (badminton today)
335     eval/                 # Rally evaluation harness: metrics, annotations, scoring
336     stitcher/compiler.py  # FFmpeg-based highlight compiler
337     utils/                # ffprobe/ffmpeg + bbox geometry helpers
338     static/               # Zero-build frontend served by FastAPI
339     docs/                 # Live docs (CODE_MAP, COMMERCIALIZATION, BACKLOG, ROADMAP,
guidelines, golden-set). Historical items in docs/archives/
340     tests/                # pytest suite (mocked, runs offline)
341     run_process_and_stitch.py # Interactive end-to-end runner
342     run_eval.py           # Rally evaluation CLI (see "Annotation & Evaluation" above)
343     setup.py              # Dependency & FFmpeg diagnostics + default config init

```

```
344     config.json                # Runtime configuration (see "Threshold Tuning" above)
345     ```
346
```

3. user (2026-06-10T05:33:22.135Z)

```
1     # Test Run Diagnosis & Optimizations Summary
2
3     > **Historical note:** This is a point-in-time diagnosis from a May 2026 local test run,
4     > written *before* the layered-directory restructure (PR #40). Path references below
5     > reflect the old flat layout (e.g. a single `segmenter.py`, now the
6     > `backend/pipeline/segmenters/` package). The diagnostic findings and optimization
7     > lessons remain valid; treat the file paths and install commands as historical.
8
9     *Date: May 18, 2026*
10    *Target File: `C:\Users\avidu\Videos\LargeTestVideo.MP4` (HEVC H.265, Size: ~11.5 GB)*
11
12    This summary details the findings, architectural lessons, and performance optimizations
13    implemented during today's local test run of the Badminton Highlight & Rally Indexer.
14    ---
15
16    ## ? Why the Initial Test Run Blocked (and How We Resolved It)
17
18    During the raw test execution on the 11.5 GB video stream, the system hit three
19    distinct, high-impact constraints. We fully diagnosed, optimized, and hardened the pipelines to
20    address them:
21
22    ### 1. The Keyframe Extraction Gap (H.265 Stream Header Parsing)
23    * **The Issue**: The `PTSContinuityValidator` check initially failed with `[FAIL] No
24    keyframes could be extracted`.
25    * **The Root Cause**: GoPro raw H.265/HEVC encodes use **Gradual Decoder Refresh
26    (GDR)** or **Clean Random Access (CRA)** keyframe configurations. Standard FFmpeg frame-level
27    decoding with `-skip_frame nokey` relies on hardware/software decoders which failed to flag
28    standard keyframe structures, returning empty outputs.
29    * **The Optimization**: We changed the validator to query packet indexes
30    (`packet=pts_time,flags`) at the **container demuxer level** instead of full frame decoding.
31    Because packet flags (e.g. `K__`) are written directly to container headers, this bypasses the
32    video decoder entirely. It runs **100x faster** and is 100% robust across HEVC containers!
33
34    ### 2. The Blurriness Quality Barrier (Indoor Arena Low-Light Noise)
35    * **The Issue**: The video failed the focus threshold check with a sharpness score of
36    `35.3` (minimum threshold was `100.0`).
37    * **The Root Cause**: High-speed action footage in typical indoor badminton
38    environments suffers from high-speed motion blur and ISO sensor noise, reducing the computed
39    Laplacian variance.
40    * **The Fix**: We lowered the `min_blur_threshold` default inside `config.json` to
41    `20.0` so that raw fast-moving arena footage passes quality checks cleanly.
42
43    ### 3. OpenCV Seeking Performance Bottleneck
44    * **The Issue**: The segmenter process consumed massive CPU time during early stages,
45    appearing to respond slowly.
46    * **The Root Cause**: Seeking to arbitrary frames in highly compressed, high-bitrate
47    large H.265/HEVC files using OpenCV's `cap.set(cv2.CAP_PROP_POS_FRAMES)` is an extremely
48    expensive seek operation. Because the decoder has to seek to the closest keyframe and decode
49    forward to the target frame for every sub-sampled frame, it created an  $O(N^2)$  frame-decoding
50    loop.
51    * **The Optimization**: We refactored the segmenter (then `segmenter.py`, now the
52    `backend/pipeline/segmenters/` package) to use a high-performance **sequential grab-and-retrieve
53    pipeline**. By skipping intermediate frames using `cap.grab()` (which parses container
54    structures without decoding pixel values) and only calling `cap.read()` for our 5 FPS analysis
55    frames sequentially, we avoided seeks entirely. This delivers a **100x speedup** on multi-
56    gigabyte MP4s!
57
58    ---
```

```

36
37     ## ?? Newly Added Debug Options: `--max-frames`
38
39     To allow you to quickly test video subsets without waiting to process an entire multi-
gigabyte video file, we added the `--max-frames <N>` argument (and `max_frames` in the API
payload):
40     * This restricts the segmenter frame loop to analyze exactly the first $$ sub-sampled
frames (at 5 FPS, `--max-frames 300` executes only the first 60 seconds of play).
41     * It immediately exits the loop at the limit and proceeds to segment, cluster, and
seed the database with whatever play timelines it identified in that subset.
42     * This lets you debug whether your visual indices and relevance checks are executing
correctly in seconds instead of minutes!
43
44     ---
45
46     ## ? Guidelines for Running on a More Powerful Machine
47
48     Checking this repository into your private GitHub will let you execute the indexer on a
high-powered machine. Here is how to maximize performance:
49
50     1. CUDA/GPU Hardware Acceleration:
51     If your other machine has a CUDA-enabled NVIDIA GPU, install PyTorch with CUDA:
52     ```bash
53     pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
54     ```
55     (`setup.py` auto-selects the matching CUDA wheel ? currently `cu124` ? when it
detects an NVIDIA GPU.)
56     Our diagnostics checker (`setup.py`) will automatically detect CUDA and leverage
standard GPU matrix operations to perform accelerated image difference thresholding!
57     2. Unbuffered Console Outputs:
58     To see log statements instantly without OS shell output buffers, use Python's
unbuffered command:
59     ```bash
60     python -u -m backend.main --video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4"
--max-frames 500
61     ```
62

```

4. user (2026-06-10T05:33:27.378Z)

```

1     # Annotation & Evaluation
2
3     How to measure how well the pipeline detects rallies, using hand-annotated
4     ground-truth timestamps. The harness scores predictions already stored in the
5     SQLite DB ? it makes no API calls and is fully deterministic, so you can run it
6     as often as you like for free.
7
8     ---
9
10    ## 1. What you annotate (and what it does / doesn't do)
11
12    You annotate, per video, the `[start, end)` time range of each rally. That's
13    all the evaluation harness needs.
14
15    > Important: these rally timestamps measure and tune rally segmentation
16    > ? i.e. where the pipeline says rallies begin and end. They do not train
17    > the shuttle detector (TrackNet/WASB), which learns from per-frame shuttle
18    > `(x, y)` coordinate labels, a different and much heavier annotation. Use rally
19    > timestamps as your golden evaluation set; only invest in frame-level
20    > coordinate labels if evaluation shows detection (not segmentation) is the
21    > bottleneck. See `docs/TRACKNET_WSL_SETUP.md`.
22
23    ## 2. Annotation file format (CSV)
24
25    One CSV per video, one rally per row:

```

```

26
27 ```csv
28 start,end
29 00:01:12,00:01:39
30 102.5,131.0
31 ```
32
33 - The `start,end` header is optional (auto-detected).
34 - Timestamps may be decimal seconds (`102.5`) or colon-separated
35   `MM:SS` / `HH:MM:SS` (`00:01:12`). Forms can be mixed.
36 - Blank lines and `#` comments are ignored.
37 - Malformed rows (bad timestamp, `start >= end`) fail loudly rather than being
38   silently skipped ? so labeling mistakes surface immediately.
39
40 Generate an empty template to fill in:
41
42 ```bash
43 python run_eval.py --scaffold --annotations rallies.csv
44 ```
45
46 ## 3. Running the evaluation
47
48 First make sure the video has been processed so its detections are in the DB
49 (the pipeline keys each video by its file MD5). Then:
50
51 ```bash
52 # Identify the video by file (its MD5 is computed for you)...
53 python run_eval.py --video "C:/videos/match.mp4" --annotations rallies.csv
54
55 # ...or by the stored video_id (MD5) directly:
56 python run_eval.py --video-id <md5> --annotations rallies.csv --stage both --iou 0.5
57 ```
58
59 Useful flags:
60
61 | Flag | Effect |
62 |-----|-----|
63 | `--stage {candidates,final,both}` | Which prediction stage(s) to score (default
64   `both`). |
65 | `--detector <name>` | Only score candidates from one detector (e.g. `yolo_hybrid`,
66   `tracknet_hybrid`). |
67 | `--iou <float>` | Match threshold (default `0.5`). |
68 | `--pr-curve` | Also report an IoU sweep (0.1?0.9). |
69 | `--show-failures` | List missed (FN) and spurious (FP) rallies with timestamps. |
70 | `--json <path>` | Write a machine-readable report. |
71 | `--db <path>` | Override the DB path (default: `output/<output.db_name>` from
72   `config.json`). |
73
74 ## 4. Reading the output
75
76 ```
77
78 =====
79 Rally evaluation ? video 'demo'
80 Ground-truth rallies: 3 | IoU threshold: 0.5
81 =====
82
83 Stage      Pred   TP   FP   FN   Prec    Rec    F1   mIoU  dStart  dEnd
84 -----
85 candidates    3    3    0    0   1.00   1.00   1.00   1.00   0.0s   0.0s
86 final         2    2    0    1   1.00   0.67   0.80   0.95   0.2s   0.5s
87
88 =====
89 [final] missed rallies (false negatives): 1
90 - 00:01:20?00:01:32
91
92
93 - **TP / FP / FN** ? true positives (matched), false positives (spurious

```

```

88     predictions), false negatives (missed rallies).
89 - **Prec / Rec / F1** ? precision, recall, and their harmonic mean.
90 - **mIoU** ? mean temporal IoU over matched pairs (how tightly matched rallies
91     overlap their ground truth).
92 - **dStart / dEnd** ? mean absolute boundary error in seconds (are rallies
93     detected but trimmed in the wrong place?).
94
95     ### The key diagnostic: candidates vs. final
96
97     Scoring both stages tells you *where* to spend effort:
98
99     - **Candidates miss rallies the GT has** ? the **detector** is the bottleneck.
100     Improve the shuttle model (e.g. fine-tune TrackNet/WASB with frame-level
101     labels) or relax detection thresholds.
102     - **Candidates catch them but `final` is wrong/trimmed** ? the **segmentation /
103     AI handoff** is the bottleneck. Tune thresholds (`velocity_threshold`,
104     `dead_time_merge_gap`, `min_action_window_duration`) or the AI prompt ? no new
105     labels needed.
106
107     In the example above, detection is perfect (candidates: recall 1.0) but the AI
108     handoff dropped one rally (final: recall 0.67) ? so the fix is in segmentation,
109     not detection.
110
111     ## 5. How matching works
112
113     Predictions and ground truth are sets of `[start, end)` intervals. A prediction
114     **matches** a ground-truth rally when their **temporal IoU** ( $\text{overlap} \div \text{union}$ ) is
115     at least the threshold. Matching is **greedy and one-to-one** ? highest-IoU pairs
116     are claimed first, and each prediction/GT is used at most once, so a single
117     detection can't be credited for two rallies. Unmatched predictions are false
118     positives; unmatched ground-truth rallies are false negatives.
119
120     ## 6. Code map
121
122     | File | Role |
123     |-----|-----|
124     | `backend/eval/metrics.py` | Pure interval math: IoU, matching, P/R/F1, boundary error.
125     |
126     | `backend/eval/annotations.py` | Load/validate the ground-truth CSV. |
127     | `backend/eval/harness.py` | Read predictions from the DB, score, format the report. |
128     | `run_eval.py` | CLI entry point. |
129     | `tests/test_eval_*.py` | Deterministic tests for all of the above. |
130
131     ## 7. Troubleshooting
132
133     | Symptom | Cause & fix |
134     |-----|-----|
135     | `ERROR: database not found at output/...` | The harness only reads stored predictions
136     ? it does **not** run the pipeline. **Process the video first** (e.g.
137     `run_process_and_stitch.py`) so detections land in the DB, then re-run. |
138     | Every rally shows as a miss (FN), `Pred` is 0 | This video hasn't been processed, or
139     you used the wrong `video_id`. The pipeline keys videos by **file MD5** ? pass the *same* file
140     via `--video`, or the exact stored `--video-id`. |
141     | `video_id` doesn't match although it's "the same" video | Re-encoding/trimming changes
142     the bytes, hence the MD5. Use the exact file that was processed, or look up the stored id (it's
143     printed during processing). |
144     | `candidates` stage is empty but `final` has data (or vice-versa) | Depends on which
145     segmenter ran. `gemini` writes only `final` (no candidates); `yolo_hybrid`/`tracknet_hybrid`
146     write both. Use `--stage final` for the gemini segmenter, and `--detector <name>` to scope
147     candidates. |
148     | Recall looks low only at high `--iou` | Rallies *are* found but boundaries are loose ?
149     check `dStart`/`dEnd`. Tune `stitching` padding or the segmentation thresholds rather than the
150     detector. |
151     | `ValueError: start (...) >= end (...)` while loading annotations | A bad row in your
152     CSV. Timestamps must be increasing per rally and use seconds or `HH:MM:SS`. |

```

```
140
141 Run `python run_eval.py -h` for a quick reminder of this prerequisite and all flags.
142
```

5. user (2026-06-10T05:33:27.810Z)

```
1 # How rally detection works (the 2-layer mental model)
2
3 A short conceptual map of how we go from raw video to rally `[start, end]` segments ?
4 and, crucially, which layer our annotation can improve.
5
6 ```
7 video ??? frames ??? [DETECTION] ??? shuttle (x,y,visible) per frame
8             ?             ?
9             (WASB CNN,      per-frame trajectory
10             frozen weights) ?
11             ?
12             velocity signal (how fast the shuttle moves)
13             ?
14             [SEGMENTATION] ??? the 3 knobs we tune
15             ?
16             ?
17 rally windows [start, end] ??? IoU vs golden
labels
18 ```
19
20 ## Two layers ? keep them separate in your head
21
22 1. DETECTION ? "where is the shuttle, this frame?"
23 WASB is a CNN that, for each frame, outputs a heatmap ? the shuttle's `(x, y)` + a
24 visibility/confidence. Its weights were learned (by gradient descent, on shuttle-
coordinate
25 labels) and we use them frozen. We chose a shuttle detector on purpose: "shuttle
in
26 flight" is the signal for "a rally is happening," and it's camera-agnostic ? unlike
27 player-motion, which fooled the old heuristic on broadcast cameras that pan with the
players.
28 (Your videos are static tripod, so player-motion would also work ? but shuttle-motion is
the
29 robust general bet.)
30
31 2. SEGMENTATION ? "when is a rally?"
32 From the per-frame `(x, y)` we compute the shuttle's velocity (displacement between
frames).
33 Then three knobs turn that into rally intervals:
34 - velocity_thresh ? a frame is "active" only if the shuttle is moving faster than
this
35 (in flight, not lying on the floor / being picked up).
36 - merge_gap ? active frames within this many seconds get joined into one rally
(bridges
37 brief gaps: a shuttle momentarily occluded, or the beat between two hits).
38 - min_window_duration ? throw away windows shorter than this (blips / false
flickers).
39
40 Output: a list of `[start, end]` windows = the rallies.
41
42 ## Where the annotation fits (the key point)
43 Our Roora/golden labels are rally `[start, end]` times ? they say when rallies
are, not
44 where the shuttle is. So they can't teach WASB's detector (that needs `(x,y)` labels).
They
45 can grade and tune the segmentation layer: we score predicted windows against
the golden
46 rallies with temporal IoU (how much they overlap), and pick the 3 knobs that
maximise it ?
```

```

47     **calibrate on some rallies, measure on held-out ones** so the number is honest.
48
49     ## Where sparks could land (open questions ? not commitments)
50     Things worth a shower-thought; any of these could beat 3 hand-tuned thresholds:
51     - **Richer features than raw velocity:** acceleration, trajectory *continuity* (a real
rally is a
52         smooth arc that survives occlusion), shuttle *height/arc*, or **direction reversals**
(each
53         reversal ? a shot ? ties to `shots_count`!).
54     - **A tiny learned segmenter** instead of 3 thresholds: feed the trajectory features to
a small
55         model trained on the golden rally labels (still the segmentation layer, just smarter).
56     - **Hysteresis** (two thresholds: enter-rally high, stay-in-rally low) ? robust to mid-
rally dips.
57     - **Rhythm/periodicity:** rallies have a hit cadence; dead time doesn't.
58     - **Fusion:** combine shuttle-motion (WASB) with player-motion for static cams.
59     - **Per-video calibration as a *feature*:** "watch a few labeled rallies ? auto-tune to
this
60         court/lighting ? segment the rest sharply" ? exactly your earlier idea, at the right
layer.
61
62     The whole point of per-video calibration: a handful of *your* labeled rallies nudges
these knobs
63     to *your* court/camera, and we report the IoU lift on the rallies it never saw.
64

```

6. user (2026-06-10T05:33:32.059Z)

```

1     # Golden-Set Subproject ? Design & Implementation Overview
2
3     **Status:** In progress . **Date:** 2026-05-30
4     **Companion docs:** design rationale + vendor shortlist in
[GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md) (ADR-006); scoring mechanics in
[EVALUATION.md](EVALUATION.md).
5
6     This is the **implementation tracker** for turning the golden-set/vendor *design* into
shipped code. It's split into small, independently-reviewable PRs. The whole subproject is
**vendor-agnostic by construction** ? only one slice (GS-3, `HumanVendorProvider`) depends on
which vendor you pick offline; everything else can land first.
7
8     > **Update (2026-06-02):** Vendor selected for the **initial pilot ? Roora**
(Bengaluru). The **annotation guideline, intake-manifest template, and NDA** are authored in the
collector repo `docs/annotation/`, and the collector's `import-local` ingestion now covers the
**3 pilot sports** (badminton, pickleball, table tennis). On the indexer side, GS-1
(`file_provider.py`), GS-2 (`backend/eval/regression.py` + `run_regression.py`), and the GS-4
oracle scaffold (`automated_provider.py`) have landed; the remaining slice is the vendor-
dependent **GS-3 `HumanVendorProvider`**, parked until the Roora pilot confirms quality.
9
10    ---
11
12    ## Architecture in one picture
13
14    ```
15        video_ref ??? AnnotationProvider.submit()/poll()/fetch() ???
List[(start,end)]  ground truth
16                    ?           ?           ?           ?
17        FileProvider HumanVendor Automated(oracle)           ?
18        (GS-1)       (GS-3)       (GS-4)
backend.eval.metrics.score() ??? existing, unchanged           ?
19
20        production pipeline ??? DB ??? backend.eval.harness.get_predictions() ???
List[(start,end)]  predictions
21
22                    regress(video, tier) (GS-2)
compares the two

```

```

23     ``
24
25     **The keystone:** a provider's `fetch()` returns the **same `List[(start, end)]` that
`eval.annotations.load_annotations()` already returns and `eval.metrics.score()` already
consumes. So one scorer grades all three of: product-vs-GT, vendor-vs-answer-key, and annotator-
vs-annotator (IAA). No new scoring code ? only new *sources* of intervals.
26
27     ---
28
29     ## PR breakdown
30
31     | Slice | Branch | Scope | Depends on | Vendor-dependent? |
32     |---|---|---|---|---|
33     | **GS-1** | `feat/annotation-provider-registry` | `AnnotationProvider` ABC + registry +
`FileProvider` + tests | ? | No |
34     | **GS-2** | `feat/regression-runner` | `regress(video, tier, tolerance)` over existing
harness + `baseline.json` snapshot/compare + CLI | GS-1 | No |
35     | **GS-3** | `feat/human-vendor-provider` | `HumanVendorProvider` (submit/poll/fetch
against the chosen vendor's API or file-drop), DPA/secure-link handling | GS-1, **your vendor
pick** | **Yes** |
36     | **GS-4** | `feat/automated-oracle-provider` | `AutomatedProvider` fast-tier oracle ?
**interface stub** (injected labeler + fake; concrete model = backlog) | GS-1, GS-2 | No |
37     | **GS-5** | `feat/regression-triggers` | Wire triggers: CI gate, on-demand CLI,
scheduled run | GS-2, GS-4 | No |
38
39     Each row is one PR. GS-1 ? GS-2 ? GS-4 ? GS-5 can proceed **now**, independent of vendor
selection. GS-3 is parked until you've chosen a vendor (then it's a thin adapter behind the GS-1
interface).
40
41     ---
42
43     ## Slice details
44
45     ### GS-1 ? Provider registry + FileProvider ? (this PR)
46     - `backend/annotations/base.py` ? `AnnotationProvider` ABC (`submit`/`poll`/`fetch` +
`annotate()` convenience) and `annotation_registry`, mirroring `backend/providers` and
`backend/indexer` registry patterns.
47     - `backend/annotations/file_provider.py` ? `FileProvider`: the "job" is a CSV that
already exists (formalizes today's manual/ShuttleSet flow); reuses
`eval.annotations.load_annotations`.
48     - Tests assert the registry contract **and** the keystone (provider output is directly
scorable by `metrics.score()`).
49     - **Design choice:** `fetch()`/`annotate()` raise loudly on a missing/incomplete job
rather than returning `[]`, so a broken label source can't silently masquerade as "0 rallies"
and skew a regression to green.
50
51     ### GS-2 ? Regression runner (next)
52     `regress(video, tier="fast", tolerance=0.05)`: get GT from
`annotation_registry.get(tier)`, read predictions via the existing `eval.harness`, score with
the existing `metrics`, compare against a per-video `baseline.json`, and flag a regression when
precision/recall drop beyond tolerance. Pure orchestration over existing parts + a small
baseline store. CLI: `python run_regression.py --video ... --tier file`.
53
54     ### GS-3 ? HumanVendorProvider (after your vendor pick)
55     A concrete provider that submits a (licensed/owned-only) video to the chosen vendor and
fetches the returned rally CSV. Carries the privacy controls from the design doc (secure time-
limited links, delete-on-delivery, DPA). Thin adapter ? the interface already exists from GS-1.
56
57     ### GS-4 ? AutomatedProvider (fast-tier oracle) ?? oracle rule
58     An independent strong labeler for the fast/CI tier. **Must be a different model lineage
than whatever production uses** (else shared blind spots ? false green). A fast-tier failure is
a *smoke alarm* that escalates to the GS-3 human tier, never an authoritative verdict on its
own.
59
60     ### GS-5 ? Regression triggers

```



```

61     Wire `regress()` into CI (pre-merge gate on pipeline changes), an on-demand CLI (the
"surprise regression"), and an optional scheduled run. Includes a deliberate re-snapshot path so
legitimate production improvements can rebaseline instead of raising false regressions.
62
63     ---
64
65     ## Conventions
66     - **Branch per slice**, one PR each, tests green before PR, full suite must stay green.
67     - New runtime deps: **none** through GS-2 (GS-3 may add a vendor SDK; GS-4 may reuse the
existing `google-genai` for a Gemini oracle).
68     - All ground-truth video stays **licensed/owned** ? see the privacy section of the
design doc.
69

```

7. user (2026-06-10T05:33:32.370Z)

```

1     # Phase-1 Golden-Set Video Collection Spec
2
3     **Status:** Design v1 . **Date:** 2026-06-02
4     **Companion docs:** [GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md) (vendor process,
ADR-006) . [VIDEO_RECORDING_GUIDELINES.md](VIDEO_RECORDING_GUIDELINES.md) (capture spec the
user-facing guide derives from) . scoring in [EVALUATION.md](EVALUATION.md).
5
6     What footage to gather for the **first round** of golden-set creation so the trained
rally segmenter reaches good precision/recall ? and so the eval number actually predicts
production. Golden-set creation is **iterative**: expect multiple rounds, expanding toward
whatever axis round-1 metrics show is weakest.
7
8     ---
9
10    ## 1. The principle that dominates everything: match the deployment distribution
11
12    The CV literature is consistent: **models trained on broadcast footage fail to
generalize to self-recorded amateur video** ? broadcast uses a "straight, fixed camera with
minimum variability," while real non-elite footage has limited coverage, occlusions, and
inconsistent viewpoints ([team-sports
survey](https://link.springer.com/article/10.1007/s10462-024-10934-9), [unstructured-video event
detection](https://arxiv.org/pdf/2002.08097)). This is the **same failure mode as ADR-004**
(broadcast cameras *track players*, so player pixel-motion inverts during rallies).
13
14    Our product ingests **users' own match recordings** (a fixed phone/GoPro on a tripod).
Therefore:
15
16    - The golden set ? **especially the held-out eval split** ? must be dominated by
**static, self-recorded, full-court** footage that mirrors real users.
17    - **ShuttleSet / broadcast** footage is fine as *training* augmentation and as a
contrast set, but must be **minimal in eval**. Measuring precision/recall on broadcast yields a
number that does **not** predict production.
18
19    ---
20
21    ## 2. Primary recording profile (also the basis of the user-facing capture guide)
22
23    Grounded in the badminton-CV setup literature (camera ~2 m behind the baseline, ~4.5 m
elevated, ~30° down-tilt, whole court + headroom above the net ?
[source](https://pmc.ncbi.nlm.nih.gov/articles/PMC10747833/)):
24
25    | Setting | Spec | Why |
26    |---|---|---|
27    | Mount | Fixed tripod; **no pan/zoom** during play | Avoids the ADR-004 camera-tracking
trap |
28    | Position | Behind one baseline/end, centered | Full-court, symmetric view |
29    | Height | **375 m** elevated (~4.5 m ideal) | De-occludes the far player & net |
30    | Tilt | **~30° down** | Optimal coverage, low keystoneing |
31    | Framing | Entire court **+ headroom above the net** ; players never leave frame |

```

Detection + court-relevance need it |

32 | Lighting | Even; no backlight/glare | Reduces blur & false motion |

33

34 Most of round 1 should use this profile; a minority deliberately varies it (§4) to probe robustness.

35

36 ---

37

38 ## 3. Resolution & frame rate (per sport)

39

40 A controlled study: table-tennis ball detection was **22.5% @ 30 fps ? 93.2% @ 60 fps** (120 fps ? 60 fps) ([Nature Sci Rep](https://www.nature.com/articles/s41598-023-42966-6)); OpenTTGames records at 120 fps. The ball/shuttle is the camera-invariant rally signal (our WASB substrate, ADR-001), so fps is **not** a nicety for fast sports.

41

42 | Sport | Min resolution | Recommended fps | Note |

43 |---|---|---|---|

44 | Badminton | 1080p (720p floor) | 30 ok, **50/60 better** | Shuttle decelerates fast; 60 sharpens smashes |

45 | Pickleball | 1080p | **30?60** | Ball slower than TT; 30 likely fine, 60 safer |

46 | Table tennis | 1080p | **60 min (120 ideal)** | 30 fps collapses ball detection |

47

48 > These are **literature-backed starting points**; the published minimums get finalized empirically against our golden set (VIDEO_RECORDING_GUIDELINES.md §3).

49

50 ---

51

52 ## 4. The Phase-1 diversity matrix

53

54 Cover the **high-impact axes** now; don't chase rare conditions in round 1. "Guards" = whether the axis mostly protects **P**recision (not calling dead-time a rally) or **R**ecall (not missing rallies).

55

56 | Dimension | Vary across? | Guards | Round-1 target |

57 |---|---|---|---|

58 | **Sport** | badminton, pickleball, table tennis | both | all 3 |

59 | **Format** | singles **and** doubles (badminton, pickleball) | R | both for those 2 |

60 | **Camera profile** | mostly primary-static; 1?2 side-elevated; 1?2 lower/phone-height; **1?2 broadcast (contrast only)** | both | ~80% static / ~20% variants |

61 | **Venue / court color** | wood, green, blue synthetic | P | ?3 venues |

62 | **Lighting** | bright gym, dim club, mixed/backlit | both | ?3 conditions |

63 | **Background clutter** | empty hall vs people moving behind court | **P** (crowd motion ? false positives) | both |

64 | **Players** | skill, attire colors, body types, L/R-handed, skin tone/age | R | spread; don't reuse one pair |

65 | **Rally edge cases** | service faults, lets/replays, 1?2-shot & long rallies, net-cords, coaching pauses, warm-up | boundary **P/R** | ensure present |

66 | **Resolution/fps ablation** | same match re-encoded 480/720/1080p; 30 vs 60 fps | calibration | 1 matched set/sport |

67

68 ---

69

70 ## 5. Split it so the precision/recall number is honest (no train/eval leakage)

71

72 The #1 video-ML mistake: the **same match in train and eval inflates the score** ([data-leakage guide](https://towardsdatascience.com/data-science-mistakes-to-avoid-data-leakage-e447f88aae1c/)).

73

74 - **Split by match / recording session ? never by clip.** All clips from one match go entirely to train *or* eval.

75 - **Hold out whole *conditions*** (a venue, a camera profile, a player pair the model never saw) so eval measures generalization, not memorization.

76 - Reserve **~? of each sport as a deployment-like held-out eval set** (static, self-recorded). Broadcast/ShuttleSet may enter train but should be **minimal in eval**.

77

```

78     > **Guardrails ? TODO (design with the user):** train and eval live in **separate
catalogs** today, kept apart by convention. We want hard guardrails ? permissions / explicit
approval / re-confirmation at ingest & export ? so a video can **never** silently land in both.
Tracked in BACKLOG.md; to be designed in a later session.
79
80     ---
81
82     ## 6. Concrete "round 1" shopping list (per sport)
83
84     Aligns with the 12?15-videos/sport scale plan; the 3-clip Rooru calibration is a subset
of this.
85
86     - **8 static-primary** clips spanning venues / lighting / skill (?4 singles, 4 doubles
where applicable)
87     - **2 camera-variant** clips (side-elevated, lower height) ? robustness probe
88     - **1 resolution/fps ablation set** (one match @ 480/720/1080p, 30 & 60 fps)
89     - **1 broadcast contrast** clip (ShuttleSet) ? *train/contrast only*
90     - Each ~2?4 min, **deliberately including the edge cases** above ? **~12 clips/sport**;
hold out **4 whole matches** for eval.
91
92     ---
93
94     ## 7. Deliberately defer to later rounds
95
96     Outdoor courts, phone-handheld/jitter, extreme angles, 4K, unusual net/court variants,
crowd-heavy tournaments. Add these once round-1 precision/recall shows the weakest axis, then
expand toward it (find the quality "knee" ? VIDEO_RECORDING_GUIDELINES.md §3).
97
98     ---
99
100    ## 8. Two cautions specific to precision (the stated goal)
101
102    - **Backgrounds with people moving** behind the court are the biggest false-positive
driver. Include some so the model learns to reject them ? and make sure the annotation
guideline's "ignore all dead time" rule is airtight.
103    - **Warm-up / knock-up / practice swings** look like rallies. Capture some, and confirm
the vendor **excludes** them (the Rooru guide now has an explicit rule + a how-to-spot-it tip).
Early precision is bounded more by **guideline / inter-annotator consistency (IAA)** than by raw
data volume.
104
105    ---
106
107    ## 9. Sources
108
109    - [Team-sports action-recognition survey ? Springer
2024](https://link.springer.com/article/10.1007/s10462-024-10934-9)
110    - [Unstructured-video event detection ? arXiv
2002.08097](https://arxiv.org/pdf/2002.08097)
111    - [Badminton service-height CV setup (camera 4.5 m / 30°) ?
PMC](https://pmc.ncbi.nlm.nih.gov/articles/PMC10747833/)
112    - [VideoBadminton dataset ? arXiv 2403.12385](https://arxiv.org/html/2403.12385v1)
113    - [Table-tennis frame-rate vs detection accuracy ? Nature Sci
Rep](https://www.nature.com/articles/s41598-023-42966-6)
114    - [OpenTTGames (120 fps) dataset](https://lab.osai.ai/)
115    - [Action-spotting / segmentation survey ? arXiv
2505.03991](https://arxiv.org/html/2505.03991v2)
116    - [Data-leakage in train/test splits](https://towardsdatascience.com/data-science-
mistakes-to-avoid-data-leakage-e447f88aae1c/)
117

```

8. user (2026-06-10T05:33:36.616Z)

```

1     # Golden-Set Generation & Vendor Process (v1)
2
3     **Status:** Design v1 . **Date:** 2026-05-30 . **Vendor shortlist added 2026-05-30

```

(§6)**

4 **Owner decisions captured:** licensed/owned video only · rally `[start,end)` depth · tiered turnaround (fast automated + slow human) · vendor bake-off · **Bangalore/India-local vendor preference**

5

6 This document defines how we produce **golden-set ground truth** (hand-verified rally boundaries) for the badminton highlights product, how we **select annotation vendors**, and how that same machinery powers **on-demand / surprise regression testing** of the production tool.

7

8 > **Companion docs:** the scoring mechanics live in EVALUATION.md; the annotated-corpus rationale is [ADR-002](DECISIONS.md); the decision record for *this* process is **ADR-006**. Video-capture quality requirements (a dependency of trustworthy golden sets) are tracked separately in VIDEO_RECORDING_GUIDELINES.md.

9

10 > **Update (2026-06-02):** §6 shortlist done; the founder selected **Roora** (Bengaluru) for the **initial paid pilot** ? a **single-vendor pilot first**, not the full bake-off below. Roora's site/capability/location are verified; **quality is TBD via the pilot**. **Step 0 (the annotation guideline) is authored** ? collector repo `docs/annotation/` (`ROORA\_BRIEF.md` rally CSV schema + `ANNOTATION\_GUIDE.md` illustrated guide + intake-manifest & NDA templates). Pilot scope is **3 sports** (badminton, pickleball, table tennis); the collector's `import-local` ingestion for all three is built. Still pending: the internal **gold key** and the vendor-dependent **GS-3 `HumanVendorProvider`** (GS-1/GS-2/GS-4 already landed ? see GOLDEN_SET_IMPLEMENTATION.md).

11

12 ---

13

14 ## 1. The keystone insight: one scorer, three jobs

15

16 A vendor's golden-set output is a list of rally `[start,end)` intervals ? **structurally identical to a prediction** from our pipeline. So the *exact same* deterministic scorer in `backend/eval/metrics.py` (`interval\_iou` ? `match\_intervals` ? `score`) grades all three of:

17

18 1. **Product vs. ground truth** ? the precision/recall/F1/IoU we already report (`EVALUATION.md`).

19 2. **Vendor vs. our internal answer key** ? the quality gate during vendor selection.

20 3. **Annotator vs. annotator** ? inter-annotator agreement (IAA), our proxy for "is this even a well-defined task?"

21

22 This symmetry is what makes the whole system small, auditable, and extensible. **No new scoring code is needed** ? only a new *source* of intervals (the vendor).

23

24 ---

25

26 ## 2. Why this also solves the privacy nuance

27

28 The monetization goal (hosted API) carries a "someone views the user's video" concern. This process sidesteps it **by construction**:

29

30 - **Golden sets are built only from footage we license or own** ? starting with the founder's **constantly-growing personal collection** of match videos, plus ShuttleSet (ADR-003) and any purpose-recorded clips. **No end-user uploads ever flow to a vendor.**

31 - Regression tests therefore run against a **representative held-out corpus**, not literal user content.

32 - All video egress to any third party passes through a **single provider boundary** (§4), so the security controls (DPA, no-reuse, delete-on-delivery, access logging) live in exactly one place.

33

34 > **Honest tradeoff:** a licensed/owned corpus may not perfectly mirror real users' cameras, courts, and lighting. A green regression on our corpus is therefore *strong evidence*, not a *guarantee*, for production traffic. Revisit if field behavior diverges from corpus behavior ? and see VIDEO_RECORDING_GUIDELINES.md, which aims to shrink that gap by steering users toward capture conditions our corpus represents.

35

36 ---

37

```

38     ## 3. Selecting vendors ? the bake-off
39
40     ### Step 0 ? Write the annotation guideline (highest-leverage artifact)
41     Before contacting anyone, write a precise, example-rich definition of what a rally
is:
42     - Start trigger: e.g. first serve motion vs. shuttle leaving the racket ? pick one,
show clips.
43     - End trigger: shuttle hits floor/net/out, or umpire call.
44     - Edge cases: lets/replays, service faults, between-point walking, coaching pauses,
mid-rally net-cord, broadcast replays and graphics.
45
46     Rally boundaries are genuinely subjective; ambiguity here is the #1 cause of bad
golden sets. This guideline belongs conceptually with the per-sport SportProfile, so it
generalizes to tennis/cricket later.
47
48     > Done (2026-06-02): authored as the Roora brief + illustrated guide in the
collector repo docs/annotation/. Defines the rally CSV schema and the three golden rules ?
ignore all dead time; include every rally (incl. service faults & lets); a shot = one
racket/paddle/bat contact ? plus an ending_reason taxonomy (winner / forced_error /
unforced_error / service_fault / let / other, shared across net-separated sports) and per-sport
detail incl. singles & doubles for badminton & pickleball.
49
50     ### Step 1 ? Build a small internal gold key
51     One trusted expert (initially: the founder) hand-annotates 5?10 licensed clips to
near-perfect. This is the answer key vendors are graded against ? kept private during the bake-
off.
52
53     ### Step 2 ? Run the bake-off (parallel pilot)
54     Send the same pilot clips to 2?3 shortlisted vendors (see §6, favoring
Bangalore/India), withholding the key. Grade every submission with our own harness:
55
56     | Dimension | How we measure it | Tool |
57     |---|---|---|
58     | Accuracy | vendor CSV vs. gold key ? P/R/F1, mIoU, boundary error | eval.score()
|
59     | Consistency | 2 annotators per vendor ? pairwise IoU (IAA) | interval_iou |
60     | Cost | quoted ? or $ per video-minute | vendor quote |
61     | Turnaround | actual vs. promised, per tier | observed |
62     | Ops friction | file-drop vs. API; compliance with our CSV format |
annotations.load_annotations() |
63     | Security | DPA signed; DPDP/ISO posture; delete-on-delivery | contract review |
64
65     ### Step 3 ? Decide on measured quality-per-dollar
66     The bake-off numbers are the decision ? not sales decks. Low IAA at a vendor means
either a weak vendor or a vague guideline (Step 0); investigate before blaming the vendor.
67
68     ### Step 4 ? Onboard the winner(s)
69     Sign DPA + the security controls in §4; codify the guideline as their spec; agree SLAs
for both tiers (§5).
70
71     ---
72
73     ## 4. The extensible architecture ? an AnnotationProvider registry
74
75     This codebase already uses pluggable registries (validators, segmenters, AI providers,
sports). The idiomatic move is the same shape for annotation sourcing:
76
77     ```python
78     class AnnotationProvider(ABC):
79         def submit(self, video_ref, guideline, sport) -> str:           # returns job_id
80             ...
81         def poll(self, job_id) -> str:                                  #
queued|running|done|failed
82             ...
83         def fetch(self, job_id) -> List[Interval]:                     # canonical

```

```

[(start,end), ...]
84     ...
85     ...
86
87     `fetch()` returns the same `List[Interval]` that `annotations.load_annotations()`
already produces, so everything downstream is unchanged.
88
89     Three implementations cover today plus both tiers:
90
91     | Provider | Role | Tier |
92     |---|---|---|
93     | `FileProvider` | Reads a pre-made CSV (formalizes today's manual / ShuttleSet flow) |
? |
94     | `HumanVendorProvider` | The bake-off winner; authoritative ground truth, corpus
growth, deep audits | Slow (days) |
95     | `AutomatedProvider` | An independent strong model labels rallies programmatically |
Fast (hours) |
96
97     The tiers are just two providers behind one interface, selectable per call.
98
99     ---
100
101     ## 5. The regression automation (the explicit ask)
102
103     > "given a video, get a golden set created and verify its precision and recall against
the production tool"
104
105     This becomes a thin wrapper over machinery that already exists:
106
107     ```python
108     def regress(video, tier="fast", tolerance=0.05):
109         gt          = annotation_registry.get(tier).fetch(          # NEW: the provider
110             annotation_registry.get(tier).submit(video, GUIDELINE, sport))
111         predictions = process(video)                                # existing production
pipeline ? DB
112         s          = eval.score(predictions, gt)                    # existing metrics.py,
unchanged
113         regressed  = (s.recall < baseline[video].recall - tolerance or
114             s.precision < baseline[video].precision - tolerance)
115         if regressed: alert(video, s, baseline[video])
116         return s
117     ```
118
119     Only one new component (the provider). `run_eval.py`, `harness.py`, and `metrics.py`
are reused verbatim. Snapshot a `baseline.json` per corpus video so "regression" is a measured
delta, not a vibe.
120
121     Triggers (all feed the same `regress()`): CI on pipeline changes · on-demand (the
"surprise" case) · scheduled cron · pre-release gate.
122
123     ### ?? The oracle problem (a real correctness trap)
124     The fast-tier `AutomatedProvider` must be a different model lineage than production.
If production becomes the offline ActionFormer segmenter (ADR-004) and the oracle is also
ActionFormer, they share blind spots ? the test shows green while both are wrong. Pairings that
stay honest:
125
126     - production = offline learned segmenter ? fast-tier oracle = paid Gemini full-video
(different lineage).
127     - production = Gemini hybrid ? fast-tier oracle = a different VLM or the slow human
tier.
128
129     Treat the fast tier as a smoke alarm, not ground truth: any fast-tier "regression"
escalates to the slow human tier for confirmation before it blocks a release.
130
131     ---

```

132

133 **## 6. Vendor shortlist (Bangalore/India-favoring)**

134

135 > ****Researched 2026-05-30 via a lean manual pass**** (targeted web search + direct site/source verification ? NOT the heavy automated workflow, which failed). ****Confidence is labelled per row.**** ****No pricing is listed because none could be verified**** ? every vendor in this space quotes per-project, so ?/clip is an offline-conversation item (see §6.3). Treat single-source rows as ***leads to verify on the call***, not vetted recommendations.

136

137 **### 6.1 Tier-1 candidates ? verified India presence + video capability (talk to these first)**

138

139 | Vendor | Location (verified) | Video / temporal labeling | Notes for the call | Confidence |

140 |---|---|---|---|---|

141 | ****iMerit**** | ****Bengaluru**** office (A101 & A118, 43 Residency Rd, 560025; tech hub opened Oct 2022); also Kolkata HQ + US | Yes ? image/video/text/audio; explicit Sports & Biomechanics domain on their site | Largest & most enterprise-ready (5,500+ annotators); has a real ****Bangalore**** footprint for in-person coordination; ask for a sports/temporal-segmentation reference + a paid pilot. Higher-touch ? likely higher cost. | ****High**** |

142 | ****TELUS International AI**** (ex-****Playment****) | Playment was ****Bangalore-founded**** (2015); acquired by TELUS 2021, team retained | Yes ? built on Playment's 2D/3D image+video+LiDAR CV pipeline | Strong CV/video pedigree, Bangalore roots; but now a large global BPO ? confirm they'll take a ****small startup pilot**** and who the India delivery contact is. | ****High**** (origin) / ****Med**** (current India ops shape) |

143 | ****Taskmonk**** | ****Bengaluru**** | Yes ? "Video" listed as a labeling modality; platform ****+**** managed-service workforce | Mid-size, India-local, does both tooling and service ? potentially the best ****cost/agility**** fit for a startup pilot. Verify temporal/event labeling specifically (their heritage is e-commerce catalog tagging). | ****Med-High**** |

144

145 **### 6.2 Tier-2 ? India video-annotation services, location/specialts to confirm on contact**

146

147 | Vendor | Stated location | Note | Confidence |

148 |---|---|---|---|

149 | ****Infolks**** | India (Kerala-based per third parties) | Targets startups/SMBs; image/video/text/audio; positions on cost-efficiency ? good cheap-pilot candidate | ****Med**** |

150 | ****Analytics**** | India + USA (since 2018) | General data-annotation incl. video; widely listed | ****Med**** |

151 | ****Objectways**** | HQ Scottsdale AZ; ****India delivery in Coimbatore**** (not Bangalore) | Does video/LiDAR; India delivery but not Bangalore-local | ****Med**** |

152 | ****Roora**** (Roora ML Pvt Ltd) | ****Bengaluru**** ? HSR Layout (verified, `rooragr.com`) | Image/video/text annotation; sports industry listed. ****Selected for the initial paid pilot (2026-06-02)**** ? existence/capability/location verified; ****quality TBD via the pilot****. | ****Verified**** (capability) . pilot pending |

153 | ****Tika Data**** / ****Zuru**** | Claimed ****Bangalore**** (single-source listicle each) | Smaller/startup annotation shops ? unverified beyond one directory mention; verify they exist & do temporal video before investing time | ****Low**** |

154

155 **### 6.3 Explicitly NOT verified / out of scope**

156 - ****Pricing for everyone**** ? all quote-only; do not assume. Get per-video-minute (or per-rally) quotes during the offline conversations.

157 - ****Karya (Bengaluru)**** ? confirmed local and reputable, but ****speech/text/translation only, no video**** ? ****excluded**** for our use case.

158 - ****Sports/temporal track record**** ? beyond iMerit's stated "Sports & Biomechanics" domain, vendors' specific ***rally-boundary / temporal-event*** experience is unconfirmed marketing copy; probe with the pilot (§3).

159 - ****DPDP / DPA / ISO posture**** ? not verified per-vendor; make it a checklist item in the bake-off (§3 Step 2 security row).

160

161 **### 6.4 Recommendation**

162 For the ****offline conversations****, start with ****iMerit**** (Bangalore-local, enterprise QA, sports domain) and ****Taskmonk**** (Bangalore-local, likely more startup-friendly cost/agility) as the two primary contacts, with ****TELUS/Playment**** as a third if you want a heavyweight CV pedigree. Run the §3 bake-off across whichever 2?3 agree to a paid pilot, and decide on measured

quality-per-rupee. ****Verify every "Confidence: Med/Low" claim on the call before committing.****

163

164 > ****Update (2026-06-02):**** the founder opted to start with ****Roora**** (Bangalore-local, startup-friendly) as a ****single-vendor pilot**** rather than the iMerit/Taskmonk multi-vendor bake-off above. iMerit / Taskmonk / TELUS remain the fallback shortlist if the Roora pilot doesn't clear the §3/§4 quality bar.

165

166 > Sources (manual pass, 2026-05-30): iMerit Bengaluru hub ? imerit.net press / PRNewswire (Oct 2022) + contact listings; Playment/TELUS ? BusinessWire & Inc42 (2021 acquisition, Bangalore-founded); Taskmonk & Karya locations ? company sites + AnalyticsIndiaMag / IndiaAI; others ? AnalyticsIndiaMag / Ziloservices / vendor sites. Pricing intentionally omitted (unverifiable).

167

168 ---

169

170 **## 7. Suggested phasing**

171

Phase	Deliverable	Depends on
0	Annotation guideline + internal gold key (5?10 clips)	nothing ? can start now
1	Vendor bake-off ? pick winner(s)	§6 research + guideline
2	`AnnotationProvider` registry + `FileProvider` + `HumanVendorProvider`	winner onboarded
3	`AutomatedProvider` (independent oracle) + `regress()` + baseline snapshots	Phase 2
4	Wire surprise triggers (CI / on-demand / scheduled)	Phase 3

179

180 Phase 0 is unblocked today and is the highest-leverage step regardless of vendor choice.

181

182 > ****Status (2026-06-02):**** Phase 0 guideline ? authored (collector `docs/annotation/`); internal gold key still pending. Phase 1 is running as a ****single-vendor Roora pilot**** across ****3 sports**** (badminton, pickleball, table tennis) rather than a multi-vendor bake-off. Phase 2's `FileProvider` (GS-1) and Phase 3's `regress()` + automated oracle (GS-2/GS-4) have already landed; the main remaining slice is the ****vendor-dependent `HumanVendorProvider`**** (GS-3), with trigger-wiring (GS-5) after it.

183

184 ---

185

186 **## 8. Open questions**

187 1. ****Guideline authorship**** ? who owns the canonical rally definition, and is one expert's key enough, or do we want 2-expert consensus for the answer key itself?

188 2. ****Corpus representativeness**** ? how well does the owned/licensed collection mirror eventual user capture? (Ties directly to VIDEO_RECORDING_GUIDELINES.md.)

189 3. ****Baseline drift**** ? when production legitimately improves, how/when do we re-snapshot `baseline.json` so old baselines don't raise false regressions?

190 4. ****Fast-tier oracle cost**** ? per-video Gemini cost at regression frequency (see `archives/MONETIZATION\_AUDIT.md` §4 for the per-video estimate).

191

9. user (2026-06-10T05:33:37.220Z)

1 # TrackNet / WASB on WSL2 ? setup & usage

2

3 Self-hosted shuttle-trajectory detectors run inside WSL2 (Ubuntu) and are bridged

4 to the Windows indexer via a subprocess adapter. See `docs/DECISIONS.md` (ADR-001)

5 for the why.

6

7 **## What's installed (WSL2 / Ubuntu 24.04)**

8

Thing	Location (WSL)
Miniconda	~/miniconda3
TrackNetV4 repo	~/models/TrackNetV4 (https://github.com/TrackNetV4/TrackNetV4)
TrackNetV4 conda env	`TrackNetV4` (Python 3.9, TensorFlow 2.17 + bundled CUDA 12.3)

9

10

11

12

13

14


```

14 | WASB repo | `~/models/WASB-SBDT` (https://github.com/nttcom/WASB-SBDT) |
15 | WASB conda env | `wasb` (Python 3.8, torch 1.11.0+cu113) ? verified
   `torch.cuda.is_available()==True` |
16 | Staging dir for videos | `~/clips` (avoids slow `/mnt/c` per-frame reads) |
17
18 GPU: RTX 2070 Super visible via `nvidia-smi` inside WSL (CUDA-on-WSL working).
19
20 Activate an env with:
21 ```bash
22 source ~/miniconda3/etc/profile.d/conda.sh
23 conda activate TrackNetV4 # or: conda activate wasb
24 ```
25
26 ## ?? Two upstream issues you must resolve before TrackNetV4 inference
27
28 1. **No pretrained weights are hosted.** The "Download" links in
29   `~/models/TrackNetV4/docs/RESULT.md` are placeholders `(#{})`. You will obtain
30   weights by **training** (`src/train.py`) on your annotated data ? which is the
31   plan. Until you have a `.h5`/`.keras` weights file, the segmenter's healthcheck
32   passes but `run_predict` will log an error that `weights_path` is unset.
33
34 2. **`predict.py` has a custom-objects bug.** `src/predict.py` references layers in
35   `custom_objects` that it never imports (`MotionIncorporationLayerV1`,
36   `MotionIncorporationLayerV2`, `CombineOutputs`, `MotionFramesInput`) ? it will
37   raise `NameError` on `load_model`. The imported layers are
38   `MotionPromptLayer`, `FusionLayerTypeA`, `FusionLayerTypeB`. Patch the
39   `custom_objects` dict in `main()` to match the layers your trained model
40   actually uses, e.g.:
41   ```python
42   model = load_model(model_weights_path, custom_objects={
43       'MotionPromptLayer': MotionPromptLayer,
44       'FusionLayerTypeA': FusionLayerTypeA, # if you trained TrackNetV4_TypeA
45       'FusionLayerTypeB': FusionLayerTypeB, # if you trained TrackNetV4_TypeB
46       'custom_loss': custom_loss,
47   })
48   ```
49
50 ## Wiring it into the indexer
51
52 The segmenter is `tracknet_hybrid` (`backend/indexer/tracknet_hybrid.py`), backed by
53 `backend/detectors/tracknet_runner.py`. Configure it in `config.json` under
54 `indexing.tracknet`:
55
56 ```jsonc
57 "tracknet": {
58   "wsl_distro": "Ubuntu",
59   "repo_dir": "~/models/TrackNetV4",
60   "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
61   "conda_env": "TrackNetV4",
62   "weights_path": "", // <-- set to the WSL path of your trained weights
63   "queue_length": 5,
64   "stage_in_wsl": true, // copy the video into ~/clips before inference
65   "wsl_stage_dir": "~/clips",
66   "timeout_sec": 0, // 0 = no timeout
67   "velocity_threshold": 0.02 // normalised shuttle displacement/sec to count as "in
flight"
68 }
69 ```
70
71 Then select it as the segmenter (e.g. set `indexing.default_segmenter` to
72 `tracknet_hybrid`, or pass it through whatever segmenter-selection your run uses).
73
74 ### Pipeline shape (mirrors `yolo_hybrid`)
75
76 1. **Phase 1 ? healthcheck:** activates the WSL env, confirms TF imports and a GPU

```

```

77     is visible. Fails fast with a clear message otherwise.
78 2. **Phase 2 ? TrackNet:** runs `predict.py` on the whole video, parses the
79   `Frame,Visibility,X,Y` CSV, and converts the shuttle trajectory into candidate
80   rally windows (visible + moving above `velocity_threshold`, grouped by
81   `dead_time_merge_gap`). Raw candidates are persisted for fine-tuning (same table
82   as YOLO candidates).
83 3. **Phase 3 ? AI handoff:** each candidate window is sent to the configured AI
84   provider for semantic boundary confirmation (identical to `yolo_hybrid`).
85 4. **Phase 4 ? DB:** confirmed segments written and linked back to their candidate.
86
87 ### Quick manual smoke test (inside WSL, once you have weights)
88
89 ``bash
90 source ~/miniconda3/etc/profile.d/conda.sh && conda activate TrackNetV4
91 cd ~/models/TrackNetV4/src
92 cp /mnt/c/Users/avidu/Projects/badminton-highlight-indexer/<clip>.mp4 ~/clips/
93 python predict.py --video_path ~/clips/<clip>.mp4 \
94   --model_weights <your_weights> --output_dir ~/clips/out
95 head ~/clips/out/<clip>_predict.csv # expect: Frame,Visibility,X,Y
96 ```
97
98 ## WASB (multi-sport ? and the faster path to a baseline)
99
100 **WASB ships pretrained weights**, including badminton, in
101 ~/models/WASB-SBDT/pretrained_weights/:
102
103 - `wasb_badminton_best.pth.tar` (the WASB model)
104 - plus `tracknetv2_badminton_best.pth.tar`, `monotrack_badminton_best.pth.tar`,
105   `restracknetv2_badminton_best.pth.tar`, `ballseg_badminton_best.pth.tar`,
106   `deepball*_badminton_best.pth.tar` ? WASB re-implements all baselines.
107
108 This means **WASB can detect the shuttle on badminton video today**, whereas
109 TrackNetV4 has no hosted weights and must be trained first. If you want a working
110 trajectory baseline before your fine-tuning data is ready, WASB is the quicker
111 route.
112
113 Caveat: WASB's repo documents dataset *evaluation* (`main.py --config-name=eval
114 ...`), not custom-video inference. To run it on arbitrary match video you'd wrap
115 its detector + temporal-consistency step in a small inference script that emits
116 the same `Frame,Visibility,X,Y` CSV the runner already parses, then add a WASB
117 variant of `TrackNetRunner`. The trajectory?candidate-window logic is shared and
118 already tested.
119

```

10. user (2026-06-10T05:33:42.153Z)

```

1 # Video Recording Guidelines (DRAFT ? thresholds pending empirical data)
2
3 **Status:** ? Draft / placeholder . **Date:** 2026-05-30
4 **Why this exists:** highlight quality is bounded by *input* quality. To get reliably
good auto-generated highlights, the source video must be recorded a certain way (camera height,
court position, resolution, frame rate, stability, framing). We want to (1) **derive** the
minimum acceptable capture spec **empirically** from the precision/recall of the solutions we
build, and (2) **publish user-facing best practices** so a user knows how to record footage that
the product can index well.
5
6 > **This is a tracked thought, not a finished standard.** The exact numeric thresholds
below are **placeholders** to be replaced with values measured against our golden set. See "How
we'll set the real numbers" (§3). Captured now (per the team's working agreement) so it isn't
lost; to be revisited.
7 >
8 > **Update (2026-06-02):** the **frame-rate** and **camera-geometry** rows now carry
**literature-backed starting values** (cited inline). They remain *starting points* to confirm
empirically per §3. The golden-set **corpus** diversity spec (what footage to collect, not just
how users should record) is now in [GOLDEN_SET_PHASE1_VIDEOS.md](GOLDEN_SET_PHASE1_VIDEOS.md).

```

```

9      >
10     > **Update (2026-06-07):** **AUDIO is now a first-class capture requirement** (new row
below), per ADR-007 ? we will use the shuttle "thwack" as a rally cue (audio+visual fusion lifts
recall). This is a hard "record with audio on, mic unobstructed" ask, not a placeholder;
efficacy on real GoPro audio is being validated (experiment E1 in
[AUDIO_RALLY_DETECTION_PLAN.md])(AUDIO_RALLY_DETECTION_PLAN.md)).
11
12     ---
13
14     ## 1. Why input capture matters (grounded in what we already know)
15
16     ADR-004 found a concrete, capture-dependent failure mode: on **broadcast** footage the
camera *tracks the players*, so player pixel-motion dips during rallies and spikes on
pans/replays ? making player-motion a poor rally proxy. The lesson generalizes: **the pipeline's
accuracy is conditional on how the video was shot.** A **static, full-court** camera (typical of
a fixed tripod recording) behaves very differently from broadcast ? and is likely much
friendlier to both the motion/detector stages and the shuttle-tracking substrate (WASB/TrackNet,
ADR-001).
17
18     So capture guidance isn't cosmetic ? it directly moves precision/recall.
19
20     ---
21
22     ## 2. The dimensions that likely matter (hypotheses to test)
23
24     | Dimension | Why it plausibly affects P/R | Placeholder guidance (UNVERIFIED) |
25     |---|---|---|
26     | **Resolution** | Shuttle is tiny & fast; too few pixels ? detector misses it | ?
**1080p** preferred; **720p** likely floor; 2K/4K may help shuttle tracking but ? compute/cost |
27     | **Frame rate** | Motion blur; boundary precision; fast-ball detection | ? **30 fps**
badminton/pickleball; **60 fps table tennis ? 30 fps collapses fast-ball detection (**22.5%
? 93.2%** at 60 fps; 120 ? 60, [Nature Sci
Rep](https://www.nature.com/articles/s41598-023-42966-6)) |
28     | **Camera position** | Static full-court view avoids the broadcast camera-tracking trap
(ADR-004) | **Fixed tripod**, elevated, behind/above one baseline, whole court in frame |
29     | **Camera height & angle** | Occlusion of players/shuttle; court keystoneing | **3?5 m**
elevated (~4.5 m), **~30° down-tilt**, behind one baseline, whole court + net headroom
([badminton-CV setup](https://pmc.ncbi.nlm.nih.gov/articles/PMC10747833/)) |
30     | **Framing / margins** | Cropping the court or players breaks detection & relevance
check | Entire court + a margin; players never leave frame |
31     | **Stability** | Handheld jitter inflates motion signals | Tripod / fixed mount; no
panning or zoom during play |
32     | **Lighting / exposure** | Blur & contrast affect detection and the existing
blur/relevance validators | Even indoor lighting; avoid backlight/glare; minimize motion blur |
33     | **Court contrast / background** | Relevance check uses court color; busy crowds add
false motion | Clear court markings; minimal moving background where possible |
34     | **Codec / container** | Pipeline already validates MP4 + decodes H.264/HEVC | MP4
(H.264/HEVC) as today; align with `archives/MONETIZATION_AUDIT.md` codec notes |
35     | **Audio (REQUIRED ? ADR-007)** | Shuttle "thwack" hits are a strong rally cue;
audio+visual fusion lifts rally *recall* (the WASB bottleneck) ? see ADR-007 /
AUDIO_RALLY_DETECTION_PLAN.md | **Record with AUDIO ENABLED** (GoPro default ? do not mute).
Keep the **mic unobstructed**; **avoid aggressive wind/noise reduction** that flattens the sharp
impact transient. Camera reasonably close to the court helps SNR (shuttle hits are quiet). |
36
37     These map onto checks that **already exist** in `backend/validators/` (resolution, fps,
blur, scene-cut, court relevance) ? so several can be enforced/measured programmatically rather
than only advised.
38
39     ---
40
41     ## 3. How we'll set the REAL numbers (empirical plan)
42
43     Replace every placeholder above with a value derived from data, using the golden-set
harness:
44

```

```

45     1. Take golden-set clips (GOLDEN_SET_VENDORING.md) spanning a range of each
dimension ? e.g. the same match re-encoded at 480p/720p/1080p/4K, or clips at different camera
heights.
46     2. Run the production pipeline; score with `backend/eval/metrics.py`.
47     3. Plot precision/recall vs. each dimension ? find the knee where quality collapses.
48     4. Set the published minimum at (knee + safety margin); set a "recommended" tier
above it.
49     5. Re-derive when the pipeline changes materially (new detector/segmenter shifts the
knees).
50
51     This turns "record in 1080p" from an assertion into a measured threshold ? and lets us
tell users *why*.
52
53     ---
54
55     ## 4. Two surfaces for the output
56
57     1. User-facing best-practices guide ? short, friendly, illustrated: "Put the camera
here, this high, record at ?X, keep it still." The goal is that a user can record once and get
great results.
58     2. Programmatic pre-flight check ? extend `backend/validators/` to *reject or warn*
on out-of-spec uploads at ingest (resolution/fps/blur/framing), so the product fails fast with
actionable feedback instead of producing poor highlights silently.
59
60     ---
61
62     ## 5. Open questions
63     1. Which dimension dominates ? resolution, camera staticness, or height? (Run the
ablations in §3 to rank them.)
64     2. Do we enforce minimums (reject uploads) or only warn? Likely warn first,
enforce once thresholds are trustworthy.
65     3. Single fixed-camera spec, or several supported "recording profiles" (e.g. baseline-
elevated vs. side-elevated)?
66     4. How does the owned/licensed golden corpus need to grow to *cover* the capture
conditions real users will actually use? (Feeds GOLDEN_SET_VENDORING.md §8.)
67

```

11. user (2026-06-10T05:33:42.846Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-255 of 330 total (27423 tokens, cap 25000). Call Read with offset=256 limit=255 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```

1      # CODE_MAP.md
2
3      **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-
hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
4
5      **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this
doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure
(this revision follows #40).
6
7      **LEGEND**
8      - `@path` = file (always repo-relative)
9      - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
10     - `->` = dataflow / call / "produces"
11     - `$` = data contract (canonical shape)
12     - `?` = gotcha / footgun
13     - `?` = registry / plugin extension point
14     - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
15
16     ---
17

```

```

18     ## 1. PIPELINE
19
20     ### 1A. Runtime: video file -> highlight reel
21     ```
22     typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
23                                     @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
24     -> video_id = MD5(whole file)          @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
25     -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
26     FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
27     [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
28     -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
29     -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')
30     -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
31     -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
32     [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
33     -> finally: emit_indexer_report(...) @backend/reporting/__init__.py (? ALWAYS runs
? even on validation failure / exception; NEVER raises; grafts response["reports"])
34     -> select segment ids -> [(start,end)] (+ stitching padding)
35     -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
36     per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
37     ```
38     Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
39     ```
40     P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
41     -> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted
via db.add_candidate_segment]
42     -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt) ? provider_registry
43     -> P4 db.add_play_segment + db.link_candidate_to_segment
44     ```
45     Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
46     Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).
47
48     ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
49     ```
50     WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
51     -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
52     -> WasbRunner.run_predict(video, out_dir)
53     stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
54     -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
55     -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
56     -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING

```

```

BRAIN]
57     ...
58     (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
59
60     ### 1C. Calibration / eval flow (NO pipeline run unless told)
61     ...
62     GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
63     DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]    stage
? {candidates, final}
64     -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
65     -> harness.evaluate -> EvalReport -> report_to_dict
66     -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
67     -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py
[CI gate]
68     Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
69     Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned
classifier) @backend/eval/{calibrate_local,distill_local}.py
70     Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
71     Gemini refinement driver (tuning, standalone):
gemini_refine.{refine_window,full_video_rallies} @backend/eval/gemini_refine.py
72     Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
73     ...
74     Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch,
can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
75
76     ---
77
78     ## 2. SUBSYSTEMS
79
80     ### 2.0 Orchestration & config
81     - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS).
Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig`
(INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces
stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI
summaries may still use `print` for direct output.
82     - `::app` FastAPI; `::db_instance`, `::global_config` (`Optional[AppConfig]`) module
globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` ->
every endpoint NPEs.
83     - `::process_video` `@POST /api/process` -> validate -> add_video -> segment ->
`finally:` always `emit_indexer_report` (never raises). `::compile_highlights` `@POST
/api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`.
`::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs
`indexing.motion_threshold` (dual model/dict read).
84     - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
85     - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
86     - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
87     - `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()`
(blocks automation), skips validation. Config via `backend.config.load_config` (typed);
segmenter, padding, and db path all read from the model (no literal fallbacks).
88     - `@run_eval.py::main` ? score one video's DB preds vs GT CSV.

```

```

`--stage{candidates,final,both}`. Exit 2=arg/I0. ? pure scorer; errors if video never processed.
`::default_db_path` resolves via `backend.config.load_config`
(`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
89 - `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate.
`load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label);
config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig`
(same object the live pipeline feeds segmenters).
90 - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic:
**0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare
(snapshots even a regressing run); `::default_db_path` resolves via
`backend.config.load_config`.
91
92 - `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports
`load_config`, `save_default_config` (from `loader`) + `AppConfig`, `Config`, `IndexingConfig`,
`ValidationConfig` (from `models`); `__all__` = exactly those 6. Use `from backend.config
import load_config, AppConfig`.
93 - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for
defaults**.
94 - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`,
`ai_model='gemini-2.5-flash'`; sections `validation`, `indexing`, `output`, `stitching`
(`default_factory`). `model_config` = {extra:'allow', validate_assignment:True}. ?
`extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
95 - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain
dicts** (`model_dump(mode='json')`, NOT the model) so chained
`config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any
section (sections discovered dynamically from `model_fields`). ? "bit every validator on real
runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises
`KeyError` for unknown top-level keys else delegates to `.get`.
96 - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`,
`min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`,
`chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`,
`ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested
`tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `gemini`. ?
`::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ?
a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)` .
97 - `::ValidationConfig` (`min_resolution_width=1280`, `min_resolution_height=720`, `min_fps=29.9`,
`min_blur_threshold=20.0`, `max_scene_cuts_per_minute=0.5`, nested `relevance`);
`::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`).
`::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`,
`conda_env='TrackNetV4'`, `weights_path='', `queue_length=5`, `velocity_threshold=0.02`).
`::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`,
`output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`).
`::StitchingConfig`
(`begin_padding=0.5`, `end_padding=0.5`, `use_cache=False`, `min_shot_count=1`). `::Sport` Literal
of 5 sports. `::Config` = `AppConfig` alias.
98 - `@backend/config/loader.py` ? load/save with defaults-merge.
99 - `::DEFAULT_CONFIG_PATH` = Path("config.json")` (? CWD-relative, NOT repo-root).
`::get_built_in_defaults` = `AppConfig().model_dump(...)` .
100 - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-
level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole
defaults dict for that section (Pydantic then re-applies field defaults for missing leaves).
Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-
fatal).
101 - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present
(used by setup.py/--setup). `::get_default_config` ? ? transitional plain-dict alias.
102 - `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**`indexing.default_segmenter='gemini'`** diverges from model default `yolo_hybrid` ? file
wins at runtime.
103 - `@setup.py` ? `::init_default_config` ? delegates to
`backend.config.save_default_config` (single-source defaults; skips if config.json exists).
`::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8,
ffmpeg/ffprobe on PATH, optional torch+CUDA).
104
105 ### 2.1 Segmenters (under pipeline/segmenters) ?
106 - `@backend/pipeline/segmenters/base.py`

```

```

107     - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`, `description`, `process_video`. $ `process_video(video_id, video_path, player_pool=None,
max_frames=None) -> List[dict{id, start_time, end_time, duration, metadata}]`.
108     - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new
segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module
bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an
IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` ->
name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-
instantiates + swallows exceptions into "No description available." (a throwing `description` is
masked).
109     - `@backend/pipeline/segmenters/__init__.py` ? imports all 5
(`motion, gemini, yolo_hybrid, tracknet_hybrid, wasb_hybrid`) = the de-facto registration manifest;
re-exports them + `segmenter_registry` via `__all__`.
110     - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`'wasb_hybrid'`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE
default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config
`indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs
`{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike
tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit
(copy-paste drift).
111     - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig`
from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`;
`detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably
yolo_hybrid).
112     - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name
`'yolo_hybrid'`, no YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005).
`::PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections).
`::lazy_load_model` (lazy `torchvision` import so module/tests load without it),
`::extract_action_windows_from_chunk`, `::make_tracker` (per-chunk ByteTrack ? track IDs
chunk-local), `::DETECTOR_FACTORIES`/`::DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ?
velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to
`frame_skip`; pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU);
`yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated
aliases still honored.
113     - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ?
pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
114     - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ?
pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM**
(random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing
server/receiver; uses `print()`; honors `max_frames`.
115
116     ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)
117     - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() ->
Tuple[bool, str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately
matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring
includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on
threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL
de-risk seam (low-regret 6 / review item 1).
118     - `@backend/pipeline/detectors/tracknet_runner.py`
119     - `::TrackNetConfig` (+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro`
field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
120     - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/~` passthrough, UNC unhandled),
`::TrajectoryPoint` (frame, visible, x, y).
121     - `*::parse_trajectory_csv` @staticmethod + `*::trajectory_to_action_windows`
@staticmethod = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path`
defaults `''` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names
`<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`,
`frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout
(hangs forever). `::stage_video` "OK" sentinel.

```



```

122     - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner)
(+:::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default
`wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (+:::INFER_WIN`) +
video into WSL, runs in `wasb` env. `+:::parse_trajectory_csv`/`+:::trajectory_to_action_windows` =
`staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-
named `<stem>_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until
staged); `_wsl_bash` duplicated (drift risk). Both runners now satisfy the DetectorRunner ABC
for future swappability.
123     - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `+:::run` core. ? imports WASB-SBDT
repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache:
`+:::DET_FILE='det_raw.jsonl'`, `+:::MANIFEST_FILE='manifest.json'`, `+:::CACHE_VERSION=1`,
`+:::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to
heal crash-truncated tail), `+:::manifest_compatible` (? `frames_dir` abspath'd but `weights`
compared raw), `+:::atomic_write_text`/`+:::atomic_write_trajectory`, `+:::dets_to_tracker` (xy MUST be
np.array), `+:::build_cfg` (`runner.gpus=[0]` hardcoded), `+:::extract_frames` (cv2-PNG SLOW),
`+:::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker
STATEFUL -> always fully re-run; `--fresh` ignores cache.
124     - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure,
no I/O/model). `+:::estimate_play_axis` (larger-spread axis; ? ties -> 'x'),
`+:::count_net_crossings` (sign changes vs median net; deadband jitter zone),
`+:::gate_windows`/`+:::apply_gate` (`min_crossings=2` default; kills single-toss service feeds).
`+:::GatedWindow` (start,end,crossings,visible_points,kept). ? window contract here is
(start,end) tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local /
distill_local / export_wasb_segments.
125
126     ### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
127     - `@backend/eval/metrics.py` ? scoring primitive (THE spine). `+:::interval_iou`,
`+:::match_intervals` (greedy, deterministic), `+:::score`->`+:::Score`(`.as_dict`) = wire shape),
`+:::pr_curve`, `+:::Match`, `+:::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN)
for mean_iou/boundary errors.
128     - `@backend/eval/annotations.py` ? generic GT loader. `+:::load_annotations` (`start,end`
CSV; RAISES on bad rows/`start>=end`), `+:::parse_timestamp` (decimal or MM:SS/HH:MM:SS),
`+:::write_scaffold`.
129     - `@backend/eval/calibration.py` ? overfit guard. `+:::cross_validate` (within-video
k-fold; ? defaults `metric='recall'` deliberately), `+:::leave_one_video_out` (honest cross-video;
needs ?2 labelled videos), `+:::improvement_report` (OPTIMISTIC ? selected on full GT),
`+:::select_best`, `+:::kfold_indices`, `+:::pct_improvement` (? from-zero -> `inf`), `+:::evaluate`. $
`PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap
>~0.1` = overfit alarm.
130     - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `+:::BASELINE=(0.02,2.0,2.0)`,
`+:::default_grid` (45 cfigs), `+:::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `+:::load_golden_gts` ($`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
131     - `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing +
net-crossing gate, no cloud at runtime). `+:::local_grid` (180 cfigs), `+:::make_local_predict_fn`
(adds `rally_gate.apply_gate` when `min_crossings>0`), `+:::build_videos` (? `SystemExit` if a
labels file yields no rallies), `+:::run`/`+:::main`. $
`LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;
manifest JSON shared with distill_local.
132     - `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison
+ optional clip cut. `+:::TUNED=(0.05,1.0,1.0)` (? from ONE video),
`+:::SEG_FIELDS`, `+:::COMP_FIELDS`, `+:::build_comparison`, `+:::seconds_to_mmss`, `+:::maybe_slice`
(lazy-imports rally_slicer). `--slice` requires `--video`. ? `write_segments_csv` output loads
straight into `rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is
load-bearing.
133     - `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in
run_phase3_eval). `+:::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1),
`+:::default_stages_for` (`auto`; `+:::FINAL_ONLY_SEGMENTERS={motion,gemini}`),
`+:::resolve_video_path` (normalizes collector `./data\..` Windows paths), `+:::format_aggregate`.
134     - `@backend/eval/harness.py` ? DB-pred scorer (no re-run).
`+:::STAGES=('candidates','final')`, `+:::get_predictions`
(candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})`); ? unknown
stage raises ValueError; reused by regression.py),

```

```

`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate), `::format_report_text`.
135     - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES`
(5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default
0.0), `::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->(X,y,intervals); `feature_fn` is the substrate plug-in seam.
136     - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `_sigmoid`
clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON "model":"logreg" tag.
137     - `@backend/eval/eval_partial_labels.py` ? score pipeline vs **PARTIAL human labels**
(PR #60). `::restrict_to_window` (drop preds outside `[0, last GT end]` so un-labeled-tail
detections aren't FPS ? THE partial-label methodology), `::run` (baseline/+gate/tuned/+gate ops
table + per-rally diff via `export_wasb_segments.build_comparison`), `::sweep`/`--sweep` (grid
vs labels; ? fit-on-self = optimistic), `::DEFAULT_GRID`, `::TUNED=(0.05,1.0,1.0)`. ?
`window_end` inferred from `max(GT end)` is WRONG if a video is labelled PAST its last rally ?
P1 fix = collector persists `labeled_window` in the manifest. Reuses calibrate_wasb + rally_gate
+ metrics.
138     - `@backend/eval/rally_seq_proto.py` ? **PROTOTYPE** learned per-FRAME rally segmenter =
owned-model **Gen-0 seed** (PR #61). 6 features over the WASB trajectory (`vis_rate, spd_mean,
spd_max, cross_rate, x_std, y_std`) -> `classifier.LogisticRegression`, honest LOVO.
`::build_frame_arrays`, `::features` (scipy `uniform/maximum_filter1d` windowed; ? speed NOT
fps-normalized yet ? 30 vs 60fps), `::labels_for`, `::probs_to_segments`
(smooth->threshold->merge/min-dur), `::roc_auc` (rank-based, no sklearn), `::run` (per-video AUC
+ LOVO-threshold F1 + oracle-threshold F1). ? DIRECTIONAL only (n?5, linear, threshold-transfer
unsolved); held-out per-frame AUC 0.83?0.92, **beats the heuristic on multi-court footage**
(Kushagra/gBaaddy). Manifest `output/human_lovo_manifest.json` (the n=5 human corpus) shared w/
calibrate_local.
139     - ? **MULTI-COURT / background-game confound (2026-06-08 finding):** WASB tracks EVERY
shuttle in frame incl. other courts; videos with background games (Kushagra, gBaaddy) have high
dead-time shuttle visibility (41?57% vs 18?22% clean) ? the velocity-windowing heuristic can't
find clean rally boundaries (F1 0.16/0.28 vs ~0.75 on single-court). Diagnostic = in-rally+dead-
time visibility "contrast" (?3.6x heuristic works, ?1.9x fails). Lever = **foreground-court
isolation** (spatial gate to the prominent court, needs real court-region detection). Academy =
multi-court = the target env. See `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
140     - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond
calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT:
duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal),
`::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`),
`::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-
one-video-out needs ?2 videos; final model trained on all.
141     - `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT
padded clips (precision lever; eval/tuning, NOT the live pipeline).
`::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (?
FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`,
`::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises
SystemExit); imports `BASELINE` from calibrate_wasb +
`TUNED`/`write_segments_csv`/`seconds_to_mms` from export_wasb_segments.
142     - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`,
`::regress_with_provider` (GT via ?`annotation_registry.get(tier)`;
"file"->FileProvider/"vendor"->HumanVendor/"auto"->oracle),
`::save_baseline`/`::load_baselines`, `::RegressionResult`. ? imports `annotation_registry` from
**`backend.annotations`** NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass).
143
144     ### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
145     - `@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector. Pure
numpy + stdlib `wave` (NO scipy); ffmpeg only for extraction. `::HitEvent`(t,strength),
`::extract_audio` (ffmpeg -> mono 16k PCM WAV), `::load_wav`, `::onset_envelope` (pure-numpy
STFT spectral flux; `band=(lo,hi)` band-limit), `::detect_hits` (adaptive `mean+k.std`;
`k`=sensitivity knob), `::detect_hits_in_video`. ? separate signal ? WASB never modified by
this; per audio-e1-findings recall 100% but discrimination ~2.2x, band-pass null -> NOT adopted
(PR #35).
146     - `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).

```

```

`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
147
148     ### 2.4 Stitcher / tools / utils
149     - `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::parse_time`
(mirrors gemini.parse_time; ? unparseable -> 0.0), `::get_video_duration`. ? if duration probe
fails (0.0) NO boundary validation; `begin_padding`(0.5)/`end_padding`(1.0) defaults; temp clips
in a TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses
`print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so
final concat can stream-copy ? change a preset and concat may break.
150     - `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure,
unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::ClipWindow` (rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (?
duplicate compiler's by copy). `read_time` -> backend/eval/annotations.parse_timestamp`.
151     - `@backend/utils/video.py` ? `::get_video_duration` (ffprobe; ? 0.0 on failure =
"unknown"), `::extract_video_chunk(...,reencode=False,scale_width=None)`. ? copy mode cuts at
nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips; `scale_width` requires
`reencode=True`.
152     - `@backend/utils/geometry.py` ? `::get_velocity_normalised` (fraction of frame-width/s;
? raw-px fallback if width<=0; used by yolo_hybrid), `::calculate_iou` (? +1 px convention
inflates IoU), `::get_center/get_distance/get_velocity`.
153
154     ### 2.5 Validators ?
155     - `@backend/pipeline/validators/base.py::VideoValidator` ABC, `::ValidationResult`
(pydantic: validator_name,passed,message,details), `::ValidationRegistry` (ordered LIST ?
registration order = exec order; ? no dedup), `::registry` (singleton). $ `validate(video_path,
config, context)` -> `ValidationResult`. `::run_all_with_context` -> (results, context) (wraps
each validate in try/except -> failed result not abort); `::run_all` discards context. ? **add a
validator**: subclass + `registry.register(MyValidator())`.
156     - `@backend/pipeline/validators/__init__.py` ? registers in EXEC order: FormatValidator,
ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator,
RelevanceValidator (? producers before consumers ? format_check first; ? import mutates global
registry as side effect).
157     - `@backend/pipeline/validators/format.py::FormatValidator` (`format_check`) ?
**PRODUCER** of `context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is
substring `mp4` (so MOV-family passes); external `ffprobe`.
158     - `@backend/pipeline/validators/resolution_fps.py::ResolutionFPSValidator`
(`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails "Run format_check first" if
absent). `min_fps` 29.9; min res 1280x720.
159     - `@backend/pipeline/validators/pts_continuity.py::PTSContinuityValidator`
(`pts_continuity_check`) ? re-probes (no context). ? 10.0s keyframe-gap threshold hardcoded
(comment says 8.0s ? drift, NOT config-driven); any backward jump fails; `<2` keyframes PASSES
(too short).
160     - `@backend/pipeline/validators/scene_cut.py::SceneCutDensityValidator`
(`scene_cut_check`) ? OpenCV. ? `cut_threshold=0.6` hardcoded (NOT config); samples ~2fps, caps
at 100 SAMPLED frames (~first 50s); `max_scene_cuts_per_minute` (0.5) is the config fail
threshold.
161     - `@backend/pipeline/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var <
`min_blur_threshold` (default 100.0 in-file, 20.0 in models/config) = fail; samples 15 frames. ?
magic absolute, not resolution-normalized.
162     - `@backend/pipeline/validators/relevance.py::RelevanceValidator` (`relevance_check`) ?
HSV court-green ratio (? purely color, no geometry despite name); lazy `import backend.sports`;
3-tier config precedence (`validation.relevance` -> sport profile -> hardcoded); ? unknown sport
-> empty cfg, falls back to defaults, does NOT fail.
163
164     ### 2.6 Sports / Providers / Annotations ?
165     - `@backend/sports/base.py::SportProfile` ABC
(`name`,`get_prompt`,`get_relevance_config`);
`@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
`@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton; ?
`register` instantiates profile to read `name`). ? **add a sport**: subclass `SportProfile`,
`sport_registry.register(MyProfile)`. $ prompt emits JSON array `{start_time(float DECIMAL SEC),
end_time, shuttle_exchange_count(int), shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt

```

actively defends against MM:SS; `get()` returns a fresh instance each call.

```

166 - `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`);
`@backend/providers/gemini.py::GeminiProvider` (lazy `::client` property; `::MAX_UPLOAD_MB=500`;
upload->poll->generate JSON temp 0->delete); `@backend/providers/__init__.py::provider_registry`
(`+::ProviderRegistry`, auto-registers `gemini`). ? **add a provider**: subclass,
`provider_registry.register('name', MyCls)` (key passed EXPLICITLY); `get(name, api_key,
model_name)`. $ `analyze_video(path, prompt, chunk_duration=0.0, label='')` -> (segments:list,
metrics:dict{upload_time,process_time,gen_time})`. ? blocking 10s poll; returns `([], metrics)`
on ANY failure (empty != error); oversize clip silently skipped; orphaned remote file possible
if process dies mid-run.
167 - `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`),
`::Interval` (= metrics.Interval), `::QUEUED/RUNNING/DONE/FAILED`/`::STATUSES`,
`::AnnotationProviderRegistry` (decorator-friendly `register` returns the class; ? keys off
`provider_cls().name`, falls back to `__name__` if no-arg ctor throws; `list_available()` ->
LIST not dict), `::annotation_registry`. ? **add a tier**: subclass (define `name`),
`annotation_registry.register(MyCls)` at module bottom + import in `__init__.py`. $ `fetch` ->
List[Interval] sorted; contract says RAISE (not `[]`) on unavailable.
168 - `@backend/annotations/file_provider.py::FileProvider` (`file`; job_id IS the
resolved CSV path; `_resolve` -> `

```

```

`::ai_stage` ($ `details.{ai_based,provider,model,api_key_present}`; metric
`confirmed_segments`), `::highlights` (coverage_pct). ? `record_run` fps fallback:
`fps_source`=='unknown' + ran -> promotes to "fallback", forces fps=30.0 (mirrors runners).
`::SPEC_PATH`/`::load_spec` resolve `spec.yaml` next to module. (Soft import of obsreport so
pure helpers like video_meta are importable for tests/coverage even without the dep.)
178 - `@backend/reporting/spec.yaml` ? $ declarative report layout + footgun engine consumed
by obsreport. `::customer_verdict` (pass/pass_with_warnings/fail messages). `::sections`
(ordered `{id,title,kind,audience?{both,customer,technical}}`). ? `::rules` ? each
`{code,severity,when[{path,op,value}],message,faq_ref,customer_message?}`; `when` clauses AND'd;
`path` dotted-navigates the recorder JSON. ? `path` strings are a HARD coupling to harvest's
stage/metric/detail names ? renaming a detail in harvest.py silently breaks the matching rule.
Rules: `silent_provider_noop` (error: no api key + 0 segs), `ai_empty_vs_no_rallies` (warn),
`fps_fallback_used` (warn), `synthetic_motion_metadata` (warn), `segmenter_divergence` (info),
`reingest_replaced_prior` (info), `validation_failed` (error: halts indexing).
179
180 ---
181
182 ## 3. DATA CONTRACTS (canonical shapes ? one place)
183
184 $ **Trajectory CSV** (produced by tracknet/wasb predict, consumed by
`parse_trajectory_csv`):
185 `Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel
coords.
186 -> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).
187
188 $ **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo
`_extract_action_windows_from_chunk`):
189 `{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.
190
191 $ **candidate_segments DB row** (`db.add_candidate_segment` /
`query_candidate_segments`):
192 `{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL,
duration:REAL, confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count},
detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}`
(stats/detection_params deserialized on read).
193 `detection_params` keys by detector:
wasb={detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}; tracknet
adds `queue_length`; yolo={velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`.
194
195 $ **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
196 `{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server,
receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count,
shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare
`model_name`; motion adds random `intensity/avg_speed_kmh`.
197
198 $ **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time,
end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start;
`_candidate_id` stamped internally).
199
200 $ **GT / golden CSVs** (TWO conventions ? do not conflate):
201 - generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal
or MM:SS/HH:MM:SS; RAISES on bad row.
202 - golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
203
204 $ **det_raw.jsonl** (one window/line): `{"w":<window_index>,
"f":[[frame_id,[x,y,score,scale],...], ...]}`. Lines MUST be contiguous (line N has `w==N`);
first violation/JSONDecodeError -> truncate-heal.
205 $ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in,
frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.
206 $ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv},

```

```

...]}'; ? DB key = file MD5, manifest `video_id` is a label.
207
208     $ **eval primitives**: `Interval=Tuple[float,float]` (start<end).
`Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP),
unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives
,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.
209
210     $ **WSL detector candidate** (in-WSL, model<->tracker): `{xy':np.array([x,y]),
'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST
rebuild `xy` as np.array (tracker does `np.linalg.norm`).
211
212     $ **HitEvent** (audio, `pipeline/audio/hit_detector`): `{t:float(sec), strength:float}`
? onset-envelope peak.
213
214     $ **observability report** (`reporting/harvest.record_run` -> `RunRecorder`): stages
`validation/segmentation/ai_handoff` + highlights + verdict ? {pass,pass_with_warnings,fail};
flags `{code,faq_ref}`; `video_meta.fps_source` ? {probed,fallback}`. ? stage/metric/detail names
are coupled to `spec.yaml::rules[].when[].path` ? pairs that MUST stay in sync:
`ai_handoff.details.{api_key_present,ai_based}`, `segmentation.metrics.segment_count`, `segmenta
tion.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinges
t}`, `validation.status`, `video_meta.fps_source`.
215
216     ---
217
218     ## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ?
claims below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
219     - Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra
`compose(config_name='eval', overrides=[dataset=<sport>, model=wasb,
detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is
single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
220     - `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH),
ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read
from cfg["model"] at runtime in wasb_infer.run.)
221     - `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats)` -> (results,
hms_vis). `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.
(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
222     - `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes
every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()`
resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls
`build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y`
? consistent.)
223     - `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh,
transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)
224     - Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-
data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial,
avoided.
225
226     ---
227
228     ## 5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified
here)
229     - Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-
encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest
tracks the exact file used.
230     - `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv`
(indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` +
`@run_phase3_eval.py`.
231     - Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) ->
consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.
232
233     ---

```

```

234
235     ## 6. GOTCHAS (cross-cutting footguns)
236     - **MD5 keying:** `video_id` = MD5 of WHOLE file via the single
`@backend/utils/hashing.py::compute_video_id` helper (64KB chunks), imported by main + all
run_*.py. Manifest `video_id` is a label, NOT the DB key.
237     - **Config access:** `backend/main.py` and all four `run_*.py` load via
`backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains
for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in
`main.py` that only fires if `global_config`/.indexing` is `None` (the ASGI-import edge case).
238     - **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `motion`
getattr fallback noted above.
239     - **`AppConfig.get` must return dict sections, not models:** the legacy
`.get("indexing", {}).get(...)` chains everywhere depend on it; the `extra="allow"` model also
silently keeps typo'd config keys (no rejection).
240     - **`output_dir`:** a real field on `OutputConfig` (default `"output"`); the CLI
`--output-dir` overrides it on the typed model in `main()`, and `compile_highlights` reads the
same `global_config.output_dir` ? honored end-to-end.
241     - **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if
`{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op. `analyze_video` also
returns `[]` on real failure -> empty != "no rallies". The reporting
`silent_provider_noop`/`ai_empty_vs_no_rallies` rules exist precisely to disambiguate this in
the report.
242     - **Reporting never breaks a run:** `emit_indexer_report` swallows all exceptions
(designed for `main.py` `finally:`) and is emitted even on validation failure. `obsreport` is a
SOFT dep ? absent -> no-op.
243     - **spec.yaml <-> harvest.py coupling:** rule `path` strings dotted-navigate harvest's
stage/metric/detail names; rename a detail in harvest.py and the matching rule silently stops
firing. Add rules in spec.yaml, not code.
244     - **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner
fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes
`59.94/1920`. The report's `fps_fallback_used` flag surfaces this.
245     - **WSL `/mnt` can't see Google Drive / network mounts:** stage video into native WSL fs
(`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
246     - **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on
PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg +
`--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo
modules).
247     - **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-
reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
248     - **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST
exist in `output/<db_name>` before any eval/regression.
249     - **Validator ordering coupling:** exec order is Format -> ResolutionFPS ->
PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless
`format_check` ran earlier (shared `context['ffprobe_meta']`). PTS 10.0s gap + scene-cut 0.6
correlation are NOT config-overridable.
250     - **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-
deletes its segments. Use `clear_video_data` deliberately. `_get_connection` MUST re-issue
`PRAGMA foreign_keys=ON` per connection.
251     - **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to
P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-`TypeError` fallback vs
tracknet's `is None` check.
252     - **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs
`backend.annotations` (provider registry). regression.py imports the latter.
253     - **`calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared
API** for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-
CSV schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by
`distill_local`; `harness.get_predictions` reused by `regression.regress`.
254     - **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`,
slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
255     - **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.

```

```

256     - **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat;
if probe fails (0.0) no boundary validation.
257     - **Audio NOT in the live pipeline:** `pipeline/audio/hit_detector` +
`eval/audio/e1_probe` are a diagnostic probe (ADR-007); recall 100% but discrimination ~2.2x ->
not adopted for fusion.
258
259     ---
260
261     ## 7. RAMP PATHS (to do X -> open Y::symbol)
262     | Task | Open |
263     |---|---|
264     | Change a config default / add a config field | `@backend/config/models.py` (single
source of truth) -> regenerate via `setup.py`/`save_default_config`; ? config.json may override
(default_segmenter) |
265     | Read config in new code | `from backend.config import load_config, AppConfig`
(`@backend/config/__init__.py`); attribute access, NOT raw `json.load` |
266     | Tune candidate windowing (velocity/merge/min-dur) |
`@backend/pipeline/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json
indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`,
`min_action_window_duration` |
267     | Add a segmenter | subclass `@backend/pipeline/segmenters/base.py::BasePlaySegmenter`;
`segmenter_registry.register(...)` at module bottom; import in
`@backend/pipeline/segmenters/__init__.py`; if AI-based add to
`@backend/reporting/harvest.py::_AI_SEGMENTERS` |
268     | Change stitching padding | `config.json stitching.{begin_padding,end_padding}`; impl
`@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror
`@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
269     | Add a sport | subclass `@backend/sports/base.py::SportProfile`;
`sport_registry.register(...)` in `@backend/sports/__init__.py`; define prompt+HSV in new file |
270     | Add an AI provider | subclass `@backend/providers/base.py::AIVideoProvider`;
`provider_registry.register('name', cls)` |
271     | Add an annotation/GT tier | subclass
`@backend/annotations/base.py::AnnotationProvider`; register + import in
`@backend/annotations/__init__.py` |
272     | Add / change a report footgun rule | `@backend/reporting/spec.yaml::rules`
(declarative; ? `when.path` must match harvest's stage/metric/detail names); facts in
`@backend/reporting/harvest.py` |
273     | Tweak what the report records | `@backend/reporting/harvest.py::record_run` + stage
builders; emit point is `@backend/main.py` `finally:` ->
`@backend/reporting/__init__.py::emit_indexer_report` |
274     | Run calibration (WASB thresholds) | `@backend/eval/calibrate_wasb.py::main`
(`--trajectory --labels --fps --frame-width`); trust `cv` recall not `fit` |
275     | Distill Gemini -> local (thresholds / learned) |
`@backend/eval/calibrate_local.py::main` (thresholds) OR `@backend/eval/distill_local.py::main`
(learned classifier); both `--manifest` |
276     | Refine WASB candidates with Gemini (tuning) | `@backend/eval/gemini_refine.py::main`
(`--full-video` skips `--trajectory`; needs `GEMINI_API_KEY`) |
277     | Export tuned rally segments / clips | `@backend/eval/export_wasb_segments.py::main`
(`--slice` needs `--video`) |
278     | Export timelines (EDL/JSON/CSV) + basic stats (Value Layer slice) |
`backend/tools/export.py` (segments_to_edl/json/csv, basic_stats); called from anywhere that has
play_segments (CLI, notebook, future API) |
279     | Run a video end-to-end | `@backend/main.py::main` (`--video-path` or `--server`) or
`@run_process_and_stitch.py::main` (edit hardcoded paths) |
280     | Score DB preds vs GT (one video) | `@run_eval.py::main` |
281     | Batch score a collector manifest | `@run_phase3_eval.py::main` (`--manifest`) |
282     | Guard a regression (CI) | `@run_regression.py::main` (`--video-id`); exit 1=regress,
2=missing DB; snapshot via `--update-baseline` |
283     | Run the whole test suite | `@run_all_tests.py` (repo root); or `pytest tests/`
directly |
284     | Resume a crashed WASB pass | nothing to do ?
`@backend/pipeline/detectors/wasb_infer.py::run` auto-resumes from
`det_raw.jsonl`+`manifest.json` next to `--out`; `--fresh` forces restart |
285     | Add a validator | subclass `@backend/pipeline/validators/base.py::VideoValidator`;
`registry.register(instance)` in `@backend/pipeline/validators/__init__.py` (mind order;

```



```

producers before consumers) |
286     | Add a DB column/table | `@backend/infrastructure/database.py::Database._init_db`
migration ladder + bump `PRAGMA user_version` (else version-assert tests fail) |
287     | Train the learned segmenter | `@backend/eval/training.py::build_training_set` ->
`@backend/eval/classifier.py::LogisticRegression.fit/save` |
288     | Probe an audio signal (diagnostic) | `@backend/eval/audio/e1_probe.py::main`; detector
in `@backend/pipeline/audio/hit_detector.py` |
289
290     ---
291
292     ## 8. TEST MAP (`tests/test_*.py` -> subsystem)
293     All pure-logic / mocked-boundary unit tests; no WSL/GPU/Gemini/ffmpeg actually invoked.
`@run_all_tests.py` (repo root) is the suite runner.
294     | Test | Covers |
295     |---|---|
296     | `test_base.py` | `pipeline/segmenters/base.SegmenterRegistry` (register/get/list via
DummySegmenter) |
297     | `test_yolo_hybrid.py` | `pipeline/segmenters/yolo_hybrid.HybridYoloSegmenter`
(torchvision+ByteTrack; `_PERSON_CLASS_ID` filter; candidate link; `store_yolo_candidates=False`
skip) |
298     | `test_tracknet_hybrid.py` |
`pipeline/segmenters/tracknet_hybrid.TrackNetHybridSegmenter` (4-phase; healthcheck-abort;
`_probe_fps_width` fallback) |
299     | `test_gemini.py` | `pipeline/segmenters/gemini.GeminiPlaySegmenter` (single vs chunked
path, per-chunk offset, `shot_count` < `shuttle_exchange_count`) |
300     | `test_tracknet_runner.py` | `pipeline/detectors/tracknet_runner` (`to_wsl_mnt_path`,
`from_indexing_cfg`, parse + `trajectory_to_action_windows` shared brain, healthcheck,
run_predict, stage) |
301     | `test_rally_gate.py` | `pipeline/detectors/rally_gate` (axis/net estimate, crossing
count, gate kills single-toss) |
302     | `test_wasb_cache.py` | `pipeline/detectors/wasb_infer` resumable cache (serialize,
contiguity heal, manifest invalidation, atomic writes, `default_cache_dir`) |
303     | `test_eval_metrics.py` | `eval.metrics` IoU/matching/score/pr_curve |
304     | `test_eval_annotations.py` | `eval.annotations` GT CSV parse/validate,
`write_scaffold` |
305     | `test_eval_harness.py` | `eval.harness` DB-pred fetch + evaluate + report_to_dict;
also `@run_eval.py::main`/`get_file_md5` (exit codes, `--scaffold`/`--json`) |
306     | `test_eval_batch.py` | `eval.batch` path resolve / stage select / aggregate
(micro/macro, zero-div) |
307     | `test_calibration.py` | `eval.calibration`
pct_improvement/kfold/select_best/cross_validate/LOVO/improvement_report |
308     | `test_calibrate_wasb.py` | `eval.calibrate_wasb`
load_golden_gts/make_predict_fn/grid/run, BASELINE |
309     | `test_calibrate_local.py` | `eval.calibrate_local` local grid + crossing gate, leave-
one-video-out |
310     | `test_export_wasb_segments.py` | `eval.export_wasb_segments` segment/comparison CSV
schemas, mmss; export-> slicer reuse |
311     | `test_eval_training.py` | `eval.training` label_candidates / feature extraction /
build_training_set |
312     | `test_eval_classifier.py` | `eval.classifier.LogisticRegression`
fit/predict_proba/balanced/JSON round-trip |
313     | `test_eval_regression.py` | `eval.regression` baseline store/compare/gate; also
`@run_regression.py::main` (exit-code CI gate) |
314     | `test_distill_local.py` | `eval.distill_local` build_candidates / predict_rallies /
learned-vs-baseline |
315     | `test_gemini_refine.py` | `eval.gemini_refine` merge_overlaps / _offset_segments (pure
helpers, no API) |
316     | `test_hit_detector.py` | `pipeline/audio/hit_detector` (detect_hits, onset_envelope,
_moving_stats) + `eval/audio/e1_probe.cluster_hits` |
317     | `test_rally_slicer.py` | `tools.rally_slicer.compute_clip_windows` (pure) + label
parse |
318     | `test_video.py` | `utils.video` duration/chunk (subprocess mocked) |
319     | `test_geometry.py` | `utils.geometry` center/distance/velocity/normalised-velocity/IoU
|
320     | `test_database.py` | `infrastructure.database.Database`

```

```

schema/CRUD/migrations/`user_version`/`EXPECTED_CANDIDATE_COLUMNS` guard |
321 | `test_config.py` | `config` (load_config defaults, save_default_config, AppConfig
defaults pinned, `extra` tolerance, `.get()` dict-compat) |
322 | `test_reporting_harvest.py` | `reporting` (`emit_indexer_report`,
`harvest.record_run`); ? `importorskip("obsreport")`; all 7 footgun rules + plain-dict-OR-
AppConfig + `report.enabled=False` |
323 | `test_reporting_spec_faq.py` | `reporting/spec.yaml` <-> `README.md` cross-ref
integrity (every `faq_ref` resolves; actionable rules cite a `faq_ref`); ?
`importorskip("obsreport")` |
324 | `test_validators_base.py` | `pipeline/validators/base.ValidationRegistry`
(register/run_all/`run_all_with_context` -> `(results,context)`; exception captured not raised)
|
325 | `test_sport_profiles.py` | `sports` registry + BadmintonSportProfile (prompt contains
"decimal seconds"/chunk value; relevance config keys) |
326 | `test_providers.py` / `test_provider_gemini.py` | `providers` registry +
GeminiProvider lifecycle (`MAX_UPLOAD_MB` guard, non-list JSON -> [], cleanup delete) |
327 | `test_annotation_providers.py` / `test_automated_provider.py` | `annotations`
registry, FileProvider, AutomatedProvider (oracle/lineage); keystone one-scorer contract ->
`eval.metrics.score` |
328 | (no dedicated test) | individual `pipeline/validators/*` impls,
`pipeline/stitcher.compiler`, `main.py`, `run_process_and_stitch.py`,
`pipeline/detectors/wasb_runner`, `pipeline/segmenters/{wasb_hybrid,motion}`,
`eval/audio/e1_probe` (diagnostic) ?? exercised only indirectly |
329 | ? stale `.pyc` only | `tests/`__pycache__`/test_gemini_api.*` ? no live source exists |
330

```

13. user (2026-06-10T05:33:55.034Z)

No files found

14. user (2026-06-10T05:33:55.509Z)

No files found

15. user (2026-06-10T05:34:00.325Z)

```

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\EVALUATION.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\TRACKNET_WSL_SETUP.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\GOLDEN_SET_IMPLEMENTATION.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\GOLDEN_SET_PHASE1_VIDEOS.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\HOW_RALLY_DETECTION_WORKS.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\COMMERCIAL_READINESS_REVIEW.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\MONETIZATION_AUDIT.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DEFERRED_HARDENING_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PMF_MARKET_RESEARCH.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\ROADMAP.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\checkpoints\2026-06-eval-
baseline\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\checkpoints\2026-06-logging-
reporting\README.md
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\checkpoints\2026-06-scaffolding-complete\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\decisions\DECISIONS.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\past_projects\low-regret-
moves\LOW_REGRET_MOVES_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\past_projects\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\research\AUDIO_E1_FINDINGS.md
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\research\RALLY_DETECTION_RESEARCH.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\COMMERCIALIZATION.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM_ARCHITECTURE.md

```

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\BACKLOG.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\GOLDEN_SET_VENDORING.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\VIDEO_RECORDING_GUIDELINES.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\checkpoints\2026-06-strategy-foundation\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\quality-iterations\2026-06-initial\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\quality-iterations\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\NEXT_STEPS.md

16. user (2026-06-10T05:34:11.416Z)

```
1      # quality-iterations/
2
3      This directory holds completed phases of quality improvement work.
4
5      As we receive new labeled videos and iterate on model accuracy, recall/precision, audio
6      fusion, candidate quality, etc., each major iteration or batch of improvements will be captured
7      here once it reaches a terminal state (DONE / COMPLETED / REJECTED).
8
9      See `../README.md` (under archives/) for the overall archival policy.
10
11     ## Organization
12     - Dated subdirectories, e.g. `2026-06-initial/`
13     - Each iteration should include:
14       - A README.md with STATUS, trigger (new data, specific goals), key changes, and links
15       to relevant checkpoints under `../checkpoints/`.
16       - References to updated eval baselines, regression results, report enhancements, etc.
17
18     This keeps the live `docs/` clean while preserving a rich historical trail for
19     regression detection across quality work.
20
21     ## Iterations
22     - **`2026-06-initial/`** ? SEED. Opened the quality phase ahead of the awaited labeled-
23     video batch (status: in progress; superseded for first results by the iteration below).
24     - **`2026-06-human-gt-tuning/`** ? **COMPLETED (2026-06-08).** First eval against
25     **human** ground truth on a partially-labeled video: built the partial-label eval harness +
26     labeled-window methodology, produced the first trustworthy F1 (baseline 0.650 ? tuned 0.742,
27     *fit-on-self*), per-rally failure analysis, a config sweep, and ? via adversarial review ? found
28     + fixed a real ingestion bug (negative-start import abort). Cross-repo: `sports-data-collector`
29     PR #13.
30     - **`2026-06-multivideo-lovo/`** ? **COMPLETED (2026-06-08 ? extended to n=6,
31     2026-06-09).** Built a **6-match** human corpus (3 singles incl. Mahadevpura, 1 doubles, 2
32     multi-court; 3 cameras/frame-rates) + cross-video **leave-one-video-out**. Key finding: the
33     unsupervised windowing heuristic is **footage-fragile** ? it fails on **multi-court** footage
34     (background games; Kushagra 0.16 / gBaaddy 0.28) where the shuttle signal is present (97% of
35     rallies have motion) but rally-vs-dead-time isn't velocity-separable. **Prototyped a learned
36     per-frame segmenter** (`backend/eval/rally_seq_proto.py`): held-out AUC **0.83?0.92 on every
37     match, decoupled from the multi-court confound**. **The learned model OVERTOOK the heuristic at
38     n=3 and holds ~+0.10**: n=6 LOVO mean **learned 0.583 vs heuristic 0.480**. The **"contrast"
39     metric** (in-rally÷dead-time shuttle visibility) predicts heuristic F1 monotonically across 5
40     matches. Data-backed green light for `../research/OWNED_MODEL_TRAINING_STUDY.md`.
41
42
```

17. user (2026-06-10T05:34:11.768Z)

```

1      # Quality Iteration: 2026-06 Initial
2
3      **Date:** June 2026
4      **STATUS: IN PROGRESS** (seed for the phase starting with new labeled videos)
5
6      > This is a seed document. When the initial quality work with the mid-next-week labeled
video batch is complete, update the status to COMPLETED / DONE and capture the final state.
7
8      > **Update (2026-06-08):** the first concrete iteration landed ? see
9      > [../2026-06-human-gt-tuning/](../2026-06-human-gt-tuning/) (first human-GT eval +
tuning,
10     > driven by an owner *self-tagged* video rather than the awaited vendor batch). This
seed
11     > stays open for the broader multi-video LOVO phase.
12
13     ## Trigger
14     - Arrival of additional labeled videos (user hunting for more, expected mid-next week).
15     - Scaffolding now mature (archives/past_projects/low-regret-moves completed with PRs
#49?#51).
16     - Shift focus from structure/contracts to measurable quality improvement.
17
18     ## Goals for this iteration (initial)
19     - Ingest new golden data and expand calibration / cross-validation.
20     - Drive improvements in detector (WASB/TrackNet) windowing, rally gate, and candidate
quality using real data.
21     - Enhance audio signals in reports (building on the audio-readiness work from PR #50).
22     - Capture and baseline current logging + reporting framework (standardized in recent
work) and eval harness state.
23     - Establish regression detection practice via dated checkpoints.
24
25     ## Related Checkpoints (see ../checkpoints/)
26     - 2026-06-logging-reporting (standardization of logging in hot paths + reporting/harvest
+ obsreport integration)
27     - 2026-06-eval-baseline (current quality / eval metrics baseline, regression setup,
golden set state at start of iteration)
28
29     ## What "Done" looks like for this seed
30     - New videos processed and added to golden set / eval.
31     - Measurable improvement in P/R/F1 on held-out data or updated regression baseline.
32     - Updated COMMERCIAL_READINESS_REVIEW.md and BACKLOG.md reflecting quality phase
progress.
33     - New dated checkpoint(s) under archives/checkpoints/ capturing the post-iteration state
(including any new report rules, eval baselines, etc.).
34
35     When this phase completes, move the final snapshot of key living docs here or create a
follow-up dated iteration directory.
36

```

18. user (2026-06-10T05:34:19.214Z)

```

1      # Quality Iteration: 2026-06 First Human-GT Eval + Tuning
2
3      **Date:** 2026-06-08
4      **STATUS: COMPLETED** (first concrete iteration of the quality phase seeded by
../2026-06-initial/)
5
6      > First time the pipeline was measured against **human** ground truth on a full video
7      > region (all prior numbers were vs Gemini *silver* labels or a single short golden
clip).
8      > This iteration built the partial-label eval harness, produced the first trustworthy
9      > P/R/F1, tuned configs against the labels, and ? via adversarial review ? uncovered and
10     > fixed a real ingestion bug. Code lives on branch `feat/eval-human-labels` (indexer)
and
11     > `sports-data-collector` PR #13.
12

```

```

13 ---
14
15 ## Trigger
16 - The owner self-tagged ~10 min of a match in the VLC rally-annotator, producing the
17   first human golden CSV: `Annotation Setup/Collect/Adarsh and Avi.rallies.csv`
18   (40 rallies, schema `rally_number,start_time,end_time,ending_reason,sport`).
19 - The video is partially labeled: labels cover `0?605s` of a 23m29s video
20   (1920x1080, 59.94 fps, 84,480 frames). This made partial-label-aware evaluation a
21   first-class requirement, not an afterthought.
22
23 ## What was built
24 - backend/eval/eval_partial_labels.py (+ `tests/test_eval_partial_labels.py`, 4
tests):
25   scores WASB rally segments against a human CSV restricted to the labeled window
26   `[0, last_labeled_end]`. Reuses `calibrate_wasb.make_predict_fn` / `load_golden_gts`,
27   `rally_gate`, `metrics`, and `export_wasb_segments.build_comparison` ? no new
28   windowing/metric logic. Adds a --sweep grid-search mode.
29 - Methodology ? the labeled-window restriction (the load-bearing idea): with partial
30   labels, the detector's correct rallies in the un-labeled tail (605s?23min) would be
31   counted as false positives and crush precision. The harness drops predicted segments
32   outside `[0, last_end]` so precision/recall are honest within the labeled region.
33   A regression test pins this (`test_run_full_window_counts_tail_detection_as_fp`).
34 - WASB trajectory for the labeled window: first 610s ? 36,564 frames ? WASB
35   inference (~18 min) ? `~/clips/adarsh_avi_traj.csv` (+ `%TEMP%\adarsh_avi_traj.csv`).
36   Run script: `~/clips/run_wasb_avi.sh`.
37
38 ## Results ? first trustworthy human-GT numbers
39 Pipeline vs 40 human rallies, IoU ? 0.5, window `[0, 605]s`:
40
41 | operating point | TP | FN | FP | precision | recall | F1 | mIoU | start err | end
err |
42 |---|---|---|---|---|---|---|---|---|---|
43 | baseline (0.02 / 2.0 / 2.0) | 26 | 14 | 14 | 0.650 | 0.650 | 0.650 | 0.714 | 1.04s
| 1.26s |
44 | baseline + gate2 | 19 | 21 | 9 | 0.679 | 0.475 | 0.559 | 0.729 | 1.09s | 1.46s |
45 | tuned (0.05 / 1.0 / 1.0) | 33 | 7 | 33 | 0.500 | 0.825 | 0.623 | 0.741 | 1.01s | 0.75s
|
46 | tuned + gate2 | 23 | 17 | 6 | 0.793 | 0.575 | 0.667 | 0.761 | 1.05s | 0.82s |
47
48 Per-rally detail: `output/adarsh_avi_vs_human.csv`.
49
50 ### Tuning sweep (64 configs x gate {0,2})
51 Best by F1 (fit-on-self): velocity 0.03 / merge_gap 1.0 / min_dur 2.0 / gate OFF ?
52 F1 0.742 (P 0.673 / R 0.825, mIoU 0.744). Lifts the best default (0.667) by ~+0.075.
53
54 | lever | finding |
55 |---|---|
56 | `min_window_duration` | Dominant lever. Lowering it recovers short 3?5s rallies
(recall ?); the best point keeps it at 2.0 trading a little recall for precision. |
57 | net-crossing gate | Over-prunes this camera. Every gate-on config caps at ~0.68;
`baseline+gate2` drops recall 0.65?0.475 (loses 7 true positives). Likely net-axis estimation is
off for this camera angle. |
58 | `velocity_thresh` | Nearly irrelevant (0.01/0.02/0.03 within noise). |
59 | `merge_gap` | Best stays at 1.0; raising it to heal fragmentation merged adjacent
distinct rallies and hurt precision (see failure mode below). |
60
61 ### Failure modes (from the per-rally diff)
62 - Long rallies: caught well ? e.g. golden #18/#28/#30/#38/#39 matched at IoU
0.83?0.95,
63   boundaries within ~1s. The detector is genuinely strong here.
64 - Short rallies (3?5s): missed ? most of the 17 misses (golden #3/#5/#8/#17/#19/#25/
65   #35/#36/#37/#40). Lever: `min_window_duration`.
66 - Long rallies fragmented ? golden #1 (a 34.8s rally, 37.4?72.2) split into two
fires
67   (23.5?52.6 and 53.7?76.5), neither hitting IoU 0.5 ? reads as 1 miss + 2 FPs. A long

```

```

68     invisibility gap exceeds any sane `merge_gap`, so raising it doesn't help (it just
merges
69     real neighbors).
70     - **The "false positives" are mostly real-rally fragments**, plus one likely *un-tagged
71       early rally* at 23.5s (before the first human label at 37.4s) ? **not
hallucinations**,
72       so precision is recoverable by tuning, not a fundamental detector failure.
73
74     ## Bugs / errors uncovered (and fixed)
75     1. **[major] Negative `start_time` aborts the whole import** (collector
76       `src/cli/normalize_annotations.py`). The normalizer guarded blank/reversed-end rows
but
77       **not** negative start. `BadmintonRally.validate_timestamps` raises **start_time must
be
78       non-negative", which the `import-local` `try/except` turns into `sys.exit(1)` ? one
bad
79       row kills the whole video, the exact landmine the normalizer claims to close. **Found
by
80       an adversarial review agent** (it reproduced the abort), **fixed** with a drop branch
+
81       regression test (PR #13, commit `1fe1c24`).
82     2. **[minor] Doc misattributed the abort mechanism.** Original text said a reversed
83       `end <= start` makes `safe_float_required` raise. It does not ? that path raises only
on
84       *blank* values; the reversed-end and negative-start aborts come from the
`BadmintonRally`
85       **pydantic validator**. Corrected in the normalizer docstring and the positioning
doc.
86     3. **[nit] `ending_reason` mischaracterized as an enforced enum.** It is `Optional[str]`
87       (free-text with a documented vocabulary), not a validated `Enum`. Wording softened.
88     4. **[methodology] The labeled-window must not be *inferred* long-term.** The harness
89       defaults `window_end = max(end_time)`, which is **wrong when a video is tagged past
its
90       last rally** (valid predictions in the labeled-but-rally-free tail get scored as
FPs).
91       Captured as a P1 follow-up: the collector should persist `labeled_window_start/end`
in
92       the manifest. See `sports-data-collector/docs/MANUAL_INGESTION_POSITIONING.md`.
93
94     ## Infrastructure gotchas (for the next person)
95     - **WSL background commands mangle `$VAR` / `${}`. Passing a long multi-statement
96       `wsl -d Ubuntu bash -lc '?'` through the Bash tool with `run_in_background` expanded
inner
97       variables to empty (the job failed instantly with `mkdir: cannot create directory
''`).
98       **Fix:** put the job in a `.sh` file (strip CRLF into WSL with `tr -d '\r'`) and
background
99       `bash file.sh` ? no inner substitutions for the wrapper to corrupt.
100    - **`ls` / `find` / `du` race on a huge actively-written directory.** While ffmpeg wrote
101      36k frames, these intermittently reported `0` files. **Use** `ls -f <dir> | grep -c
jpg`
102      (or read `/proc/<pid>/fd`) for a reliable count; trust the job log over a directory
listing.
103
104    ## Cross-repo outcome
105    - **Collector PR #13** (`sports-data-collector`, branch `feat/manual-ingestion-
normalizer`):
106      `normalize_annotations.py` (the two-landmine + negative-start guard) + `docs/
107      MANUAL_INGESTION_POSITIONING.md`. Validated the full `normalize ? import-local ?
export`
108      chain on the real file against a temp DB ? 40-interval indexer CSV. Suite 124 green.
109
110    ## Honest caveats (do not oversell)
111    - **F1 0.742 is fit-on-self on ONE video ? optimistic.** The transferable number is
112      leave-one-video-out across **?3 distinct matches**. n=2 is the actual blocker for the

```

```

113     learned model beating baseline.
114 - **Breadth > depth for LOVO.** Chunks of one match are correlated (same court/lighting/
115     players); using them as separate "videos" leaks. Tag *different matches*, don't slice
one.
116
117     ## Reproduce
118     ```bash
119     # 1) trajectory (WSL): bash ~/clips/run_wasb_avi.sh    (extract 610s frames + WASB ~18
min)
120     # 2) eval across operating points + per-rally diff:
121     python -m backend.eval.eval_partial_labels \
122         --trajectory "%TEMP%/adarsh_avi_traj.csv" \
123         --labels "?/Adarsh and Avi.rallies.csv" --fps 59.94 --frame-width 1920
124     # 3) tuning sweep:
125     python -m backend.eval.eval_partial_labels --sweep --top 14 \
126         --trajectory "%TEMP%/adarsh_avi_traj.csv" \
127         --labels "?/Adarsh and Avi.rallies.csv" --fps 59.94 --frame-width 1920
128     ```
129
130     ## What's next (not in this iteration)
131     - Owner's 60-min tagging ? full-video re-run (extract full frames + WASB + eval; window
132     auto = last labeled end). **Spread tagging across 3?4 matches** for honest LOVO.
133     - Re-run the sweep across the multi-video manifest (`calibrate_local` LOVO) to get a
134     *transferable* operating point, then commit it as the new default.
135     - Investigate the gate's net-axis estimation on this camera (it over-prunes here).
136     - Collector P1: persist the labeled window + provenance/maturity in the manifest.
137
138     ## Related
139     - Seed: `../2026-06-initial/README.md`
140     - Checkpoints: `../checkpoints/2026-06-eval-baseline/` (baseline at phase start)
141     - Research: `../research/RALLY_DETECTION_RESEARCH.md`,
`../research/AUDIO_E1_FINDINGS.md`
142

```

19. user (2026-06-10T05:34:19.770Z)

```

1     # Quality Iteration: 2026-06 Multi-video LOVO + learned-segmenter prototype
2
3     **Date:** 2026-06-08
4     **STATUS: COMPLETED** (second concrete iteration; follows `../2026-06-human-gt-tuning/`)
5
6     > The session that turned "tune one match" into "is the *method* right across matches."
Built a 2-match
7     > human-labeled corpus, ran the first honest cross-video LOVO, uncovered that the
unsupervised windowing
8     > heuristic is **footage-fragile even when WASB recall is excellent**, and **proved with
a prototype that the
9     > rally signal is learnable** where the heuristic fails. Motivates + de-risks the owned
model
10    > (`../research/OWNED_MODEL_TRAINING_STUDY.md`).
11
12    ## Corpus built this iteration
13    | video | role | rallies | res / fps | trajectory |
14    |---|---|---|---|---|
15    | Adarsh-and-Avi (full) | train | **96** | singles · 1080p / 59.94 |
`~/clips/adarsh_avi_full_traj.csv` (84,480 fr) |
16    | Kushagra-Yash-Avi | train | 32 | singles · 4K?1920 / 59.94 |
`~/clips/kushagra_traj.csv` (32,232 fr) |
17    | **LargeTestVideo** | **train** | **49** | **doubles · 5.3K?1920 / 29.97** |
`~/clips/largetestvideo_traj.csv` (23,040 fr) |
18    | **gBaaddy (20 May)** | **train** | **47** | **1080p / 59.94 · multi-court** |
`~/clips/gbaaddy_traj.csv` (52,232 fr) |
19    | **TestLargeVideo** | **train** | **6** | **5.3K?1920 / 29.97 · short** |
`~/clips/testlarge_traj.csv` (3,019 fr) |
20

```

```

21      *(Grew to **n=5 distinct matches** over the session: 2 singles + 1 doubles + 2 more,
across 3 cameras/frame-rates,
22      incl. 2 multi-court ? see the n=3 and n=5 updates below.)*
23
24      All WASB trajectories generated on one 8 GB GPU, **serialized via file-based wait-
loops** (`run_wasb_.sh`;
25      GPU never double-booked, race-free via a `kushagra_traj.csv`-exists gate). Manifest:
`output/human_lovo_manifest.json`.
26
27      ## Result 1 ? full-match Adarsh (96 rallies, IoU?0.5)
28      baseline F1 **0.632**, tuned+gate2 **0.727** (P0.80/R0.67), sweep ceiling **0.750**
(vel0.02/merge1.0/min_dur2.0/gate off).
29      vs the opening-10-min slice (40 rallies, prior iteration): 0.667 / 0.742. **Best-config
region is identical across the
30      slice and the full match** (merge_gap 1.0, min_dur 2.0; velocity irrelevant) ? tuning is
stable. Per-rally:
31      `output/adarsh_avi_full_vs_human.csv`.
32
33      ## Result 2 ? first cross-video LOVO (`calibrate_local`, the number to trust)
34      ```
35      per-video best-fit:  adarsh 0.777   |   kushagra 0.163 (ceiling ? NO config beats it)
36      leave-one-video-out: baseline mean 0.316 ? calibrated 0.443 ± 0.279 (overfit gap
+0.027, tiny)
37      held-out adarsh    0.722   (config tuned on Kushagra ? Adarsh's own optimum ? config
TRANSFERS)
38      held-out kushagra 0.163   (config tuned on Adarsh = Kushagra's own ceiling)
39      ```
40      **The asymmetry is the finding.** Tuning generalizes fine *where the heuristic can solve
the video*
41      (Kushagra?Adarsh = 0.722). But **Kushagra is unsegmentable by *any* velocity-windowing
config (0.163)** ? and
42      this is NOT a WASB-recall failure: WASB tracks the shuttle **better** on Kushagra than
Adarsh (**97% of GT
43      rallies contain ?2 net-crossings**, median 5, 310 visible pts/rally, vs Adarsh 76%). The
likely mechanism:
44      Kushagra's shuttle is visible 67.5% of the time (vs Adarsh 40%) ? tracked *through*
inter-rally activity ? the
45      "shuttle-is-moving" heuristic can't place clean boundaries (mean boundary error ~5 s).
**The ±0.279 spread is
46      method variance, not config.** This also **corrects the earlier "~0.73 baseline"** ?
that was fit-on-self; the
47      honest cross-video heuristic bar is **~0.44.**
48
49      ## Result 3 ? learned per-frame prototype (the proof) ?
`backend/eval/rally_seq_proto.py`
50      A dependency-light prototype (numpy + scipy + the repo's pure-Python
`LogisticRegression`): 6 hand-built per-frame
51      features over the WASB trajectory ? visibility rate, mean/max shuttle speed, **net-
crossing rate**, x/y spread ?
52      labeled rally(1)/non-rally(0), **honest leave-one-video-out** (train on one match, test
the other; split by match).
53
54      | held-out | per-frame AUC | learned F1 (real LOVO) | learned F1 (oracle threshold) |
heuristic ceiling |
55      |---|---|---|---|---|
56      | Adarsh | 0.878 | 0.704 | 0.721 | 0.73 |
57      | **Kushagra** | **0.841** | 0.049 | **0.578** | **0.163** |
58
59      **The signal is learnable.** Held-out per-frame AUC is **0.84?0.88** ? including on
Kushagra (trained only on
60      Adarsh), the match the heuristic *cannot* segment. With a calibrated threshold the
learned model hits
61      **Kushagra F1 0.578 ? 3.5× the heuristic's 0.163 ceiling.** The naive cross-video
threshold gives 0.049, so the
62      remaining gap is **threshold/operating-point calibration at n=2, not representation**

```



```

(the probability scale
63     shifts across footage; fixed by more videos / per-video calibration / a scale-invariant
sequence model).
64
65     > This directly answers the owned-model study's "dominant empirical unknown" ? *can a
low-dim WASB trajectory
66     > feed a learned temporal segmenter?* **Yes (AUC 0.84).** The prototype is the seed of
the study's **Gen-0
67     > (ASFormer-over-WASB-features)**. ? It is a *directional* proof, not production: n=2,
linear model, hand
68     > features, threshold-transfer unsolved. The real ship gate is a segment-level A/B on
the golden set at n?5
69     > (needs the GPU/WSL machine ? not runnable in this dev container).
70
71     ## Update ? n=3: doubles match added ? **the crossover** (the final number)
72
73     Added **LargeTestVideo** (doubles, 5.3K?1920, **29.97 fps**, 49 rallies, fully tagged) ?
a different play
74     type *and* camera *and* frame rate. Single-video: heuristic best F1 **0.750** (tuned, R
0.918), best-fit 0.827
75     ? so **doubles is heuristic-friendly; Kushagra remains the lone heuristic failure.**
Per-rally:
76     `output/largetestvideo_vs_human.csv`. Manifest now n=3
(`output/human_lovo_manifest.json`).
77
78     **Honest cross-match LOVO, heuristic vs learned, n=2 ? n=3:**
79
80     | held-out match | heuristic LOVO | learned LOVO (AUC) | learned LOVO F1 |
81     |---|---|---|---|
82     | Adarsh (singles) | 0.725 | 0.878 | 0.701 |
83     | **Kushagra (singles, the hard one)** | **0.151** | 0.850 | **0.417** |
84     | LargeTest (doubles) | 0.755 | **0.914** | 0.729 |
85     | **mean held-out F1** | **0.544 ± 0.278** | ? | **0.615** |
86     | *(same metric at n=2)* | *0.443* | ? | *0.377* |
87
88     **Findings (the data-backed case, honestly):**
89     1. **The learned model OVERTAKES the heuristic at n=3** (mean **0.615 vs 0.544**); at
n=2 it *trailed*
90     (0.377 vs 0.443). The crossover the plan predicted ? the learned path **scales with
data**, the heuristic
91     does not.
92     2. **Both means rose with n, for opposite reasons.** The heuristic rose only because the
new match (doubles)
93     happens to suit it; it **still cannot rescue Kushagra at any n** (held-out 0.151,
capped). The learned model
94     rose because **more data closed its calibration gap** ? Kushagra's held-out F1 went
**0.049 ? 0.417** just
95     from one added match (oracle ceiling 0.593, AUC 0.850 ? it will keep climbing with
n).
96     3. **The decisive edge is on the *hard* footage** ? where the heuristic fails
(Kushagra), the learned model is
97     **2.8× better and rising** ; on the easy matches (Adarsh, doubles) they're comparable
(~0.72?0.75). The owned
98     model's value is exactly the generalization the heuristic lacks.
99     4. **Generalizes across play type** ? the **doubles** match, held out and trained only
on the **two singles**
100     matches, scores AUC **0.914** / F1 0.729. Different player count, camera, and frame
rate, and the per-frame
101     rally signal still transfers. (Notably *despite* the prototype's speed features not
yet being fps-normalized
102     ? 30 vs 60 fps ? which is a known refinement that should only help.)
103
104     ### Final quality metrics (n=3, honest leave-one-video-out)
105     - **Heuristic (current pipeline):** mean held-out F1 **0.544 ± 0.278** ? the ±0.278 is
the footage-fragility

```

```

106      (Kushagra ~0.15 vs others ~0.75); **hard-capped, does not improve on the videos it
can't represent.**
107      - **Learned prototype (owned-model seed):** mean held-out F1 **0.615**, per-frame **AUC
0.85?0.91**, **~0.24
108      for one added match and still climbing.** This is a *directional* number (n=3, linear
model, hand features,
109      fps-norm + per-video calibration still on the table) ? the trend, not the absolute, is
the signal.
110
111      **Conclusion (reassures the plan ? with data, not optimism):** the owned/learned path
has **already overtaken
112      the heuristic at n=3 and improves with every match**, while the heuristic is **ceilinged
by footage it cannot
113      represent**. The bottleneck is confirmed to be **data scale + calibration (n?5), not
method.** Reusing every
114      labeled video for training is the right strategy ? each one lifts the learned model and
none can fix the
115      heuristic's hard cases. ? Proceed with `../../research/OWNED_MODEL_TRAINING_STUDY.md`
and
116      `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (Gen-0 first), gated on n?5 + owner sign-off.
117
118      ## Update ? n=5 + the MULTI-COURT root cause (the defining finding)
119
120      Added **gBaaddy** (47 rallies, 1080p/60) + **TestLargeVideo** (6 rallies, 5.3K/30 ? *too
short to be a reliable
121      fold*). Corpus now n=5. Single-video heuristic: gBaaddy **0.277**, TestLarge 0.316 ?
both low. gBaaddy is 1080p/60
122      *like Adarsh* yet fails, so it's **not** a resolution/fps issue.
123
124      ### Root cause ? confirmed with data (owner's insight): background games
125      WASB tracks **every** shuttle in frame, including **other courts**. Videos with
background games have high shuttle
126      visibility *during the foreground game's dead time* ? the velocity-windowing heuristic
can't place clean boundaries.
127      The ***"contrast"*** metric (in-rally ÷ dead-time shuttle visibility) cleanly predicts
heuristic success:
128
129      | match | heuristic best F1 | in-rally vis | **dead-time vis** | contrast | court |
130      |---|---|---|---|---|---|
131      | Adarsh (singles) | 0.73 | 69% | 18% | **3.9** | single |
132      | LargeTest (doubles) | 0.75 | 78% | 22% | **3.6** | single |
133      | Mahadevpura (singles, +n=6) | 0.67 | 74% | 33% | **2.2** | borderline (some
background) |
134      | gBaaddy | 0.28 | 78% | **41%** | 1.9x | **multi (background games)** |
135      | Kushagra | 0.16 | 82% | **57%** | 1.4x | **multi (background games)** |
136
137      The contrast metric predicts heuristic F1 **monotonically across 5 matches** (3.9x?0.73
? 1.4x?0.16) ? Mahadevpura's
138      intermediate 2.2x ? 0.67 lands exactly on the curve. ?3.6x ? heuristic works; ?1.9x ?
fails; ~2x ? borderline. **Academies are multi-court by definition**, so Kushagra/gBaaddy are
the
139      **most market-representative** videos ? "focus on the prominent game" is a core product
requirement, not an edge case.
140      **Lever = foreground-court isolation** (spatial gate to the prominent court before
windowing). The dead-time
141      detections are laterally offset (side courts), but isolation needs **real court-region
detection** ? a naive centroid
142      is muddled because the foreground shuttle also rests low between points. Near-term, no
new model; also a *design
143      requirement* for the e2e owned model (which would learn foreground attention natively).
144
145      ### Learned prototype LOVO (n=5)
146      | held-out | per-frame AUC | learned F1 (LOVO) | oracle F1 | heuristic best |
147      |---|---|---|---|---|
148      | Adarsh (singles) | 0.879 | 0.709 | 0.722 | 0.73 |

```

```

149 | Kushagra (multi) | 0.853 | **0.561** | 0.633 | 0.16 |
150 | LargeTest (doubles) | 0.915 | 0.713 | 0.756 | 0.75 |
151 | gBaaddy (multi) | 0.833 | **0.574** | 0.574 | 0.28 |
152 | TestLarge (6 rallies) | 0.726 | 0.286 | 0.471 | 0.32 |
153 | **mean** | ? | **0.568** | ? | ? |
154
155 - **Recovers BOTH multi-court videos** (Kushagra 0.561, gBaaddy 0.574 vs heuristic
0.16/0.28) ? the learned model's
156 spatial-spread features (`x_std`/`y_std`) help discount background courts.
157 - **Kushagra keeps climbing with n:** 0.049 ? 0.417 ? **0.561**.
158 - ? **Honest mean nuance:** the learned mean *dipped* 0.615 (n=3) ? 0.568 (n=5) ?
**entirely the 6-rally TestLargeVideo
159 fold** (F1 0.286, far too short to read). **Excluding it: 0.639** ? above n=3. The
mean is a blunt tool across folds
160 ranging 6?96 rallies; the **per-video learned-vs-heuristic gap on the hard videos is
the real signal**, and it's decisive.
161
162 ### Final quality metrics & conclusion (n=6)
163 **Honest cross-video LOVO mean, heuristic vs learned, by corpus size:**
164
165 | | n=2 | n=3 | n=5 | **n=6** |
166 |---|---|---|---|---|
167 | heuristic (`calibrate_local`) | 0.443 | 0.544 | 0.463 | **0.480 ± 0.242** |
168 | learned prototype | 0.377 | 0.615 | 0.568 | **0.583** (~0.64 excl. 6-rally fold) |
169
170 - **The learned model has led since n=3 and holds ~+0.10** (n=6: learned 0.583 vs
heuristic 0.480). It stays robust as
171 the corpus diversifies; the heuristic stays capped by the multi-court videos it can't
represent (per-fold n=6: Adarsh
172 0.751, doubles 0.737, Mahadevpura 0.657, but Kushagra **0.157** / gBaaddy **0.309**).
173 - **n=6 added MahadevpuraSingles** (borderline multi-court, contrast 2.2×): held out and
trained on the *other 5*, the
174 learned model hit **AUC 0.902 / F1 0.712** (vs heuristic 0.657) ? clean generalization
to an unseen match.
175 - **The decoupling is the headline:** learned held-out **AUC is 0.83?0.92 on every match
*regardless of contrast**,
176 while the heuristic swings 0.16?0.75 *with* contrast. The learned model has
**decoupled from the multi-court problem**
177 the heuristic can't escape.
178 - **The #1 near-term lever is the multi-court confound** (foreground-court isolation) ?
both the heuristic's biggest
179 failure and the product's core requirement. Bottleneck = **data scale + calibration +
multi-court isolation, not method.**
180
181 ## Heuristic discovery for multi-court (what we tried, what's promising)
182
183 Motivated by the contrast finding, two label-free trajectory pre-filters were prototyped
+ measured at the tuned
184 operating point across all 5 videos (scripts not committed ? discovery):
185
186 | candidate | multi-court ? (Kushagra/gBaaddy) | clean ? (Adarsh/doubles) | verdict |
187 |---|---|---|---|
188 | **Foreground spatial gate** (keep dominant 2D-density court region) | +0.03 / +0.03 |
**?0.13 / ?0.12** | ? net ?0.04 ? helps multi-court a little but removes legit wide foreground
points on clean videos |
189 | **Track-continuity filter** (drop implausible position jumps) | 0.00 | 0.00 | ? no-op
? the confusion isn't transient jumps; WASB tracks a court coherently, switching at gaps |
190
191 **Conclusion:** simple single-cue trajectory heuristics **cannot** isolate the
foreground court ? courts aren't
192 separable by raw density, and the learned model already handles multi-court better (AUC
0.83) by combining cues.
193
194 **Audio for foreground discrimination ? TESTED (2026-06-08), WEAK ?.** Hypothesis: the
GoPro mic sits at the

```

195 foreground court, so foreground hits are loud/near and background games quiet/distant ?
audio energy gates "is the
196 prominent game active?" ? the signal the visual trajectory can't give. **Probed on the
2 multi-court videos + a clean
197 control** (extract audio ? onset-flux + RMS energy, AUC of in-rally vs dead-time):
onset/RMS AUC only **0.53?0.65**;
198 audio contrast 1.1?1.9× (below the visual contrast). High-frequency bands (8?22 kHz @ 48
kHz) raise the *contrast*
199 (Kushagra 1.1?1.7×, gBaaddy 1.3?1.9× ? confirming HF-attenuates-with-distance physics)
but **AUC stays ~0.6**. **Why it
200 fails:** "dead time" isn't quiet ? the FOREGROUND court is loud during dead-time too
(footwork, talk, shuttle pickup) +
201 gym reverb + background games, so audio loudness ? *court occupancy*, not *rally*. Same
wall as E1 (~2.2× ceiling).
202 **Verdict: audio alone is NOT the multi-court gate.** (A learned timbre/rhythm model or
audio *fused into* the learned
203 segmenter might add marginal value, but the visual learned model already hits AUC 0.85
on these videos.)
204
205 **Court-region detection ? SMOKE-TESTED (2026-06-08), LOW HEADROOM ?.** Feasibility
probe: of the DEAD-TIME shuttle
206 detections, what fraction fall OUTSIDE the foreground rally region (= separable
background courts a court gate could
207 remove)? **Only 12?14% on the multi-court videos** (Kushagra 12%, gBaaddy 14%) ?
**86?88% of the dead-time confusion is
208 INSIDE the foreground court region.** A *perfect* court gate lifts contrast only to
~1.6?2.2× (Kushagra 1.4?1.6, gBaaddy
209 1.9?2.2) ? still below the ?3.6× "works" threshold. So the confusion is **not**
spatially-separable background courts;
210 it's the **foreground court's own between-point activity** (footwork, serves, shuttle
pickup) and/or background courts
211 that overlap the foreground image area. ? **Court-region detection (heuristic OR a
dedicated model) would NOT
212 meaningfully help.** (Doesn't need an own model ? but isn't worth building either.)
213
214 ### The verdict on heuristics: it's a TEMPORAL problem, only the learned model solves it
215 Three heuristic ideas were tested this session and all came up short ? **foreground
spatial gate** (net ?0.04),
216 **audio energy** (AUC ~0.6), **court-region separability** (12?14% out-of-region, low
headroom). They fail for the SAME
217 reason: the multi-court dead-time confusion is **rally-vs-non-rally activity in the
same place and frequency band**, not
218 a spatial or loudness difference. Distinguishing it needs the learned model's
temporal/structural features (sustained
219 net-crossing rhythm, activity duration, spatial-spread dynamics) ? which is exactly why
the prototype already hits **AUC
220 0.83** on Kushagra/gBaaddy where every hand-crafted heuristic fails. **There is no cheap
heuristic shortcut; the learned
221 segmenter is the path.** (A multi-feature heuristic combining cross-rate + speed +
spread would just be re-deriving the
222 prototype by hand ? so prefer the prototype.)
223
224 ## Learnings (carry forward)
225 - **The bottleneck is the method + data, not WASB recall or labels.** WASB delivers
strong shuttle signal on both
226 matches; the unsupervised heuristic extracts it on one and not the other. A learned
model does (AUC 0.84).
227 - **Tuning is real but footage-bounded** ? `merge\_gap 1.0 / min\_dur 2.0` transfers; the
gate trades recall?precision;
228 velocity is ~irrelevant. But no config rescues a video the heuristic class can't
represent.
229 - **Calibration, not representation, is the n=2 wall** ? learned probabilities separate
(AUC 0.84) but the operating
230 point doesn't transfer across footage with one training video. **n?5 distinct matches
is the highest-ROI unblock.**

```

231     - **Group by match, not file** for any cross-val (Adarsh-main + Adarsh-last-game = one
unit; `calibrate_local` has no
232     grouping, so keep one entry per distinct match).
233     - **Reuse the labels** ? Kushagra's labels + trajectory are valuable *training data for
the learned model* even though
234     the current pipeline scores 0.16 on it. Validated empirically.
235
236     ## Reproduce
237     ```bash
238     # trajectories: bash ~/clips/run_wasb_{avi_full,kushagra,gbaaddy}.sh    (serialized on
one GPU)
239     python -m backend.eval.eval_partial_labels --trajectory
"%TEMP%/adarsh_avi_full_traj.csv" \
240     --labels "?/Adarsh and Avi.rallies.csv" --fps 59.94 --frame-width 1920      #
full-match + --sweep
241     python -m backend.eval.calibrate_local --manifest output/human_lovo_manifest.json
--metric f1 # heuristic LOVO
242     python -m backend.eval.rally_seq_proto --manifest output/human_lovo_manifest.json
# learned prototype
243     ```
244
245     ## What's next
246     - **? Foreground-court isolation experiment** (the multi-court fix) ? spatially gate the
trajectory to the prominent
247     court before windowing; needs real court-region detection (densest player/shuttle
cluster, court-line detection, or
248     largest-court region), NOT a naive centroid. Would likely fix the heuristic on
Kushagra/gBaaddy AND clean the learned
249     features. Near-term, no new model. Highest-value lever given the academy/multi-court
target env.
250     - **Keep growing the corpus** (more varied matches tomorrow) ? re-run LOVO + prototype;
per-video threshold calibration
251     as n grows. Add fps-normalization to the prototype's speed features.
252     - Build the corpus format + the ASFormer Gen-0 A/B (owner sign-off; per
`../../research/OWNED_MODEL_TRAINING_STUDY.md`
253     + `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`, after scaffolding).
254
255     ## Related
256     - Prior iteration: `../2026-06-human-gt-tuning/` . Study:
`../../research/OWNED_MODEL_TRAINING_STUDY.md`
257     - Code: `backend/eval/{eval_partial_labels,calibrate_local,rally_seq_proto}.py` .
Checkpoints: `../../checkpoints/2026-06-eval-baseline/`
258

```

20. user (2026-06-10T05:34:26.747Z)

```

1     # Rally-detection methods ? research (2026-06-06)
2
3     Deep-research survey (23 sources, 110 claims ? 25 adversarially verified, 6 refuted, 12
final findings)
4     on alternative ways to find rally start/end in racket-sport video, for our **local-
first, permissive,
5     single-GPU** pipeline. Question: is Gemini-teacher ? local-distillation the best, or are
there competitive/
6     complementary methods?
7
8     ## Verdict
9     Teacher?student distillation is **sound but NOT uniquely best**. The evidence points to
a **complementary
10    multi-cue LOCAL stack**. VLM/MLLM teachers are validated specifically as **training-time
priors**, not runtime
11    localizers ? which is exactly our offline-Gemini framing. The single highest-leverage
near-term lever is to
12    **add a fully-local AUDIO hit-detection cue and fuse it with the shuttle trajectory**;
pose/court serve-gating

```

```

13     is the strong #2.
14
15     ## Three fully-local, permissive, competitive method families
16     1. **Audio hit detection (shuttle "thwack") ? top cheap lever.** Energy-peak + small-CNN
gives sub-ms impact
17         timing (peak precision ~0.98, recall ~0.95+); classical MFCC+GMM is CPU-light. ? Only
as good as its
18         post-processing: raw peaks ~41250 F-score ? **likelihood-ratio confidence raises
audio-only to ~68277**;
```

19 fully-automatic on-court systems degrade (~85%, and one tennis system hit 46% precision). Mostly validated

20 on tennis/table-tennis (louder balls) ? shuttle SNR transfer is an open risk.

```

21     2. **Pose + court-geometry state machine** (= our "gate B" idea, validated). Serve-ready
stance (court+pose
22         features, BiLSTM/CNN over a 12x26 hand-engineered matrix) ? rally **START**; shuttle-
flight gradient
23         (?80% consecutive no-motion frames) ? rally **END**.
```

Cheap (engineered features, no big model).

```

24     3. **Compact HitNet GRU** (MonoTrack, ~16K params, 2-layer GRU over 12-frame window of
court corners + both
25         players' poses + shuttle xy) ? consumes exactly our available inputs; trivially
runnable. Detects *strokes*
26         (a segmentation primitive), not whole rallies.
27
28     ## Key supporting evidence
29     - **Audio+visual fusion beats either alone:** tennis HMM rally **recall 83% (fused) vs
54% visual / 63% audio**
30         at ~unchanged precision; soccer late-fusion +427% mAP. This is the core case for
adding audio.
31     - **Distillation is legit:** MLLM4WTAL uses the MLLM as a training-time prior only;
SSL+KD (COMEDIAN) is
32         top-tier action-spotting. Our teacher?student is a valid backbone.
33     - **Our windowing is "coarse action spotting" (5260 s tolerance), not frame-exact PES**
? don't over-engineer
34         for frame precision.
35
36     ## ? Caveats (honest)
37     - **Domain transfer:** nearly all high numbers come from **BWF/tennis BROADCAST**
footage (replays, score
38         graphics, director cuts) or curated lab audio. Our target is single-fixed-camera
amateur video ? broadcast
39         CNNs (97299%) and scoreboard-OCR are inapplicable/degrade.
40     - **Metric mismatch:** headline numbers are **per-frame / per-component accuracy, NOT
IoU-0.5 boundary F1**.
```

41 The "~95% beats your 0.44" comparison was **refuted (0-3)** ? not comparable.

```

42     - **License gap (hard product req):** ShuttleSet, MonoTrack/HitNet weights, and WASB
*weights* commercial
43         terms were NOT confirmed ? must verify before commercial use. (WASB code = MIT;
weights = academic-trained.)
44     - Scoreboard/score-overlay OCR: usable only when a stable on-screen score exists ?
mostly N/A for amateur clips.
45
46     ## Datasets
47     - **ShuttleSet** ? 104 sets / 3,685 rallies / 36,492 strokes / 44 BWF matches (2018221),
18 shot classes +
48         hitting/player locations. Stroke-level structure for train/eval, but broadcast-domain
+ license TBD.
49
50     ## Ranked recommendation (local-first, permissive, single-GPU)
51     Gemini-teacher ? local-distillation is a valid **backbone to AUGMENT, not rely on
alone**. Cheapest local adds,
52     in order: *(1) audio hit detection fused with trajectory** (attacks the WASB recall
bottleneck with inputs we
53     already have); *(2) pose/court serve-ready START + flight-gradient END gating**;
```

*(3) a compact HitNet-style

```

54     GRU** on pose+trajectory+court. All MIT/permissive-compatible and CPU/GPU-light. Keep
Gemini offline-only.
55
56     ## Concrete cheap next experiments (with what we already have)
57     - **E1 ? audio recall probe:** extract audio ? energy-peak detection (high-pass + EMA
threshold) ? candidate
58         impact timestamps ? overlay hit density on existing WASB windows ? does audio recover
rallies WASB misses?
59         (zero new training; directly tests the recall bottleneck.)
60     - **E2 ? audio FP suppression:** add likelihood-ratio confidence (raw peaks ~41?50 F ?
~68?77 F).
61     - **E3 ? fuse into the distillation student:** add audio-hit-rate / time-since-last-hit
/ applause-energy as
62         features to the logistic-regression student ? re-eval at IoU 0.5.
63     - **E4 ? pose-free serve gate:** use the existing person detector for a coarse "both
players settled in court"
64         gate at window boundaries (proxy for the pose state machine, no pose model needed).
65
66     ## Open questions
67     1. ShuttleSet / MonoTrack-HitNet / WASB-weights **commercial license** terms (hard
requirement).
68     2. Does shuttle-"thwack" audio detection hold on **real amateur single-camera**
badminton (weaker SNR, gym
69         reverb), or degrade below the tennis ~73?77 F audio-only ceiling?
70     3. On OUR footage at IoU 0.5, do audio/pose cues actually fix the **WASB recall**
bottleneck ? or does WASB
71         miss the underlying trajectory regardless of how windows are derived?
72     4. Most label-efficient path given n=2: more Gemini labels vs audio/pose weak-
supervision vs a small
73         hand-labeled golden set ? biggest IoU-0.5 lift per labeling hour?
74
75     ## Sources (primary)
76     Springer 978-3-031-69073-0_15 (+ SN CompSci 6(5):447) · MonoTrack arXiv:2204.01899 ·
arXiv:2409.11760 (audio
77         peaks) · Kijak tennis AV-HMM (irisa) · ShuttleSet arXiv:2306.04948 · broadcast-CNN
arXiv:2308.12199 /
78         2307.16000 · MLLM4WTAL arXiv:2411.08466 · action-spotting survey arXiv:2505.03991 ·
WASB-SBDT (github nttcom)
79         · CoachAI license (github wywyWang).
80

```

21. user (2026-06-10T05:34:32.724Z)

```

1     # Audio cue (E1) ? quantitative findings & reusable lessons
2
3     **Date:** 2026-06-07 · **Status:** decision recorded (classical-onset audio NOT adopted
for fusion).
4     Companion to ADR-007 and `AUDIO_RALLY_DETECTION_PLAN.md` §10. Purpose: record the
**gains and losses
5     quantitatively** so we don't repeat the dead ends when adding the next signal
(pose/stance, etc.) or
6     when building the Option-B end-to-end model.
7
8     ## TL;DR
9     On real GoPro badminton audio, a classical onset hit-detector has a **strong recall
signal but a precision
10    ceiling of ~2.2x** ? too weak to be a clean rally cue. **Band-pass filtering did not
help.** Net: keep the
11    cheap, tested detector; **do not build audio fusion on the classical onset cue**; the
only path past ~2.2x is
12    a *learned timbre classifier* (E2), which needs labels. The E1 gate returned this
verdict **before** any fusion
13    was built ? a few hours, no wasted downstream work.
14
15    ## Quantitative scorecard (existing GoPro audio vs Gemini **silver** labels)

```

```

16
17     **GAINS ? what audio gave us:**
18     | Signal | Adarsh (36 rallies, 676 s) | TestLargeVideo (7 rallies, 101 s) |
19     |---|---|---|
20     | Rally coverage (rallies with ?2 detected hits) | **100%** (36/36) | **100%** (7/7) |
21     | WASB-missed rallies that contain an audio hit-cluster (recall-recovery) | **100%**
(14/14) | **100%** (5/5) |
22     | Hits detected (@k=3.5) | 486 | 94 |
23
24     **LOSSES / LIMITS:**
25     | Metric | Adarsh | TestLargeVideo |
26     |---|---|---|
27     | Discrimination = in-rally ÷ dead-time hit density (fair, mid-match) | **~2.2x** |
**~2.3x** |
28     | ?at k=2.5 / best-coverage k=4 | 1.3x / 2.8x (coverage drops to 30/36) | 1.5x |
29     | Audio-only proposal F1 (P / R) | **0.27** (0.22 / 0.36) | **0.13** |
30     | WASB-only F1 (reference baseline) | 0.54 | 0.33 |
31     | Band-pass effect on discrimination | none / negative (HF bands worse) | ? |
32     | Sensitivity cliff | k=6 ? ~0 hits (43 ? 0.6 hits/min) | ? |
33
34     What we built/ran (cost): `backend/pipeline/audio/hit_detector.py` (ffmpeg?numpy
spectral-flux onset?adaptive peak,
35     no scipy) + `backend/eval/audio/e1_probe.py` + band*k sweep + warm-up/label-artifact
check. ~209 tests; one
36     session; no fusion built (gated correctly).
37
38     ## Root cause of the ceiling
39     Between-point activity ? the loser tossing the shuttle back (literally our false-
fire), footwork, and talk ?
40     produces transients a classical onset detector cannot distinguish from rally hits. The
signal is band-agnostic
41     (HF "shuttle-click" bands gave lower discrimination, not higher), so the separation
problem is timbre, not
42     frequency band ? a filter can't fix it; a learned classifier might.
43
44     ## Caveats on the numbers
45     - Labels are Gemini silver, not human-gold ? dead-time may include unlabeled
activity, so ~2.2x is a plausible
46     floor. But the warm-up check (mid-match dead-time alone is ~0.6 hits/s vs ~1.3 in-
rally on both clips) says
47     human-gold is unlikely to move it much.
48     - Only 2 videos, 1 GoPro setup. Re-confirm on human-gold when available.
49
50     ## Generalizable lessons ? apply to the NEXT cue (pose/stance) and Option B
51     1. Gate before build. A cheap probe on real target data against existing labels
gave a decisive go/no-go
52     before any fusion code. Make every new signal earn an E1-style probe first.
53     2. Test on the real distribution, not the literature. The research ranked audio #1
(from broadcast/tennis);
54     on amateur GoPro badminton it underdelivered ? the exact SNR/domain caveat the
research flagged. Lit rankings
55     are hypotheses; the probe is ground truth.
56     3. Discrimination ratio (in-class ÷ out-of-class signal density) is a fast, label-
light cue-quality screen.
57     Compute it first. Rule of thumb from this run: >>2x to be worth fusing; ~2x =
weak.
58     4. Separate the recall role from the precision role. Audio was excellent at recall
(100% coverage) and poor
59     at precision ? a cue can be one and not the other. Decide its job before fusing.
60     5. Band ? timbre. Frequency band-passing can't separate same-band, different-timbre
sounds. If discrimination
61     is band-agnostic, the lever is a learned timbre model, not a filter.
62     6. Silver for the gate, gold for the verdict. Gemini silver labels were enough to
reach go/no-go cheaply;
63     reserve scarce human-gold for confirmation.

```



```

64     7. **Keep the cheap knob, record the negative.** We kept the `band=` param (may help
other mics/human-gold) but
65     documented it didn't help here, so we don't re-explore it.
66     8. **The recurring blocker is golden labels, not cleverness.** Past ~2.2× needs a
learned classifier (labels) ?
67     echoing the distilled-model finding (n=2 too few). Across cues, *more golden labels*
is the highest-leverage move.
68
69     ## Decision recorded
70     - **Classical-onset audio: NOT adopted for fusion now.** Detector + E1 probe kept
(merged, tested).
71     - **Next audio step:** E2 learned **timbre** classifier *iff* labels make it cheap; else
re-prioritize.
72     - **Sequenced behind golden-label generation** (owner self-rating via the VLC
annotator). The next session resumes
73     on either the **E2 audio classifier** or **golden evals** (owner's pick).
74

```

22. user (2026-06-10T05:34:32.947Z)

```

1     # Audio-based rally detection ? detailed plan
2
3     **Status:** Accepted direction (ADR-007) · experiments gated on golden eval labels ·
**Date:** 2026-06-07
4     **Decision in one line:** add a fully-local **audio "thwack" hit-detection cue, fused
with the WASB shuttle
5     trajectory**, as the next rally-detection lever ? because it directly attacks our recall
bottleneck using an
6     input we currently throw away. Pose/stance detection is the parked #2 (see ADR-007 /
[research](RALLY_DETECTION_RESEARCH.md)).
7
8     ## 1. Why audio (grounded in the research)
9     - **#1 cheap local lever** in the verified survey (`docs/RALLY_DETECTION_RESEARCH.md`).
Audio+visual **fusion
10     lifts rally RECALL** ? tennis HMM: 83% fused vs 54% visual-only / 63% audio-only at
~unchanged precision ?
11     and recall is exactly where WASB-derived windows fail us (IoU-0.5 candidate recall
ceilinged at ~0.43 on
12     TestLargeVideo).
13     - **Fully local, permissive, CPU-light** (classical MFCC/energy methods; no restrictive
weights) ? keeps the
14     privacy lever and adds ~zero marginal cost.
15     - **Uses an input we currently ignore.** Our GoPro clips already carry audio.
16
17     ## 2. What we detect, and the rally model
18     - **Hit event** = racket?shuttle contact ("thwack"), a short broadband transient.
19     - **Rally** = a temporally-clustered run of hits with rhythmic cadence (a shot roughly
every ~0.4?2 s,
20     alternating ends). **Dead time** = no/sparse hits. A **single isolated hit** = the
between-point service
21     feed/toss ? i.e. exactly the false-fire pattern we've been fighting. So hit
*clusters*, not single hits,
22     define rallies.
23
24     ## 3. The GoPro-audio reality (honest, and the key risk)
25     All clips provided so far have audio (GoPro records it by default). BUT: a shuttle
"thwack" is **much quieter**
26     than a tennis/table-tennis impact (lighter projectile), so the **SNR transfer is the
dominant open risk** ? the
27     research's audio numbers come from louder sports. Gym reverb, wind, and crowd add noise.
**Mitigations:**
28     1. **E1 (below) is an explicit GO/NO-GO probe on the real GoPro audio** before we build
anything heavy.
29     2. Make **"record with audio enabled, mic unobstructed"*** a capture requirement (added
to

```

```

30     `docs/VIDEO_RECORDING_GUIDELINES.md`).
31
32     ## 4. Architecture (all local, CPU-light)
33     - **Stage 0 ? Extract:** `ffmpeg` ? mono WAV (~16?22 kHz; keep HF ? clicks are
broadband). Cached per video,
34     keyed like the frame/trajectory caches.
35     - **Stage 1 ? Onset/peak candidates (NO training):** band-emphasis/high-pass ? short-
time energy + spectral
36     flux ? adaptive (EMA/median) threshold ? peak-pick ? `(timestamp, strength)` candidate
impacts. (Research E1.)
37     - **Stage 2 ? Hit confidence / FP suppression:** a likelihood-ratio confidence
(research: raw peaks ~41?50 F ?
38     ~68?77 F) and optionally a tiny MFCC-window classifier (GMM/logreg, hit-vs-not),
bootstrapped from the
39     user's golden rally spans (hits inside rallies = positives). (Research E2.)
40     - **Stage 3 ? Hits ? rally structure:** cluster impacts by inter-hit gap; a cluster with
?K hits and plausible
41     cadence = a rally; **START ? first hit (the serve), END ? last hit + a short tail**;
drop isolated single
42     hits (the feed).
43     - **Stage 4 ? Fuse with the WASB trajectory** (in OUR rally layer ? WASB stays a frozen
black box; we do NOT
44     feed audio into WASB. See ADR-007 "Integration architecture" for the full rationale +
the Option-B end-state):
45     - **Recall recovery (the main win):** propose a candidate rally wherever there's a
dense hit-cluster but WASB
46     proposed nothing.
47     - **Precision / features:** add per-candidate audio features ? `hit_count`,
`hit_rate`, inter-hit-interval
48     mean/std (cadence regularity), `time_since_last_hit`, end-of-window energy ? to the
existing
49     `distill_local` student (Research E3).
50     - **Joint rule:** a rally needs shuttle-motion **and** a hit-cluster (precision), OR
an audio cluster
51     recovers a WASB miss (recall).
52     - **Stage 5 ? Eval:** vs **golden labels** (the rater CSVs from the VLC annotator) at
IoU 0.5, leave-one-video-
53     out; report recall lift + precision vs the WASB-only baseline and the trajectory-only
distilled model.
54
55     ## 5. Experiments (sequenced ? E1 is the gate)
56     | # | Experiment | Needs | Decides |
57     |---|---|---|---|
58     | **E1** | Extract audio ? energy-peak hits ? overlay hit-density on WASB windows
**and** golden labels | audio + ?1 golden-rated video | **GO/NO-GO**: do hit-clusters align with
real rallies + recover WASB-missed ones on *real GoPro* audio? |
59     | **E2** | Add likelihood-ratio / MFCC confidence to suppress false peaks | E1 + a few
labeled hits | F-score lift of the hit stream |
60     | **E3** | Add audio features to `distill_local`; re-eval IoU-0.5 LOVO | E1/E2 + ?2
golden videos | does fused beat WASB-only + trajectory-only distilled? |
61     | **E4** | Use first/last hit to tighten rally start/end | E1 | boundary-IoU improvement
|
62
63     ## 6. Code & dependencies
64     - New module `backend/pipeline/audio/hit_detector.py` (extract + onset/peak +
confidence). Pure DSP helpers
65     unit-tested on **synthetic click audio** (no fixtures); ffmpeg via subprocess (already
a dep).
66     - Reuse: feed audio features into `backend/eval/distill_local.py` (pluggable
`feature_fn`); score with the
67     existing `metrics.py` / golden loaders.
68     - **Deps:** **numpy + stdlib `wave` (no scipy needed)** for filtering/STFT ? the
spectral-flux onset is a pure
69     numpy STFT, so the originally-anticipated scipy dependency was not added. Avoid
heavy/AGPL audio stacks. Any

```

```

70     later small CNN = trained in-house (no restrictive weights).
71     - **Licensing:** all permissive ? no new model weights for Stage 1. Consistent with
ADR-005.
72
73     ## 7. Risks & mitigations
74     - **Shuttle SNR (top risk):** E1 is the empirical gate; if NO-GO, audio is deferred, not
forced.
75     - **No crowd applause** in amateur single-camera clips (a cue the literature leans on
for end-of-point) ? rely
76         on **hit-cadence cessation**, not applause.
77     - **Mic occlusion / wind** ? recording guideline (audio on, unobstructed, avoid
aggressive wind-suppression).
78     - **A/V sync:** audio and frames share the same file timeline ? no sync work needed.
79
80     ## 8. Gating & sequencing
81     - All measurement is **gated on golden labels** (user rating via the VLC annotator ?
`import-local` / the
82         in-repo `--labels` loaders). So: rate a few videos ? **E1 go/no-go** ? E2/E3.
83     - Roadmap order: **AUDIO (now)** ? *[a TBD owner hunch ? slot reserved]* ? **POSE/STANCE
heuristic (#2, parked;
84         expected substantially harder)** ? **OPTION B: an OWNED end-to-end multimodal rally
model** (frames + audio +
85         pose + ? ? rallies), fine-tuned on our golden corpus instead of depending on WASB's
pretrained weights ? the
86         intended destination once the corpus is healthy. The modular cues here are how we earn
it (each one cheap,
87         debuggable, and a generator of the labels Option B needs). Full rationale: ADR-007
"Integration architecture".
88
89     ## 9. Success criteria
90     - **E1:** audio hit-clusters recover a meaningful share of WASB-missed golden rallies on
real GoPro audio
91         (recall lift), with hit-clusters aligning to true rallies. Pass ? proceed; fail ?
defer audio, revisit mic/setup.
92     - **E3:** the fused model's held-out (LOVO) F1 beats both the WASB-only baseline and the
trajectory-only
93         distilled student.
94
95     ## 10. E1 RESULTS ? preliminary, on existing GoPro audio vs Gemini SILVER labels
(2026-06-07)
96     Ran the E1 probe (`backend/eval/audio/e1_probe.py`) + a band*k sweep on Adarsh (36
rallies) and
97     TestLargeVideo (7). **Verdict: AMBER ? leaning negative for the *classical* detector.**
98     - ? **Recall signal is real:** the detector fires in **100% of rallies** and **100% of
WASB-missed rallies
99         contain hit-clusters** ? audio *does* light up where rallies are.
100    - ?? **Precision/discrimination ceilings ~2.2x** (in-rally vs *mid-match* dead-time hit
density), on BOTH videos.
101    Audio-only proposal F1 (0.13?0.27) trails WASB-only.
102    - ? **Band-pass did NOT help** (tested low/mid/high bands x k at 32 kHz): HF "shuttle-
click" bands gave *lower*
103        discrimination, not higher. Shuttle hits have no band-separable signature in this
audio.
104    - ? **Not a warm-up/label artifact:** mid-match dead time (between rallies) still has
~0.6 hits/s vs ~1.3
105        in-rally ? the ~2.2x ceiling holds even excluding pre/post-match regions. Root cause:
between-point activity
106        (the loser tossing the shuttle back ? our very false-fire ? plus footwork/talk) makes
transients a classical
107        onset detector can't distinguish from rally hits.
108    - `k` (threshold) is the only real lever and trades coverage for discrimination (~3x
costs half the rallies).
109    - A `band` knob was added to `onset_envelope`/`detect_hits` and KEPT (cheap, optional) ?
but it is **not** the fix.
110

```

```
111    **Implication / next:** the cheap classical-onset cue is **too weak to be a clean
precision signal**, and a
112    band-pass won't rescue it. Remaining audio paths, in order of honesty:
113    1. **E2 ? a learned TIMBRE classifier** (MFCC/log-mel ? shuttle-thwack vs
toss/footstep/voice). This separates by
114    *timbre*, which a band-pass/onset cannot ? the only plausible way to break the ~2.2x
ceiling. **Needs labeled
115    hits** (bootstrap from human-gold rally spans, or hand-label a few minutes). Real
work.
116    2. **Re-confirm E1 on HUMAN-GOLD labels** ? cheap, but the warm-up check already
suggests it won't move the
117    ceiling much (dead-time activity is mid-match, not just warm-up).
118    3. **Re-prioritize:** given the classical ceiling, consider the owner's "other hunch" /
the pose-stance #2 /
119    leaning on more golden data for the WASB+distilled path, rather than over-investing
in classical audio.
120
```

23. user (2026-06-10T05:34:38.032Z)

```
1    # past_projects/
2
3    This subdirectory is for **completed small tech debt, starter projects, or low-regret
initiatives**.
4
5    ## Criteria for past_projects/
6    - The work was a bounded "project" (e.g. low-regret moves, scaffolding maturation,
specific tech debt cleanup).
7    - It has reached a terminal state (STATUS: DONE / COMPLETED / REJECTED).
8    - It is no longer the active focus (new work happens via the live docs + BACKLOG).
9
10   ## Example
11   - `low-regret-moves/LOW_REGRET_MOVES_PLAN.md` ? the 2026 low-regret + scaffolding phase.
Marked COMPLETED after the three scaffolding PRs were merged.
12
13   Future small completed starter projects or tech debt cleanups that are similar in nature
should land here (with their own subdir and clear STATUS header).
14
15   Larger ongoing quality iterations or major feature work will typically use dated entries
under `../checkpoints/` or `../quality-iterations/` instead.
16
```

24. user (2026-06-10T05:34:38.157Z)

```
1    # archives/
2
3    Umbrella directory for all **historical and completed** material.
4
5    ## Live vs Historical
6    - **Live docs** (for ramp-up, constant updates, and current work) live at the top level
of `docs/`.
7    - Everything here is no longer the primary source of truth. It is kept for context,
decision history, and regression detection.
8
9    ## Classification inside archives/
10   - `past_projects/` ? Completed small tech debt, starter projects, or low-regret
initiatives (e.g. the original low-regret moves + scaffolding work). These are "past" once they
reach a terminal state.
11   - `decisions/` ? Historical ADRs and major architectural decisions.
12   - `research/` ? Completed or superseded research notes and plans.
13   - `checkpoints/` ? Snapshots of major project states (especially useful for detecting
regressions in quality, priorities, or baselines over time as we do quality iterations).
14   - Other one-off audits, etc.
15
16   ## Archival Rule
```

```
17     Only move something into `archives/` (or its subdirectories) when the work is in a clear
**terminal state**:
18     - DONE
19     - COMPLETED
20     - REJECTED
21
22     Never archive in-flight or pending work.
23
24     When archiving a project/phase:
25     1. Ensure the main document has an explicit `**STATUS: COMPLETED**` (or DONE/REJECTED)
header at the top.
26     2. Link back to the current living documents in `../` (e.g.
`COMMERCIAL_READINESS_REVIEW.md`, `BACKLOG.md`).
27     3. For regression detection value, consider adding a dated entry under `checkpoints/`.
28
29     See `past_projects/README.md` for the specific guidance on that category.
30
```

25. user (2026-06-10T05:34:44.470Z)

```
1     # Low-Regret Moves Plan ? Config Model First
2
3     **STATUS: COMPLETED** (all items merged June 2026 ? PRs #49, #50, #51)
4
5     > Historical record of the low-regret + scaffolding maturation phase.
6     > Current state and next priorities live in
[../../../COMMERCIALIZATION.md](../../../COMMERCIALIZATION.md)
7     > and [../../../BACKLOG.md](../../../BACKLOG.md).
8
9     **Date:** 2026-06-07 (final update)
10    **Owner / Driver:** Grok (second-in-command) + avidullu + Claude
11
12    ---
13
14    ## 1. Purpose & Philosophy
15
16    These are low-regret moves: changes that are high-confidence improvements to code
health today while directly de-risking the commercial and self-improving vision (offline learned
segmenter, hosted path, multi-sport, data flywheel).
17
18    Guiding principles
19    - Small, reviewable increments (one or two files per PR where possible).
20    - Payoff within the same or next session.
21    - Improve the developer + product experience immediately.
22    - Lay infrastructure that makes future items cheaper and safer.
23    - Respect the existing culture (registries, eval harness, ADRs, excellent docs).
24
25    Rule: Every item must be traceable to a concrete pain/risk from the Commercial
Readiness Review or existing BACKLOG/ADRs.
26
27    ---
28
29    ## 2. Overall Sequence (Approved Order)
30
31    1. Config Model (Pydantic) + Central Loader (as a proper package) ? foundational.
32    2. Directory Structure Refactor (high-pri, atomic follow-up PR) ? move to the
approved layered layout.
33    3. Error / Result Contract standardization.
34    4. Logging standardization + removal of stray `print()` in hot paths. ? (done in
feat/logging-standardization; central setup in main; replaced in motion.py, wasb_infer.py,
stitcher/compiler.py, config/loader.py, main internal diagnostics; user-facing output in
run_*.py and main CLI kept as print where appropriate for direct console UX. Docs updated in
CODE_MAP and this plan.)
35    5. Packaging modernization (`pyproject.toml` + clean entry points). ? (done in
feat/packaging-modernization; pyproject.toml with PEP 621 + [project.scripts]; pip install -e .
```

```

supported; setup.py retained for diagnostics; docs + tests updated.)
36     6. Detector Runner Abstraction (start + finish de-risking WSL). ? (start sketched;
finished in feat/finish-detector-runner-abstraction + PR #49: base.py ABC now matches the real
(ok, msg) contract; runners inherit; hybrids type against DetectorRunner; contract tests for
both; rich docs for adding Linux-native runners. 224 passed @64%.)
37     7. Unify / extract shared logic from the two hybrid segmenters.
38     8. Input Quality UX ? reframed as extend existing reports (audio readiness + capture
guidance from VIDEO_RECORDING_GUIDELINES). ? (feat/extend-reports-input-quality + PR #50,
merged)
39     Value Layer (exports EDL/JSON/CSV timelines + basic stats as minimal first slice). ?
(feat/value-layer-minimal-exports + PR #51, merged)
40     9. (Stretch) First slice of async job model.
41
42     **Important sequencing decision (approved):** Complete the Config work **first** (this
introduces `backend/config/` as a package immediately). Then perform the broader directory
refactor as a **separate, atomic, high-priority PR**.
43
44     Rationale:
45     - Keeps the Config PR small and focused.
46     - Allows the structural change to land atomically with minimal merge conflicts.
47     - Gives Claude a clean window with no concurrent file moves.
48     - Still delivers the "level of indirection" benefit early via the `config/` package.
49
50     ---
51
52     ## 3. Approved Directory Structure Direction
53
54     Current flat layout under `backend/` (many sibling dirs + root .py files) feels
unprofessional for long-term maintenance. We will trend toward clear layers without creating 15+
tiny single-file directories.
55
56     **Target layered structure (directional, not all at once):**
57
58     ```
59     backend/
60     ??? __init__.py
61     ??? main.py                # Thin CLI + FastAPI bootstrap only
62     ?
63     ??? config/                # ? Start here with Deliverable 1
64     ?   ??? __init__.py
65     ?   ??? models.py          # Pydantic AppConfig + nested models
66     ?   ??? loader.py          # load_config, save_default_config, etc.
67     ?
68     ??? api/                   # Presentation layer
69     ?   ??? __init__.py
70     ?   ??? app.py              # FastAPI instance, middleware, routes
71     ?   ??? static/             # Frontend assets (HTML/JS/CSS)
72     ?
73     ??? pipeline/              # Core processing engine
74     ?   ??? validators/
75     ?   ??? segmenters/        # (renamed/moved from current indexer/)
76     ?   ??? stitcher/
77     ?   ??? audio/
78     ?   ??? detectors/
79     ?
80     ??? infrastructure/        # External adapters

```

26. user (2026-06-10T05:34:44.948Z)

```

1     # docs/
2
3     Live documentation for the badminton-highlight-indexer.
4
5     ## For new contributors / ramp-up (read these first)
6     - [CODE_MAP.md](CODE_MAP.md) ? Complete navigation map of the codebase and data

```

contracts.

```
7      - [COMMERCIALIZATION.md](COMMERCIALIZATION.md) ? Living tracker for the commercial
dimension (licensing, codec/output economics, hosting, vendoring, packaging) + PMF risks and the
quality/data-phase priorities. Supersedes the archived `COMMERCIAL_READINESS_REVIEW.md`.
8      - [PMF_MARKET_RESEARCH.md](PMF_MARKET_RESEARCH.md) ? Cited market study: sizing,
customer segments, competitor pricing, and the recommended beachhead (feeds COMMERCIALIZATION
$6).
9      - [PLATFORM_ARCHITECTURE.md](PLATFORM_ARCHITECTURE.md) ? Strategy for the multi-user
"assistant coach" platform: decoupled layers + pluggable storage/compute (BYO), multi-tenant
data model, orchestration, and a phased roadmap.
10     - [DEFERRED_HARDENING_PLAN.md](DEFERRED_HARDENING_PLAN.md) ? Design (pending approval)
for the larger deferred items: packaging, train/eval guardrails, and the async job model (#16,
the first infra slice toward the platform).
11     - [BACKLOG.md](BACKLOG.md) ? Deferred items with rationale and revisit triggers.
12     - [ROADMAP.md](ROADMAP.md) ? High-level four-phase product direction.
13     - [VIDEO_RECORDING_GUIDELINES.md](VIDEO_RECORDING_GUIDELINES.md) ? Capture best
practices (referenced by reports).
14     - [HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md) ? Mental model of the
two-layer detection system.
15     - [EVALUATION.md](EVALUATION.md) ? How we measure and improve quality.
16
17     ## Practical / ongoing
18     - [TRACKNET_WSL_SETUP.md](TRACKNET_WSL_SETUP.md)
19     - Golden set docs (`GOLDEN_SET_*`)
20     - Evaluation harness and calibration drivers (in `backend/eval/`)
21
22     ## Historical documents
23     Completed plans, old research, ADRs, and checkpoints live in [`archives/`](archives/).
24
25     See `archives/README.md` for the umbrella policy and `archives/past_projects/README.md`
for classification of small tech debt / starter projects.
26
```

27. assistant (2026-06-10T05:35:45.861Z)

Now verifying against code. Let me check the repo structure and key entry points.

28. user (2026-06-10T05:35:47.475Z)

```
CLAUDE.md
README.md
TEST_RUN_SUMMARY.md
__pycache__
backend
config.json
docs
output
pytest.ini
requirements.txt
run_all_tests.py
run_eval.py
run_phase3_eval.py
run_process_and_stitch.py
run_regression.py
scratch
scripts
setup.py
tests
----
__pycache__
annotations
api
audio
config
detectors
```

```
eval
indexer
infrastructure
main.py
pipeline
providers
reporting
results.py
sports
stitcher
tools
utils
validators
```

29. user (2026-06-10T05:35:48.591Z)

```
56: p.add_argument("--annotations", help="Path to the ground-truth rally CSV (start,end per
row).")
57: p.add_argument("--video", help="Path to the video file (its MD5 becomes the video_id).")
58: p.add_argument("--video-id", help="Stored video_id (MD5) to evaluate, instead of
--video.")
59: p.add_argument("--db", default=None, help="SQLite DB path (default: from config.json).")
60: p.add_argument("--stage", choices=("candidates", "final", "both"), default="both",
61:               help="Which prediction stage(s) to score (default: both).")
62: p.add_argument("--detector", default=None,
63:               help="Only score candidates from this detector (e.g. yolo_hybrid,
tracknet_hybrid).")
64: p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
65: p.add_argument("--pr-curve", action="store_true", help="Include an IoU sweep (0.1..0.9)
in the report.")
66: p.add_argument("--show-failures", action="store_true",
67:               help="List missed (FN) and spurious (FP) rallies with timestamps.")
68: p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report to this
path.")
69: p.add_argument("--scaffold", action="store_true",
70:               help="Write an empty annotation CSV to --annotations and exit.")
71: return p
```

30. user (2026-06-10T05:35:57.244Z)

```
Exit code 1
backend/api:
static

backend/audio:
__pycache__

backend/detectors:
__pycache__

backend/indexer:
__pycache__

backend/stitcher:
__pycache__

backend/validators:
__pycache__
====
```

31. user (2026-06-10T05:36:08.968Z)

```
backend/main.py
```


backend/results.py

backend/eval:

__init__.py
__pycache__
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
export_wasb_segments.py
gemini_refine.py
harness.py
metrics.py
rally_seq_proto.py
regression.py
training.py

backend/pipeline:

audio
detectors
segmenters
stitcher
validators

backend/tools:

__init__.py
__pycache__
export.py
rally_slicer.py
===

backend/pipeline/detectors:

__init__.py
__pycache__
base.py
rally_gate.py
tracknet_runner.py
wasb_infer.py
wasb_runner.py

backend/pipeline/segmenters:

__init__.py
__pycache__
base.py
gemini.py
motion.py
tracknet_hybrid.py
wasb_hybrid.py
yolo_hybrid.py

32. user (2026-06-10T05:36:17.923Z)

```
282:     parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
283:     parser.add_argument("--output-dir", default="output", help="Directory where processed
clips/highlights are saved")
284:     parser.add_argument("--setup", action="store_true", help="Launch dependency checker and
setup utility")
285:     parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video
clip to run detailed diagnostics on")
286:     parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
```

```

287:     parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
288:     parser.add_argument("--max-frames", type=int, help="Limit processing to the first N sub-
sampled frames for debugging")
289-
--
291:     parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
292:     parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
293:     parser.add_argument("--trim-end", type=float, help="End time in seconds for the debug
trim segment feature")
294:     parser.add_argument("--trim-output", help="Output filename for the debug trimmed segment
(saves in output-dir unless absolute path)")
295-

```

33. user (2026-06-10T05:36:19.477Z)

```

default_segmenter= gemini
top keys: ['sport', 'ai_provider', 'ai_model', 'validation', 'indexing', 'output', 'stitching']
ai_model: gemini-2.5-flash
report: None
indexing keys: ['default_segmenter', 'use_gpu', 'motion_threshold', 'min_segment_duration',
'dead_time_merge_gap', 'chunk_duration', 'chunk_overlap', 'detector_model',
'detector_score_threshold', 'yolo_velocity_threshold', 'yolo_frame_skip', 'yolo_tracker',
'yolo_parallel_chunks', 'yolo_chunk_workers', 'min_action_window_duration',
'log_yolo_candidates', 'store_yolo_candidates', 'ai_handoff_padding', 'ai_max_workers',
'tracknet']

```

34. user (2026-06-10T05:36:33.391Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, Literal
12
13     from pydantic import BaseModel, Field, field_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):

```

```

34     wsl_distro: str = "Ubuntu"
35     repo_dir: str = "~/models/TrackNetV4"
36     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37     conda_env: str = "TrackNetV4"
38     weights_path: str = ""
39     queue_length: int = 5
40     stage_in_wsl: bool = True
41     wsl_stage_dir: str = "~/clips"
42     timeout_sec: int = 0
43     velocity_threshold: float = 0.02
44
45
46     class IndexingConfig(BaseModel):
47         default_segmenter: str = "yolo_hybrid"
48         use_gpu: bool = True
49         motion_threshold: float = 0.08
50         min_segment_duration: float = 3.0
51         dead_time_merge_gap: float = 2.0
52         chunk_duration: float = 90.0
53         chunk_overlap: float = 10.0
54         detector_model: str = "fasterrcnn_resnet50_fpn"
55         detector_score_threshold: float = 0.5
56         yolo_velocity_threshold: float = 0.15
57         yolo_frame_skip: int = 3
58         yolo_tracker: str = "bytetrack.yaml"
59         yolo_parallel_chunks: bool = False
60         yolo_chunk_workers: int = 2
61         min_action_window_duration: float = 2.0
62         log_yolo_candidates: bool = True
63         store_yolo_candidates: bool = True
64         ai_handoff_padding: float = 2.0
65         ai_max_workers: int = 5
66
67         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
68
69         @field_validator("default_segmenter")
70         @classmethod
71         def segmenter_must_be_registered(cls, v: str) -> str:
72             # Registry check happens at runtime in main/segmenter selection.
73             # Here we just allow any string for forward compatibility.
74             return v
75
76
77     class OutputConfig(BaseModel):
78         default_highlights_name: str = "highlights.mp4"
79         db_name: str = "sports_indexer.db"
80         # Directory where the DB, highlight reels, and processed clips are written.
81         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
82         output_dir: str = "output"
83
84
85     class StitchingConfig(BaseModel):
86         begin_padding: float = 0.5
87         end_padding: float = 0.5
88         use_cache: bool = False
89         min_shot_count: int = 1
90
91
92     class AppConfig(BaseModel):
93         """Root configuration object.
94
95         All runtime configuration should be accessed via an instance of this class
96         (or the loader) rather than raw dict lookups.
97         """
98

```

```

99     sport: Sport = "badminton"
100    ai_provider: str = "gemini"
101    ai_model: str = "gemini-2.5-flash"
102
103    validation: ValidationConfig = Field(default_factory=ValidationConfig)
104    indexing: IndexingConfig = Field(default_factory=IndexingConfig)
105    output: OutputConfig = Field(default_factory=OutputConfig)
106    stitching: StitchingConfig = Field(default_factory=StitchingConfig)
107
108    # Allow extra keys during transition so existing config.json files do not break.
109    model_config = {
110        "extra": "allow",
111        "validate_assignment": True,
112    }
113
114    # Temporary migration shim for legacy dict-style access (config.get("section",
dict    {})).get(...)).
115    # Will be removed once all call sites (validators, segmenters, etc.) move to typed
116    # attribute access / the typed config. A nested *section* is returned as a plain
dict    dict
117    # via model_dump, so legacy chained access keeps working ? returning the Pydantic
118    # section model instead breaks it (a model has no .get), which bit every validator
on
119    # real runs. Scalar leaves are returned as-is.
120    def get(self, key: str, default: Any = None) -> Any:
121        _MISSING = object()
122        val: Any = _MISSING
123        if key in type(self).model_fields or hasattr(self, key):
124            val = getattr(self, key, _MISSING)
125        if val is _MISSING:
126            # Legacy convenience: find a leaf key inside any nested section. Sections
are
127            # discovered dynamically from the model fields, so a future top-level
section
128            # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
129            for fname in type(self).model_fields:
130                section = getattr(self, fname, None)
131                if isinstance(section, BaseModel) and hasattr(section, key):
132                    val = getattr(section, key)
133                    break
134            if val is _MISSING:
135                return default
136            if isinstance(val, BaseModel):
137                return val.model_dump(mode="json")
138            return val
139
140    def __getitem__(self, key: str) -> Any:
141        val = self.get(key)
142        if val is None and key not in type(self).model_fields:
143            # Re-raise like a real dict for unknown top-level
144            raise KeyError(key)
145        return val
146
147
148    # Convenience type alias for callers that want to annotate
149    Config = AppConfig

```

35. user (2026-06-10T05:36:53.359Z)

```

11: def run_pip_install():
22:     pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
35: def init_default_config():
59: def run_diagnostics():

```

36. user (2026-06-10T05:36:53.793Z)

```
1      # Observability report spec for the badminton-highlight-indexer.
2      # The report ENGINE (config language, renderers) lives in the shared `obsreport`
3      # library; this file is the indexer-specific layout + footgun rules. Every
4      # actionable rule cites a README/FAQ section so a reader can self-debug.
5      name: indexer
6      title: Badminton Highlight Indexer ? Video Report
7
8      # Customer here = whoever receives the highlight reel.
9      customer_verdict:
10         pass: "? Your highlights are ready."
11         pass_with_warnings: "? Your highlights are ready. (A couple of minor notes above.)"
12         fail: "?? We couldn't finish generating highlights for this video. Please contact
support."
13
14     sections:
15         - { id: run,          title: "Run",          kind: identity,  audience: both }
16         - { id: video,       title: "Video",        kind: video_meta, audience: both }
17         - { id: highlights,  title: "Highlights",    kind: highlights, audience: customer
18     }
19         - { id: cust_notes,  title: "Things to know", kind: flags,    audience: customer
20     }
21         - { id: stages,     title: "Processing stages", kind: stages,    audience:
technical }
22         - { id: findings,   title: "Findings & warnings", kind: flags,    audience:
technical }
23         - { id: verdict,    title: "Verdict",        kind: verdict,   audience: both }
24
25     rules:
26         # The classic silent failure: 0 segments because the AI provider no-op'd on a
27         # missing API key ? NOT because the video has no rallies.
28         - code: silent_provider_noop
29           severity: error
30           when:
31             - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: false }
32             - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
33           message: >-
34             0 segments AND no AI provider API key was set. The segmenter (see Processing
35             stages > segmentation > segmenter_actual) hands its candidate windows to the AI
36             provider, which silently did nothing without a key ? so this is a MISSING
37             CREDENTIAL, not 'the video has no rallies'. Fix: put GEMINI_API_KEY=<key> in a
38             .env file in the project root and re-run (see README#Q7).
39           faq_ref: "README#Q7"
40           customer_message: >-
41             We couldn't generate highlights for this video due to a configuration issue
42             on our side. Please contact support.
43
44         # API key WAS present but an AI segmenter still produced nothing ? likely a
45         # provider error / quota / WSL healthcheck issue rather than a truly empty video.
46         - code: ai_empty_vs_no_rallies
47           severity: warn
48           when:
49             - { path: "stages.ai_handoff.details.ai_based", op: eq, value: true }
50             - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: true }
51             - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
52           message: >-
53             An AI-based segmenter returned 0 segments even though the API key is set.
54             Empty can mean a provider error, exhausted quota, or (for WASB/TrackNet) a
55             failed WSL healthcheck ? these all return [] silently. Re-run with -u for
56             unbuffered logs and check the provider/WSL output before concluding 'no rallies'.
57           faq_ref: "README#Q7"
58
59         # fps was assumed rather than probed -> velocity thresholds + timing may be wrong.
60         - code: fps_fallback_used
```

```

59     severity: warn
60     when:
61     - { path: "video_meta.fps_source", op: eq, value: fallback }
62     message: >-
63         fps was NOT probed from the file ? a fallback default (shown as the fps value)
64         was used. Because rally timing and motion thresholds scale with fps, boundaries
65         and motion sensitivity may be miscalibrated. This usually means validation didn't
66         run (it probes the file) or ffprobe couldn't read r_frame_rate. Confirm the real
67         fps with ffprobe and re-run.
68     faq_ref: "README#Q14"
69
70     # Audio stream missing or disabled. Per VIDEO_RECORDING_GUIDELINES (and ADR-007)
71     # the shuttle "thwack" is a first-class rally cue for recall; recording with audio
72     # on (mic unobstructed, no aggressive noise suppression) is now a hard requirement
73     # for good results with the shuttle-tracking substrates.
74     - code: audio_disabled_or_missing
75     severity: warn
76     when:
77     - { path: "video_meta.audio_present", op: eq, value: false }
78     message: >-
79         No audio stream was detected in the input (or it could not be read). The shuttle
80         "thwack" impact sound is a strong cue that improves rally recall (especially for
81         the WASB/TrackNet detector paths). Record with audio enabled, keep the camera
82         mic unobstructed, and avoid heavy wind/noise-reduction filters that flatten the
83         transient. See VIDEO_RECORDING_GUIDELINES.md (the "Audio (REQUIRED)" row) and
84         AUDIO_RALLY_DETECTION_PLAN.md.
85     faq_ref: "VIDEO_RECORDING_GUIDELINES"
86     customer_message: >-
87         For best highlight results, record with audio on and the mic unobstructed.
88         The shuttle impact sound helps the detector find rallies. Re-record or
89         re-process after enabling audio (see VIDEO_RECORDING_GUIDELINES.md).
90
91     # The motion baseline fabricates its per-segment metadata/events ? they are not
92     # measured. Warn so nobody trusts them as ground truth.
93     - code: synthetic_motion_metadata
94     severity: warn
95     when:
96     - { path: "stages.segmentation.details.segmenter_actual", op: eq, value: motion }
97     message: >-
98         The 'motion' segmenter was used: its per-segment metadata (intensity, speed,
99         events, server/receiver) are PLACEHOLDER values (unmeasured) ? fabricated by the
100        heuristic, NOT derived from the video ? so a PASS with segments here does not mean
101        the metadata is trustworthy. Use yolo_hybrid/gemini for real shot metadata.
102    faq_ref: "README#Q10"
103
104    # The segmenter that ran differs from config's default -> you may be looking at a
105    # different detector than expected (config/setup/CLI defaults are known to diverge).
106    - code: segmenter_divergence
107    severity: info
108    when:
109    - { path: "stages.segmentation.details.matches_config_default", op: eq, value:
false }
110    - { path: "stages.segmentation.details.segmenter_explicitly_requested", op: eq,
value: false }
111    message: >-
112        The segmenter that ran differs from config.json's default_segmenter, and no
113        explicit --segmenter was given ? so a fallback default kicked in. The indexer's
114        defaults are known to diverge across config.json / setup.py / the CLI; confirm
115        you ran the detector you meant.
116    faq_ref: "README#Q7"
117
118    # Re-processing an already-indexed video replaced its prior segments (INSERT OR
119    # REPLACE cascades). Note it so a changed result isn't mistaken for instability.
120    - code: reingest_replaced_prior
121    severity: info

```

```

122     when:
123         - { path: "stages.segmentation.details.was_reingest", op: eq, value: true }
124     message: >-
125         This video was already in the database; re-processing REPLACED its previous
126         segments (videos are keyed by file MD5; re-ingest cascades a delete). Expected
127         when you intend to re-run; just know the earlier results are gone.
128     faq_ref: "README#Q11"
129
130     # A validation check failed -> segmentation was halted. Point the reader at the
131     # report's Validation section + the per-check FAQ.
132     - code: validation_failed
133       severity: error
134     when:
135         - { path: "stages.validation.status", op: eq, value: error }
136     message: >-
137         One or more validation checks failed, so indexing was halted. See the
138         Processing stages > validation block above: each failed check states the
139         reason (relevance -> README#Q2, fps/keyframes -> README#Q5).
140     faq_ref: "README#Q13"
141     customer_message: >-
142         This video couldn't be processed because it didn't pass our quality checks
143         (for example: too blurry, too low resolution, or not a badminton court).
144

```

37. assistant (2026-06-10T05:37:08.306Z)

spec.yaml verified. Now let me run the test suite for current counts and check a batch of CODE_MAP line-level claims.

38. user (2026-06-10T05:37:24.921Z)

```

=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====
..... [ 28%]
..... [ 56%]
..... [ 85%]
..... [100%]
254 passed in 13.36s
=====
RESULT: PASS - all tests green
=====

```

39. user (2026-06-10T05:37:35.790Z)

```

32: _PERSON_CLASS_ID = 1
41: _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
81:     model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
83:     logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
84:     model_name = _DEFAULT_DETECTOR
125:     mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)

[Showing results with pagination = limit: 6]

```

40. user (2026-06-10T05:37:37.207Z)

```

14: MAX_UPLOAD_MB = 500
44:     if size_mb > MAX_UPLOAD_MB:
45:         logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB
cap ? "

```

41. user (2026-06-10T05:37:38.603Z)

```
26:from backend.annotations import annotation_registry
30:DEFAULT_BASELINE_PATH = os.path.join("output", "regression_baseline.json")
31:DEFAULT_TOLERANCE = 0.05
55:def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
68:     f1: float, path: str = DEFAULT_BASELINE_PATH) -> None:
90:     tolerance: float = DEFAULT_TOLERANCE,
91:     baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
132:     tolerance: float = DEFAULT_TOLERANCE,
```

[Showing results with pagination = limit: 8]

42. user (2026-06-10T05:37:39.728Z)

```
12:     return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for timeline
gaps, jumps, or editing splices."
57:     # Analyze gaps
58:     max_gap = 0.0
60:     gaps = []
63:     gap = timestamps[i] - timestamps[i-1]
64:     gaps.append(gap)
65:     if gap < 0:
67:     if gap > max_gap:
68:         max_gap = gap
71:     # If there's a keyframe gap larger than 8.0 seconds or a negative gap (backward
jump), it's highly likely spliced.
74:     "max_keyframe_gap_seconds": round(max_gap, 2),
88:     # Threshold for sudden gap (e.g. 10.0 seconds)
```

[Showing results with pagination = limit: 12]

43. user (2026-06-10T05:37:49.218Z)

```
backend/pipeline/validators/pts_continuity.py:89:         if max_gap > 10.0:
backend/pipeline/validators/scene_cut.py:47:         cut_threshold = 0.6
backend/pipeline/validators/scene_cut.py:63:         if score < cut_threshold:
backend/pipeline/validators/scene_cut.py:70:         # Cap check at 100 frames max (e.g.
first 50 seconds or evenly spread) to keep validation instant
backend/pipeline/validators/scene_cut.py:71:         if frame_idx >= total_frames or
sampled_count > 100:
===
69:     min_blur_threshold = val_cfg.get("min_blur_threshold", 100.0)
71:     if avg_variance < min_blur_threshold:
75:         message=f"Video quality rejected: Focus check failed. Average sharpness score
of {round(avg_variance, 1)} is below the minimum required threshold of {min_blur_threshold}.",
===
31:BASELINE: Config = (0.02, 2.0, 2.0) # the WasbConfig / trajectory_to_action_windows
defaults
67:def default_grid() -> List[Config]:
82:     base = evaluate(predict_fn(BASELINE), gts, iou)
84:     fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou) # OPTIMISTIC (fit on
full GT)
89:     print(f"baseline cfg {BASELINE}: precision={base.precision:.3f} recall={base.recall:.3f}
"
===
37:TUNED: Config = (0.05, 1.0, 1.0)
===
58:         type TEXT NOT NULL,
65:         # Versioned migrations live here. PRAGMA user_version is the schema
68:         version = cursor.execute("PRAGMA user_version").fetchone()[0]
98:         cursor.execute("PRAGMA user_version = 1")
99:         # future: if version < 2: cursor.execute("ALTER TABLE ..."); PRAGMA user_version
= 2
130:     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
```



```

Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
135:         (segment_id, timestamp, event_type, player, json.dumps(details or {}))
220:         - event_type: string (e.g. smash, foul)
237:         if filters.get("event_type"):
240:             params.append(filters["event_type"])
===
test_gemini_api.cpython-311-pytest-9.0.3.pyc
test_gemini_api.cpython-313-pytest-9.0.3.pyc
===
22:def basic_stats(segments: Iterable[Dict[str, Any]]) -> Dict[str, Any]:
48:def segments_to_json(
75:def segments_to_csv(segments: Iterable[Dict[str, Any]], include_header: bool = True) -> str:
97:def segments_to_edl(segments: Iterable[Dict[str, Any]], title: str = "Highlights") -> str:
===
5:defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
17:# 64 KB matches the value used by the live pipeline (backend/main.py) before this
20:_CHUNK_SIZE = 65536

```

44. user (2026-06-10T05:38:03.723Z)

```

        velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
    )
    return [(w["start"], w["end"]) for w in wins]

return predict_fn

def default_grid() -> List[Config]:
    return [(vt, mg, mwd)
            for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
            for mg in (1.0, 2.0, 3.0)
            for mwd in (1.0, 2.0, 3.0)]

def run(trajecory_csv: str, labels_csv: str, fps: float, frame_width: float,
        iou: float = 0.5) -> dict:
    gts = load_golden_gts(labels_csv)
    if not gts:
        raise SystemExit(f"no usable golden rallies in {labels_csv}")
    predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
    grid = default_grid()
===
1:"""Tests for the indexer observability harvest + footgun rules.
101:     # motion is not AI-based -> the provider-noop rule must NOT fire even with no key
===
11

```

45. user (2026-06-10T05:38:06.573Z)

```

backend/eval/eval_partial_labels.py -> ['--trajectory', '--labels', '--fps', '--frame-width', '
--window-start', '--window-end', '--gate', '--iou', '--comparison-out', '--sweep', '--top']
backend/eval/calibrate_local.py -> ['--manifest', '--iou', '--metric']
backend/eval/rally_seq_proto.py -> ['--manifest']
backend/eval/calibrate_wasb.py -> ['--trajectory', '--labels', '--fps', '--frame-width', '--
iou']
run_regression.py -> ['--video-id', '--tier', '--annotations', '--stage', '--detector', '--iou',
'--tolerance', '--db', '--baseline', '--update-baseline', '--json']
run_phase3_eval.py -> ['--manifest', '--videos-root', '--segmenter', '--stage', '--detector', '
--iou', '--only', '--limit', '--max-frames', '--score-only', '--reprocess', '--db', '--json', '
--show-failures']

```

46. user (2026-06-10T05:38:22.896Z)

```

===

```

```

75:     meta["audio_present"] = len(audio_streams) > 0
===
1: """Golden guard: every footgun rule's faq_ref must resolve to a real README heading.
3: This is the committed backbone of report debuggability ? if a rule cites README#Qn,
41: def test_every_rule_faq_ref_resolves_to_a_readme_heading():
46:     if not rule.faq_ref:
48:         file_part, _, anchor = rule.faq_ref.partition("#")
51:         unresolved.append((rule.code, rule.faq_ref))
58:         unresolved.append((rule.code, rule.faq_ref))
59:     assert not unresolved, f"faq_ref anchors with no matching README heading: {unresolved}"
62: def test_actionable_rules_have_faq_refs():
66:     missing = [r.code for r in spec.rules if r.severity in ("error", "warn") and not
r.faq_ref]
67:     assert not missing, f"actionable rules missing faq_ref: {missing}"

```

47. user (2026-06-10T05:38:24.252Z)

```

docs/annotation/:
ANNOTATION_GUIDE.md
INTAKE_MANIFEST_TEMPLATE.csv
NDA_TEMPLATE.md
ROORA_BRIEF.md

src/cli/:
__pycache__
curate.py
ingest.py
normalize_annotations.py
===
52:     def import_local(self, args):
440:     def export_eval_set(self, args):
546:         `import-local`; any blocker issue must be resolved first, since the
578:             print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-local will reject "
582:             print(f"    python -m src.cli.curate import-local --sport <sport> "
656:             print("\n GATE: PASS ? review the flagged rallies, then `import-local`.")
849:     subparsers.add_parser("status", help="Display summary status of sports metadata
database")
852:     subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")
855:     import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
866:     fetch_parser = subparsers.add_parser(
897:     cvat_parser = subparsers.add_parser(
903:             help="Path to write the canonical per-rally CSV (import-local
input)")
914:     vd_parser = subparsers.add_parser(
927:     export_parser = subparsers.add_parser(
945:     elif args.command == "import-local":
946:         cli.import_local(args)

```

48. user (2026-06-10T05:38:37.540Z)

```

1     """Golden guard: every footgun rule's faq_ref must resolve to a real README heading.
2
3     This is the committed backbone of report debuggability ? if a rule cites README#Qn,
4     that section must exist, so a reader who follows the pointer always lands somewhere.
5     (A fresh-reader eval found a rule citing the wrong-but-existing section; an existence
6     test can't catch *semantic* mismatch, but it locks the floor: no dangling anchors.)
7     """
8     import re
9     from pathlib import Path
10
11     import pytest
12
13     pytest.importorskip("obsreport")

```

```

14     from obsreport import load_spec # noqa: E402
15
16     ROOT = Path(__file__).resolve().parent.parent
17     SPEC = ROOT / "backend" / "reporting" / "spec.yaml"
18     README = ROOT / "README.md"
19
20
21     def _headings(md: str):
22         return [ln.lstrip("#").strip() for ln in md.splitlines() if
23 ln.lstrip().startswith("#")]
24
25     def _slug(h: str) -> str:
26         s = re.sub(r"^[^\w\s-]", "", h.lower())
27         return re.sub(r"\s+", "-", s.strip())
28
29
30     def _anchor_resolves(anchor: str, headings) -> bool:
31         a = anchor.lower()
32         for h in headings:
33             hl = h.lower()
34             if hl == a or hl.startswith(a + ":") or hl.startswith(a + " ") or
35 hl.startswith(a + "."):
36                 return True # Q-number style: "Q7" -> "### Q7: ..."
37             if _slug(h) == a:
38                 return True # full GitHub slug match
39         return False
40
41     def test_every_rule_faq_ref_resolves_to_a_readme_heading():
42         spec = load_spec(SPEC)
43         headings = _headings(README.read_text(encoding="utf-8"))
44         unresolved = []
45         for rule in spec.rules:
46             if not rule.faq_ref:
47                 continue
48             file_part, _, anchor = rule.faq_ref.partition("#")
49             if file_part in ("README", "README.md"):
50                 if not _anchor_resolves(anchor, headings):
51                     unresolved.append((rule.code, rule.faq_ref))
52             elif file_part in ("VIDEO_RECORDING_GUIDELINES",
53 "VIDEO_RECORDING_GUIDELINES.md",
54 "AUDIO_RALLY_DETECTION_PLAN",
55 "AUDIO_RALLY_DETECTION_PLAN.md"):
56                 # These are dedicated guidance docs surfaced from report rules (Input
57 Quality
58                 # extension). The whole document is the destination; no #Q anchor required.
59                 pass
60             else:
61                 unresolved.append((rule.code, rule.faq_ref))
62         assert not unresolved, f"faq_ref anchors with no matching README heading:
63 {unresolved}"
64
65     def test_actionable_rules_have_faq_refs():
66         """error/warn findings must cite a FAQ (info may omit). Belt-and-suspenders on top
67 of obsreport's own spec-load validation."""
68         spec = load_spec(SPEC)
69         missing = [r.code for r in spec.rules if r.severity in ("error", "warn") and not
70 r.faq_ref]
71         assert not missing, f"actionable rules missing faq_ref: {missing}"
72
73

```

```

# Export command
export_parser = subparsers.add_parser(
    "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
export_parser.add_argument("--out-dir", default="data/eval_export",
                           help="Output directory for per-video CSVs and manifest.json")
export_parser.add_argument("--sport",
                           help="Restrict to a single sport (default: all
supported_sports)")
export_parser.add_argument("--include-staging", action="store_true",
                           help="Also export videos still in staging (default: curated
only)")
export_parser.add_argument("--include-noncommercial", action="store_true",
                           help="Include videos whose license is not commercially safe
(default: commercial only)")

args = parser.parse_args()
cli = SportsDataCLI()

if args.command == "status":
    cli.status()
elif args.command == "curate":
    cli.curate()

```

50. user (2026-06-10T05:39:01.130Z)

```

done
LICENSE
README.md
pyproject.toml
===
generate_khelacharya_flyer.py

```

51. user (2026-06-10T05:39:02.214Z)

```

19:*  **Three Pluggable Rally Segmenters** (`backend/indexer/`):
25:*  **SQLite Indexing**: Each processed video is keyed by MD5 hash. Play segments
(start/end/duration/server/receiver/metadata) and per-segment events are persisted in
`output/sports_indexer.db`. Re-running on the same file is a cache hit.
36:The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract
keyframes, and stitch clips.
142:      "db_name": "sports_indexer.db"
175:1. Create a new python file in `backend/validators/` (e.g. `my_check.py`).
177:3. Register your class inside `backend/validators/__init__.py` using
`registry.register(MyCheckValidator())`.
331: indexer/          # Pluggable rally segmenters: motion, gemini, yolo_hybrid,
tracknet_hybrid

```

52. user (2026-06-10T05:39:15.848Z)

```

97:    was_reingest = db_instance.get_video(video_id) is not None
167:        was_reingest=was_reingest, segmentation_ran=segmentation_ran,
185:    video = db_instance.get_video(req.video_id)
360:        was_reingest = db_instance.get_video(video_id) is not None
412:        was_reingest=was_reingest, segmentation_ran=segmentation_ran,

```

53. user (2026-06-10T05:39:36.149Z)

```

C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\tracknet_hybrid.py:164:                candidate_ids[i] =
self.db.add_candidate_segment(
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\tracknet_hybrid.py:165:                video_id,
detector="tracknet_hybrid",

```

```

C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\wasb_hybrid.py:145:           candidate_ids[i] =
self.db.add_candidate_segment(
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\wasb_hybrid.py:146:           video_id,
detector="wasb_hybrid", start_time=cw["start"], end_time=cw["end"],
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py:466:           candidate_ids[i] =
self.db.add_candidate_segment(
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\segmenters\yolo_hybrid.py:467:           video_id,
detector="yolo_hybrid",

```

54. user (2026-06-10T05:40:00.114Z)

```

rc=1
31:class WasbConfig:
32-     """Resolved settings for a WASB WSL run (built from config.json -> indexing.wasb)."""
33-     distro: str = "Ubuntu"
34-     repo_dir: str = "~/models/WASB-SBDT"
35-     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36-     conda_env: str = "wasb"
37-     weights_path: str = "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38-     sport: str = "badminton"
39-     wsl_stage_dir: str = "~/clips"
40-     timeout_sec: int = 0 # 0 = no timeout
41-
42-     @classmethod
43-     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
44-         w = (idx_cfg or {}).get("wasb", {}) or {}
45-         return cls(
46-             distro=w.get("wsl_distro", "Ubuntu"),
47-             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
48-         )
49-
50-
fastapi>=0.111.0
uvicorn>=0.30.1
opencv-python>=4.9.0.80
numpy>=1.26.4
pydantic>=2.7.1
jinja2>=3.1.4
python-dotenv>=1.0.0
google-genai>=2.4.0
torch>=2.12.0
torchvision>=0.27.0
supervision>=0.21.0,<0.30.0

```

Per-video observability reports (SOFT dependency: the pipeline degrades to a no-op if absent). Shared, permissively-licensed report toolkit. Pin a tag once the private repo is published, e.g.:

obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3

For local development install it editable from a sibling checkout:

pip install -e ../sports-obsreport --no-build-isolation

55. user (2026-06-10T05:40:10.980Z)

```

rc=0
docs/next-steps-multisport
26ee2de docs: next-steps + multisport findings (WASB transfers to table tennis, not pickleball)
4d04a73 Quality iteration: human-GT multi-video LOVO + learned prototype + multi-court finding +
owned-model study/plan (#61)

```

90fb8b3 Human-GT partial-label eval harness + tuning + iteration doc (#60)
3fd7232 docs: handoff-consistency sweep + strategy checkpoint + CLAUDE.md (#59)
7609593 Merge pull request #58 from avidullu/claude/platform-architecture-research

56. user (2026-06-10T05:40:41.605Z)

```
44:TUNED: Config = (0.05, 1.0, 1.0)
47:def restrict_to_window(intervals: List[Interval], w0: float, w1: float) -> List[Interval]:
97:     ("tuned", TUNED, 0),
98:     (f"tuned+gate{gate_crossings}", TUNED, gate_crossings),
134:DEFAULT_GRID = [
142:def sweep(trajjectory_csv: str, labels_csv: str, fps: float, frame_width: float,
152:     grid = grid or DEFAULT_GRID
===
42:FEAT_NAMES = ["vis_rate", "spd_mean", "spd_max", "cross_rate", "x_std", "y_std"]
89:     n = uniform_filter1d(vis, size, mode="constant") # mean visibility
(== vis_rate)
91:     spd_mean = uniform_filter1d(speed, size, mode="constant") / np.maximum(sm, 1e-9)
92:     spd_max = maximum_filter1d(speed, size, mode="constant")
93:     cross_rate = uniform_filter1d(cross, size, mode="constant") * fps # crossings /
second
96:     x_std = np.sqrt(np.maximum(x2 - xm ** 2, 0.0)) / fw
99:     y_std = np.sqrt(np.maximum(y2 - ym ** 2, 0.0)) / fw
102:     X = np.stack([n[idx], spd_mean[idx], spd_max[idx], cross_rate[idx], x_std[idx],
y_std[idx]], axis=1)
===
39:     Keeps `AutomatedProvider()` constructible (so the registry can key it as 'auto')
45:     "annotation_registry.get('auto', labeler=<fn>, lineage='<model-family>')."
82:     return "auto"
===
261:### Q14: The report says fps was a "fallback" (not probed) ? what does that mean?
262:The `Video` section shows `FPS | 30 (fallback)` and a `fps_fallback_used` warning when the
pipeline could **not probe** the real frame rate from the file and fell back to a hardcoded
default (30 fps). Because rally detection converts frames?seconds and scales motion thresholds
by fps, a wrong fps **miscalibrates timing and motion sensitivity** ? segment boundaries and
which motion counts as "play" can be off.
```

57. user (2026-06-10T05:40:53.517Z)

```
def init_default_config():
    """Initialize a default config.json if it doesn't exist.

    Now delegates to the canonical backend.config loader so that defaults
    stay in sync with the Pydantic models (single source of truth).
    """
    try:
        from backend.config import save_default_config
    except Exception as e:
        print(f"[ERROR] Could not import backend.config loader: {e}")
        print("[INFO] Falling back to no-op for config initialization.")
        return

    config_path = "config.json"
    if os.path.exists(config_path):
        print(f"[INFO] {config_path} already exists. Skipping initialization.")
        return

    try:
        saved_path = save_default_config(config_path)
        print(f"[SUCCESS] config.json initialized with default settings at {saved_path}.")
    except Exception as e:
        print(f"[ERROR] Failed to create config.json via loader: {e}")

    """
69:     api_key_env_var = f"{provider_name.upper()}_API_KEY"
```

```

===
13:110:- `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`wasb_hybrid`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE
default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config
`indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs
`{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike
tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit
(copy-paste drift).
15:122:- `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner)
(+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default
`wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::_INFER_WIN`) +
video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` =
`staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-
named `_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until
staged); `_wsl_bash` duplicated (drift risk). Both runners now satisfy the DetectorRunner ABC
for future swappability.
===
rc=0

```

58. user (2026-06-10T05:41:16.450Z)

```

36:_DETECTOR_FACTORIES = {
37-     "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
38-     "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
39-     "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
40-}
41:_DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
42-
43-
44-def _fmt_ts(seconds: float) -> str:
45-    --
82:         if model_name not in _DETECTOR_FACTORIES:
83-             logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
===
4d04a73 Quality iteration: human-GT multi-video LOVO + learned prototype + multi-court finding +
owned-model study/plan (#61)
06963a0 docs(quality): n=5 + MULTI-COURT root cause; CODE_MAP wrap
d965845 docs: retire footguns fixed by this change (CODE_MAP + BACKLOG)
===
36:     .env file in the project root and re-run (see README#Q7).
37:     faq_ref: "README#Q7"
55:     faq_ref: "README#Q7"
68:     faq_ref: "README#Q14"
102:    faq_ref: "README#Q10"
116:    faq_ref: "README#Q7"
128:    faq_ref: "README#Q11"
140:    faq_ref: "README#Q13"

```

59. user (2026-06-10T05:41:26.992Z)

```

src/cli/curate.py:17:from src.core.schemas.pickleball import PickleballRally
src/cli/curate.py:97:         elif sport_str == "pickleball":
src/cli/curate.py:98:             is_valid, errors =
self.validator.validate_pickleball_rallies(annotations)
src/cli/curate.py:133:         elif sport_str == "pickleball":
src/cli/curate.py:134:             count = self.db.insert_pickleball_rallies(annotations)
src/cli/curate.py:208:         elif sport == "pickleball":
===

```

124 tests collected in 0.46s

60. user (2026-06-10T05:41:43.793Z)

```
from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
from backend.pipeline.segmenters.motion import MotionPlaySegmenter
from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter

__all__ = [
    "segmenter_registry",
    "BasePlaySegmenter",
    "MotionPlaySegmenter",
    "GeminiPlaySegmenter",
    "HybridYoloSegmenter",
    "TrackNetHybridSegmenter",
    "WasbHybridSegmenter",
]
===
3:## ADR-001: Precise shuttle tracking via self-hosted TrackNetV4/WASB on WSL2
55:## ADR-002: Use the sports-data-collector's rally annotations as a golden eval + temporal-
segmenter training set
71: (TrackNetV4/WASB ? see ADR-001), which needs coordinate labels.
93: file recorded in the manifest; acquisition (ADR-003) therefore precedes processing.
105:## ADR-003: Video acquisition ? always best resolution, upgrade in place, authenticated
cookies
150:## ADR-004: Add an offline, API-free *learned* rally segmenter as a first-class mode
159:rally-annotated corpus (ADR-002/003), we can learn the refinement step offline instead.
174: fully offline but heaviest (needs the WSL2 WASB runner, ADR-001); can't train WASB
218:shuttle signal), the offline segmenter's substrate is **WASB shuttle-motion** (ADR-001),
232:## ADR-005: Permissive Dependency Stack for the Hosted Monetization Path
```

61. user (2026-06-10T05:41:54.079Z)

```
258:## ADR-006: Golden-Set Generation via Vendor Bake-off + a Pluggable Annotation Provider
295:## ADR-007: Audio "thwack" hit-detection (fused with WASB) is the next rally-detection cue;
pose/stance parked
===
docs/TRACKNET_WSL_SETUP.md:4:to the Windows indexer via a subprocess adapter. See
`docs/DECISIONS.md` (ADR-001)
docs/GOLDEN_SET_VENDORING.md:8:> **Companion docs:** the scoring mechanics live in
[EVALUATION.md](EVALUATION.md); the annotated-corpus rationale is [ADR-002](DECISIONS.md); the
decision record for *this* process is **ADR-006**. Video-capture quality requirements (a
dependency of trustworthy golden sets) are tracked separately in
[VIDEO_RECORDING_GUIDELINES.md](VIDEO_RECORDING_GUIDELINES.md).
README.md:7:> **Where this is heading:** the project is evolving from an AI-API-dependent
prototype into a self-improving tool that learns rally detection *offline* from a commercially-
licensed, annotated corpus. See **[docs/ROADMAP.md](docs/ROADMAP.md)** for the four-phase plan
and **[docs/archive/decisions/DECISIONS.md](docs/archive/decisions/DECISIONS.md)** (historical
ADRs) for the rationale.
```

62. user (2026-06-10T05:42:07.053Z)

```
38
tests/test_annotation_providers.py
tests/test_automated_provider.py
tests/test_base.py
tests/test_calibrate_local.py
tests/test_calibrate_wasb.py
tests/test_calibration.py
tests/test_config.py
tests/test_database.py
tests/test_distill_local.py
tests/test_eval_annotations.py
tests/test_eval_batch.py
```



```
tests/test_eval_classifier.py
tests/test_eval_harness.py
tests/test_eval_metrics.py
tests/test_eval_partial_labels.py
tests/test_eval_regression.py
tests/test_eval_training.py
tests/test_export_wasb_segments.py
tests/test_gemini.py
tests/test_gemini_refine.py
tests/test_geometry.py
tests/test_hashing.py
tests/test_hit_detector.py
tests/test_provider_gemini.py
tests/test_providers.py
tests/test_rally_gate.py
tests/test_rally_seq_proto.py
tests/test_rally_slicer.py
tests/test_reporting.py
tests/test_reporting_harvest.py
tests/test_reporting_spec_faq.py
tests/test_sport_profiles.py
tests/test_tracknet_hybrid.py
tests/test_tracknet_runner.py
tests/test_validators_base.py
tests/test_video.py
tests/test_wasb_cache.py
tests/test_yolo_hybrid.py
```

63. user (2026-06-10T05:42:23.771Z)

```
193:         with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
302:             max_segment_duration = 90.0 # Singles play segments almost never exceed this
344:             if dur > max_segment_duration:
346:                 f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
60:         # A: Check for debug_duration to trim the video first
61:         debug_duration = self.config.get("indexing", {}).get("debug_duration")
68:         if debug_duration:
```

64. assistant (2026-06-10T05:43:04.904Z)

All verification complete. Compiling the structured findings.

65. user (2026-06-10T05:44:32.156Z)

Structured output provided successfully

66. assistant (2026-06-10T05:44:32.995Z)

You've hit your session limit · resets 3:50pm (Asia/Calcutta)